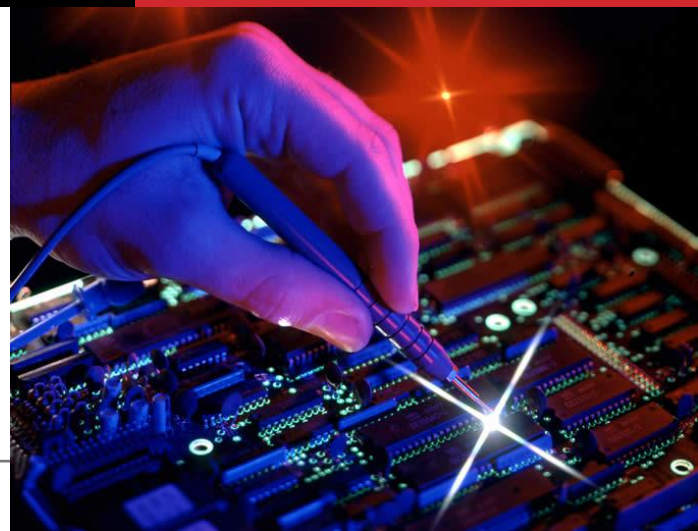


VLSI

VIVADO DESIGN EXAMPLE

Dennis A. N. Gookyi



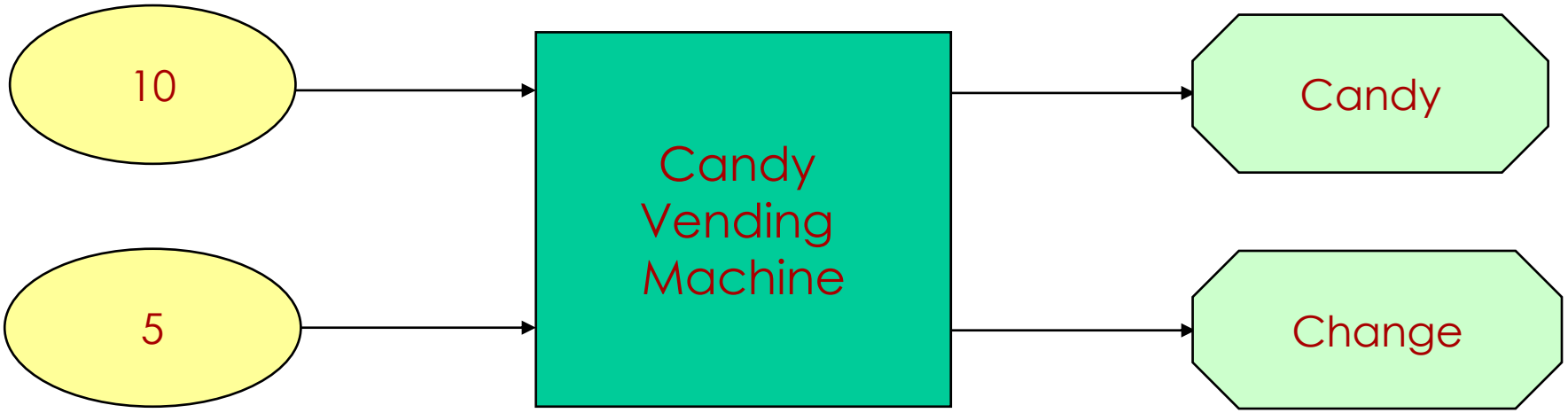
CONTENTS

- **VIVADO Design Example**
  - Candy Vending Machine
    - Finite State Machine (Candy Vending Machine)
    - Design Specifications
    - Input/output Specifications
    - Mealy-type State Machine
    - Candy Vending Machine Project Design Using Vivado



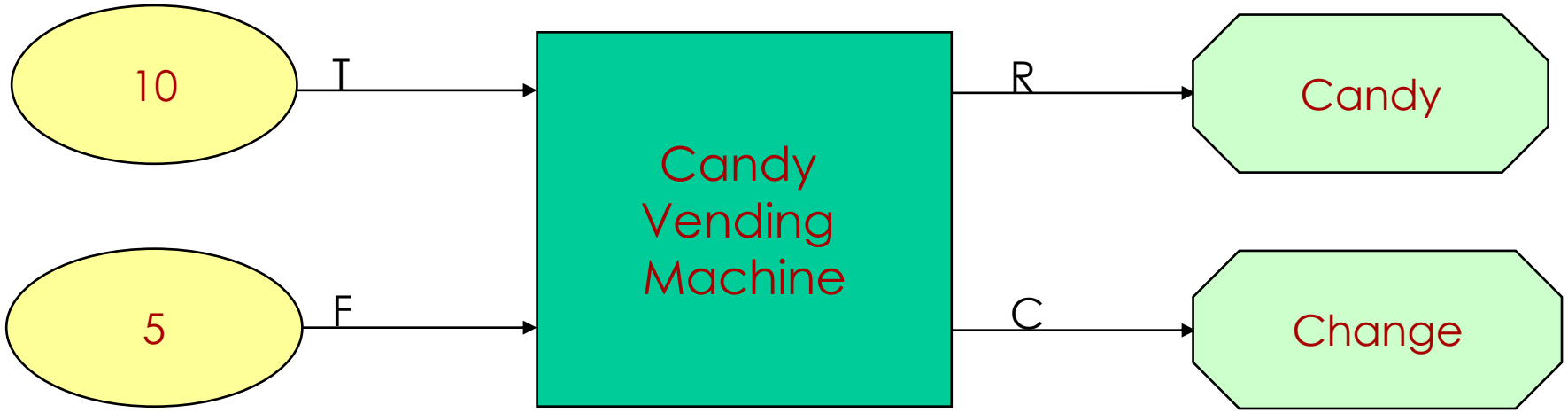
DESIGN EXAMPLE

- Design Specification: Candy Vending Machine
  - One candy cost 20
  - Only 10 and 5 coins are accepted
  - When money exceeding 20 comes in, candy and change are released
    - The money received cannot exceed 25 won



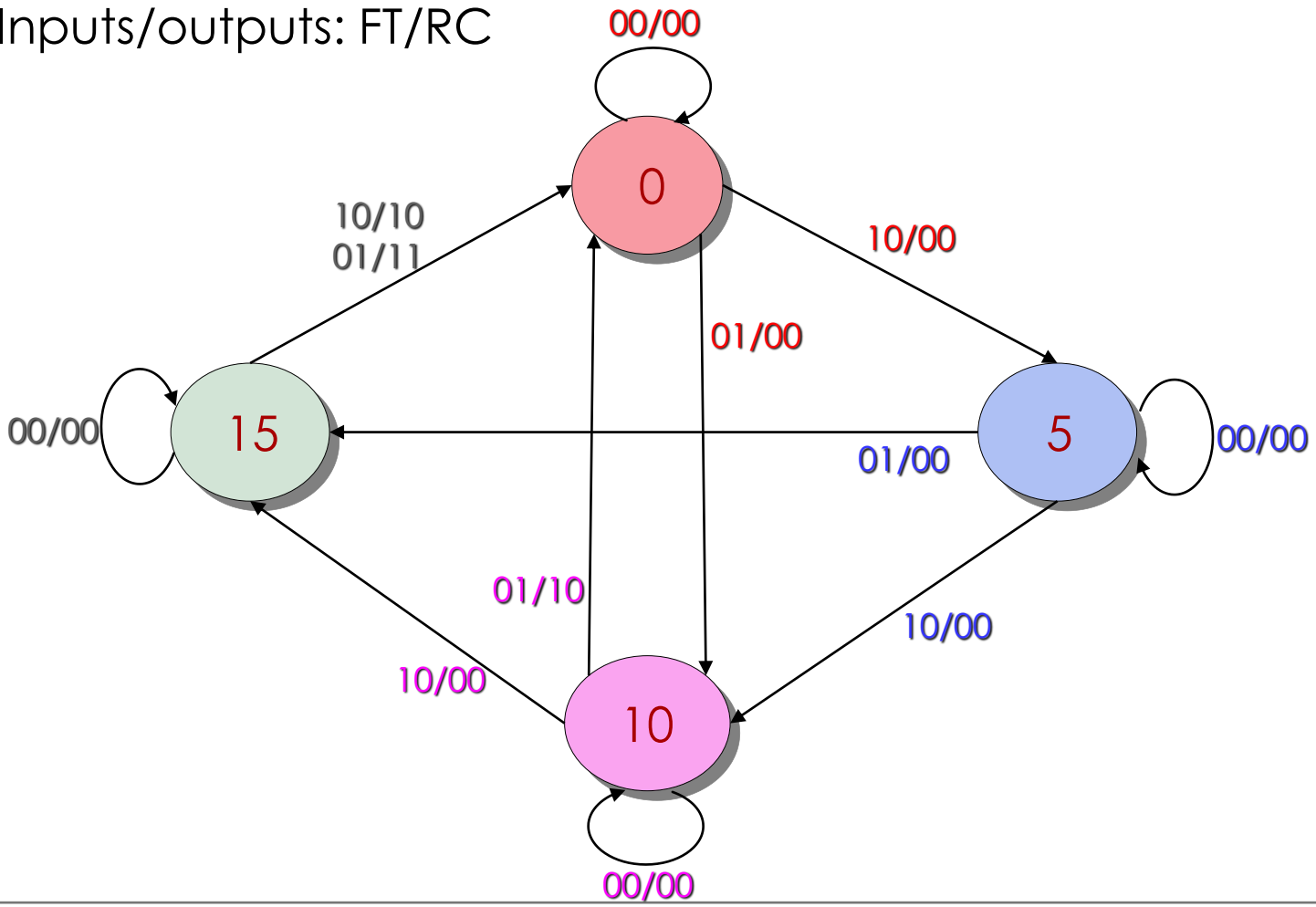
DESIGN EXAMPLE

- Input/Output Specifications
  - Inputs
    - T (Ten): Enter 10 coin
    - F (Five): Enter 5 coin
  - Outputs
    - R (Release): Candy release
    - C (Change): Change release



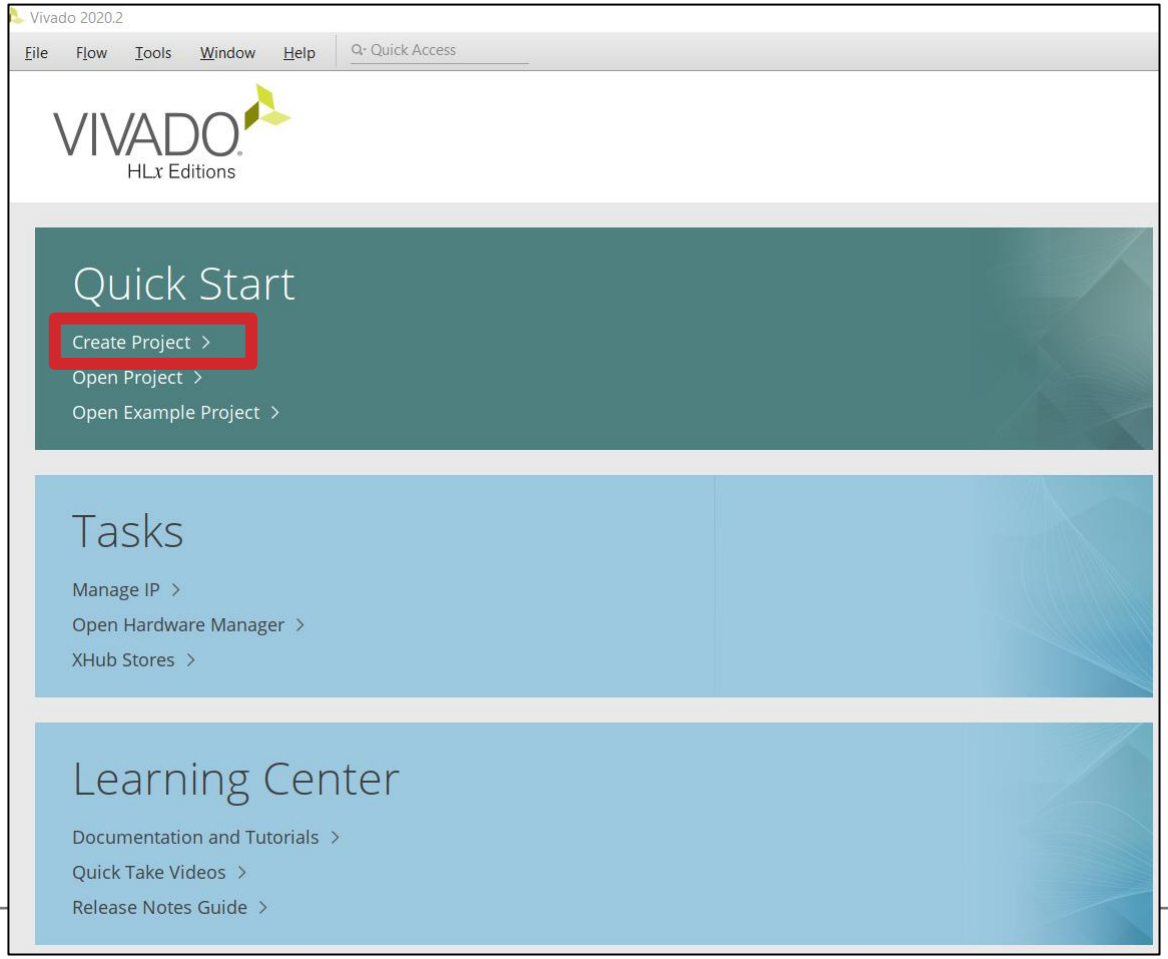
DESIGN EXAMPLE

- Mealy-type State Machine
  - Inputs/outputs: FT/RC



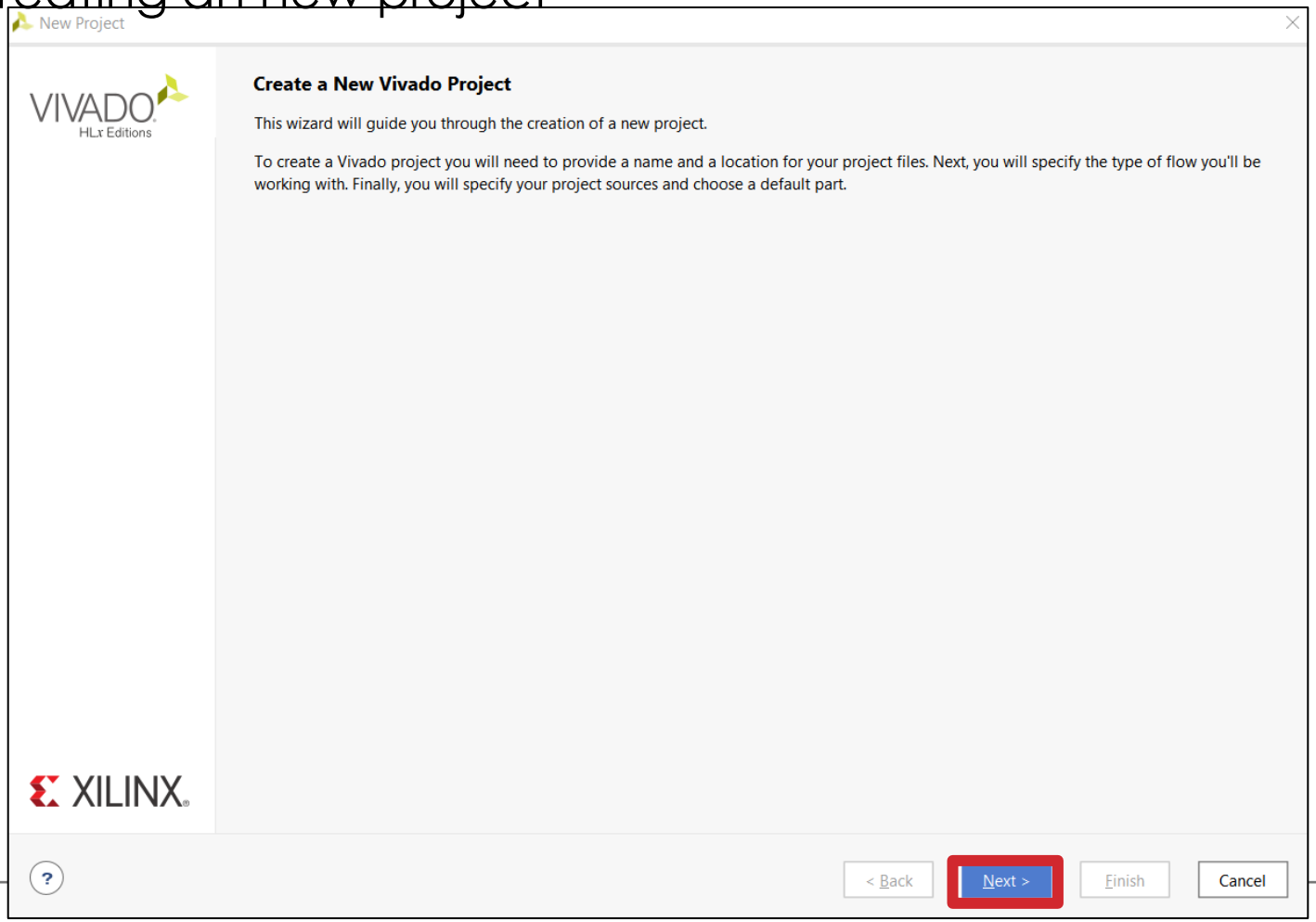
DESIGN EXAMPLE

- Vivado project
  - Creating a new project



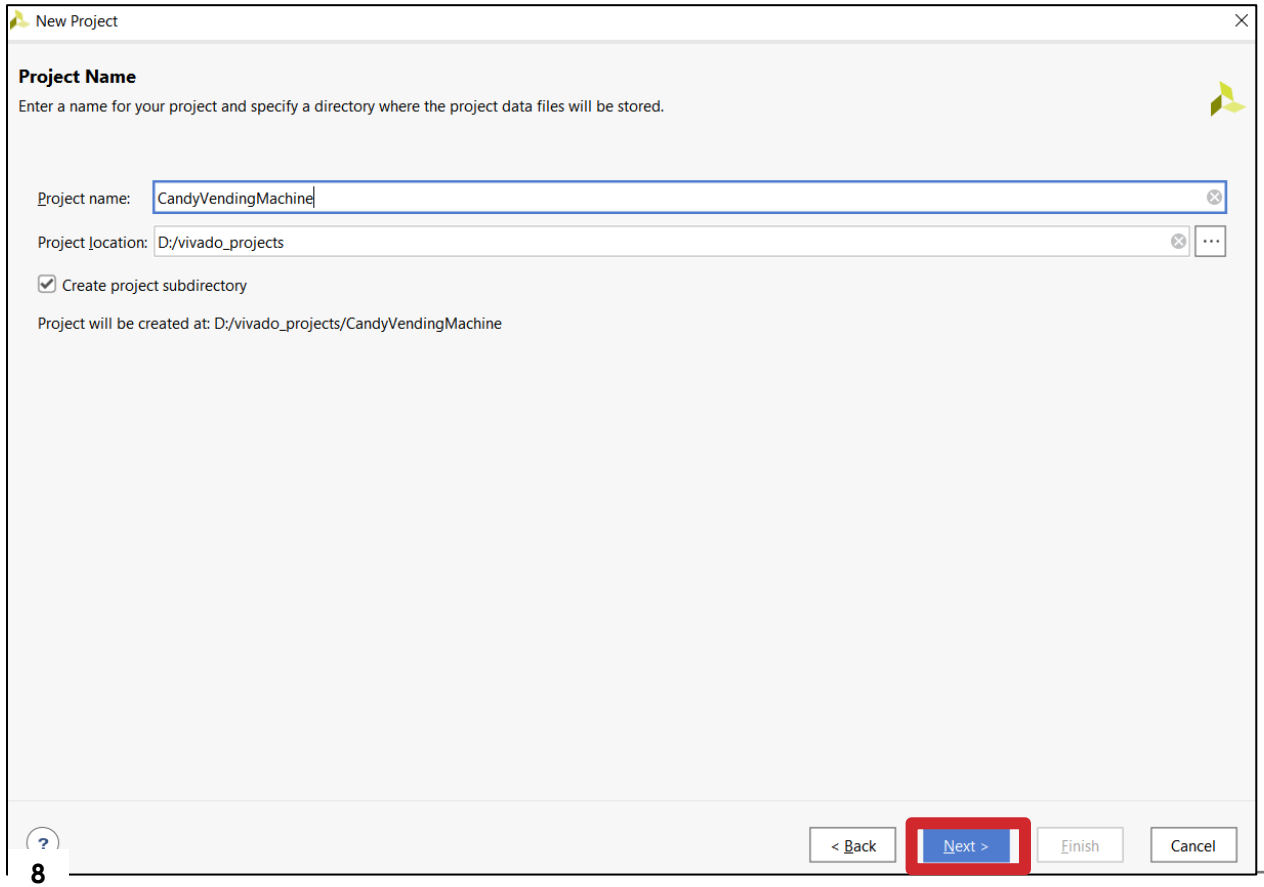
DESIGN EXAMPLE

- Vivado project
  - Creating an new project



DESIGN EXAMPLE

- Vivado project
  - Creating a new project

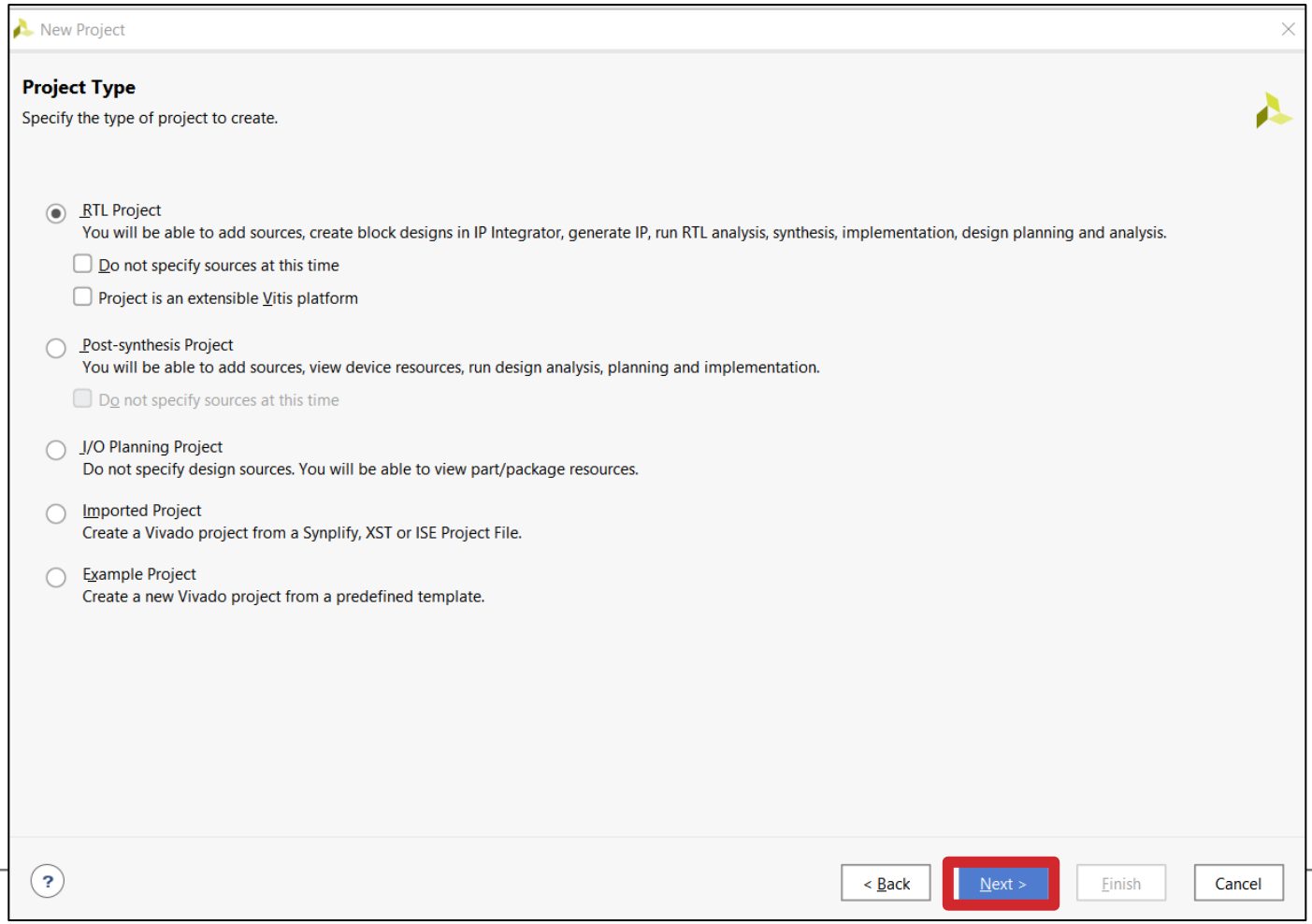


Project name
→ must start with an alphabet
→ can include "_" and numbers

Location
→location of the project
→subfolders can be created

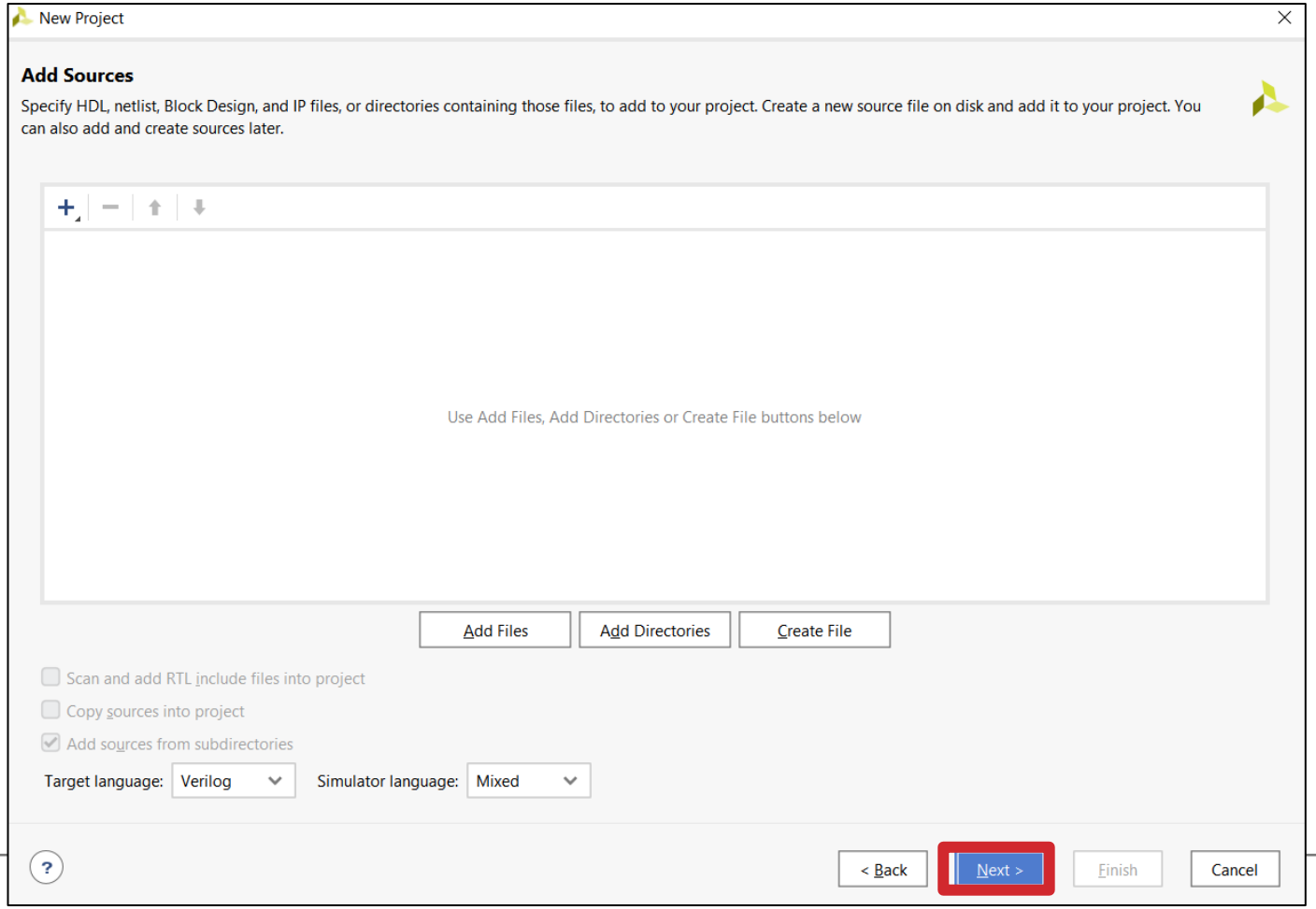
DESIGN EXAMPLE

- Vivado project
  - Creating an new project



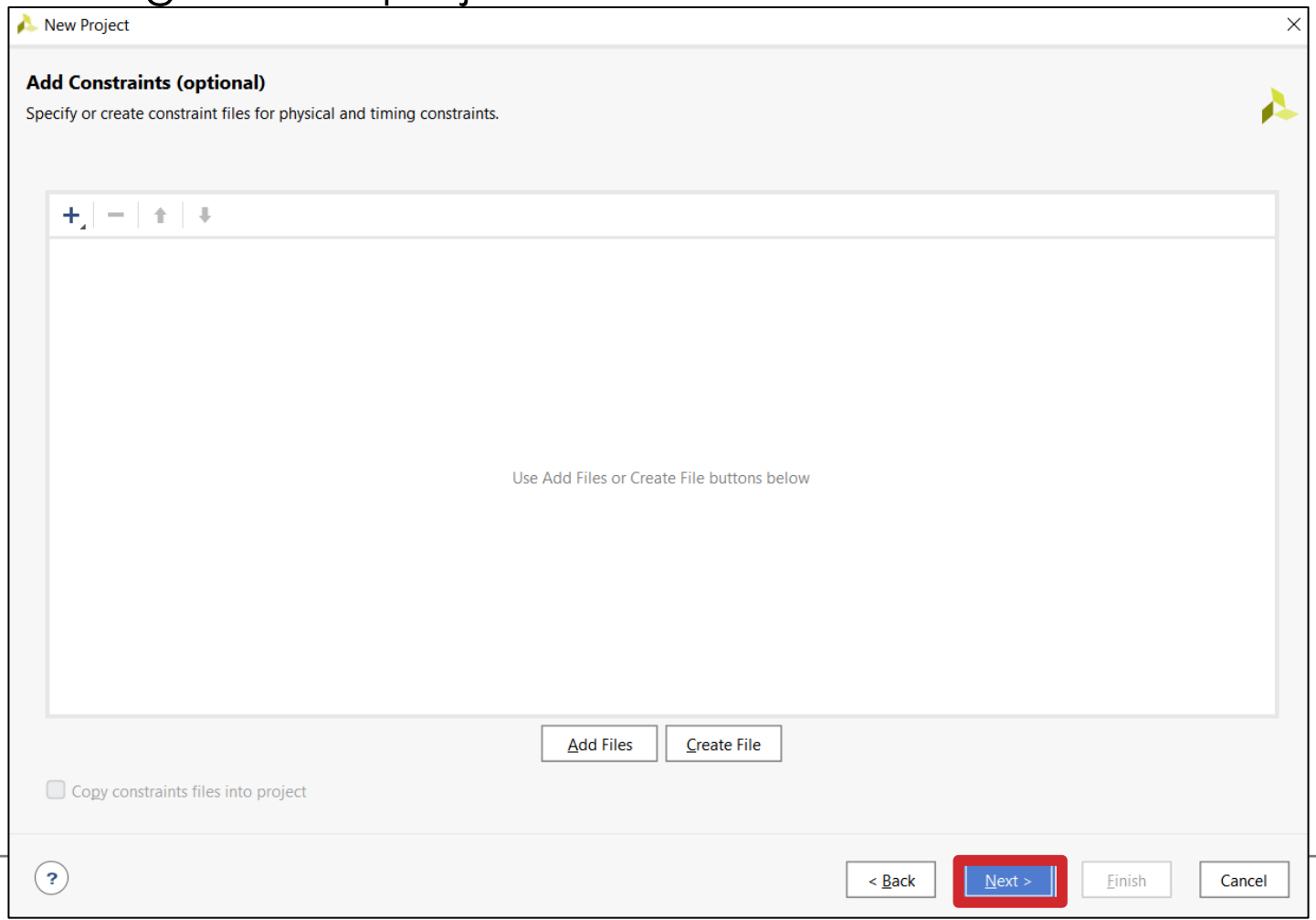
DESIGN EXAMPLE

- Vivado project
  - Creating an new project



DESIGN EXAMPLE

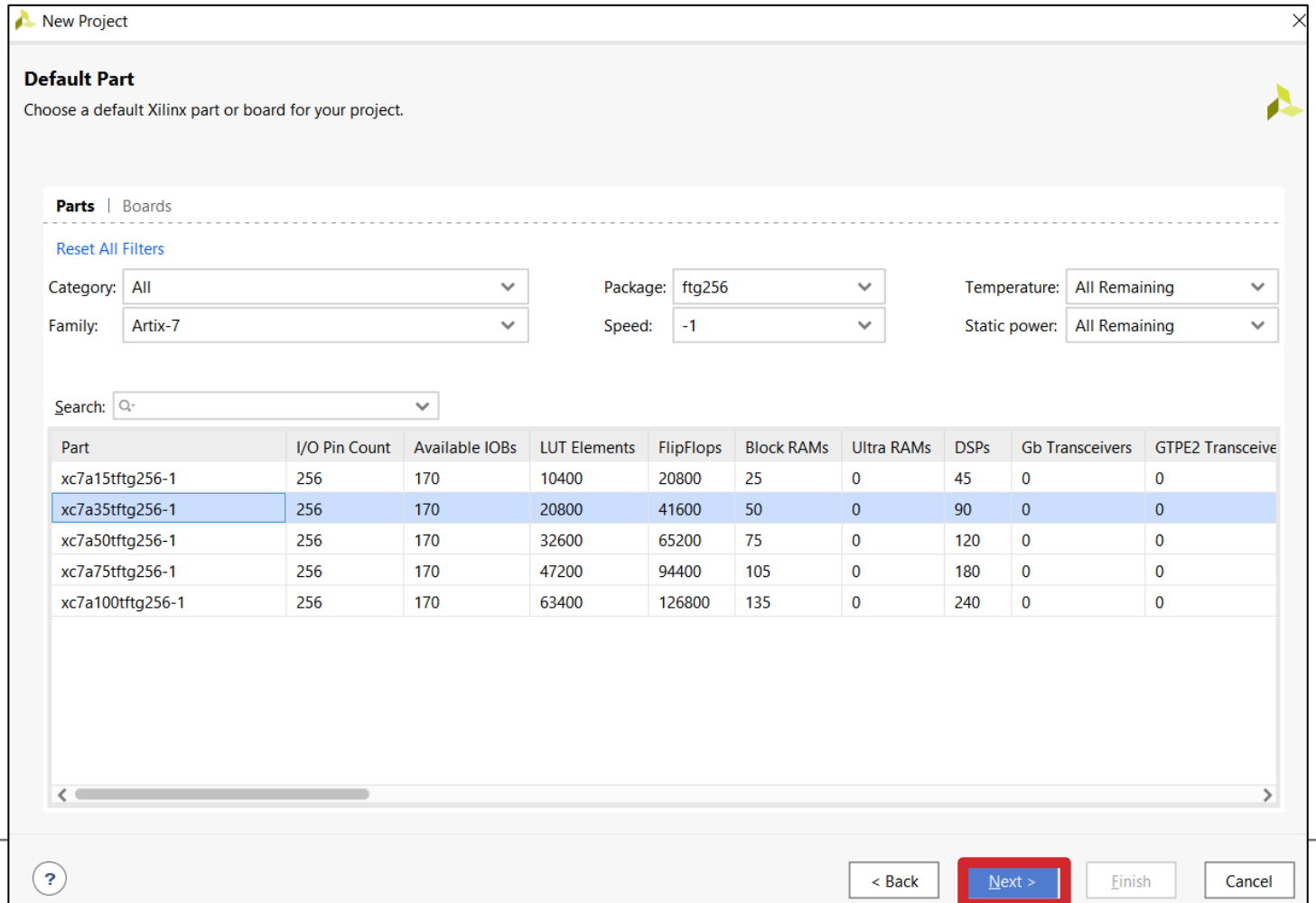
- Vivado project
  - Creating an new project



DESIGN EXAMPLE

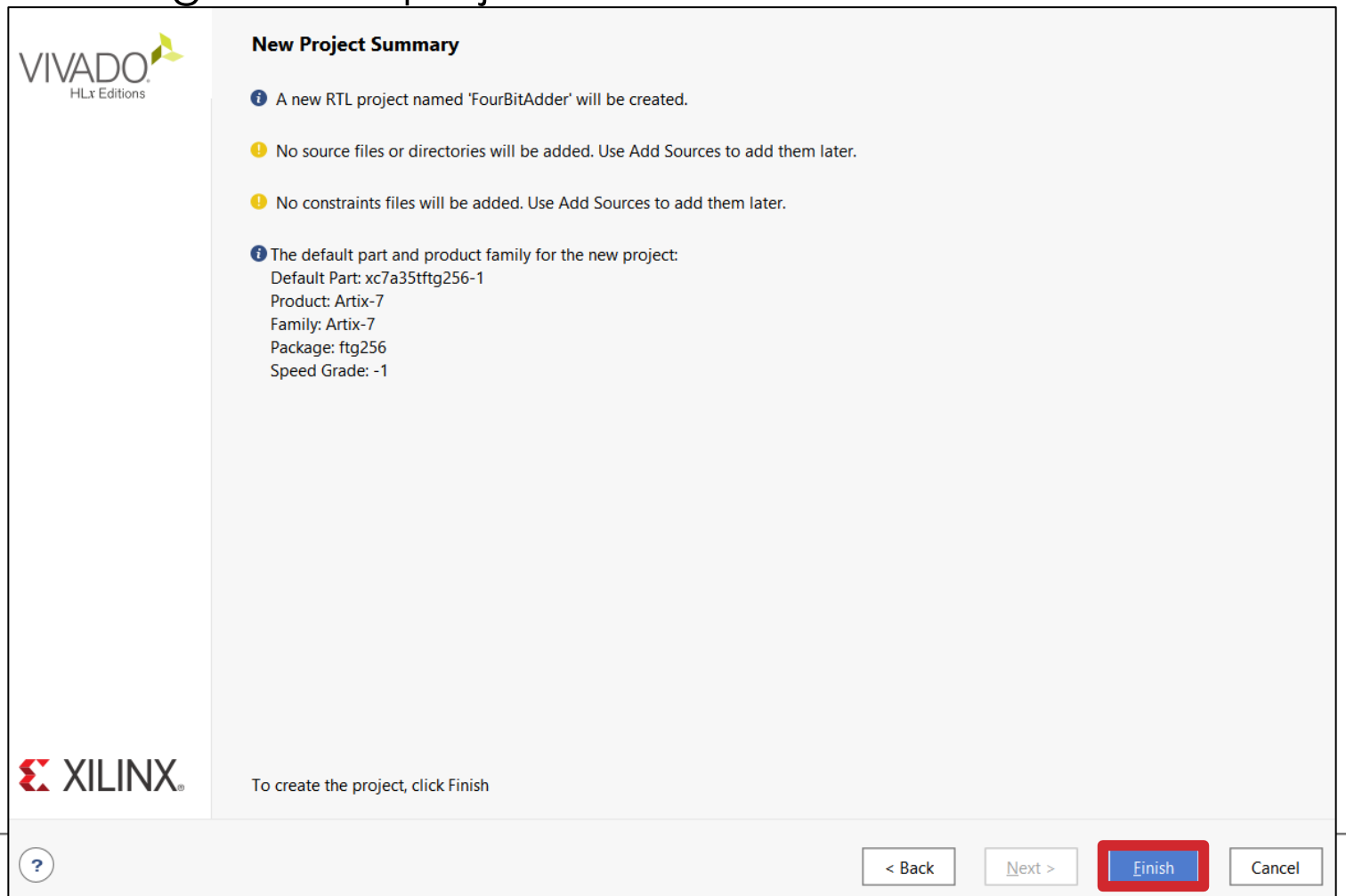
- Vivado project
  - Creating an new project

FPGA Device: Xc7a35tftg256-1



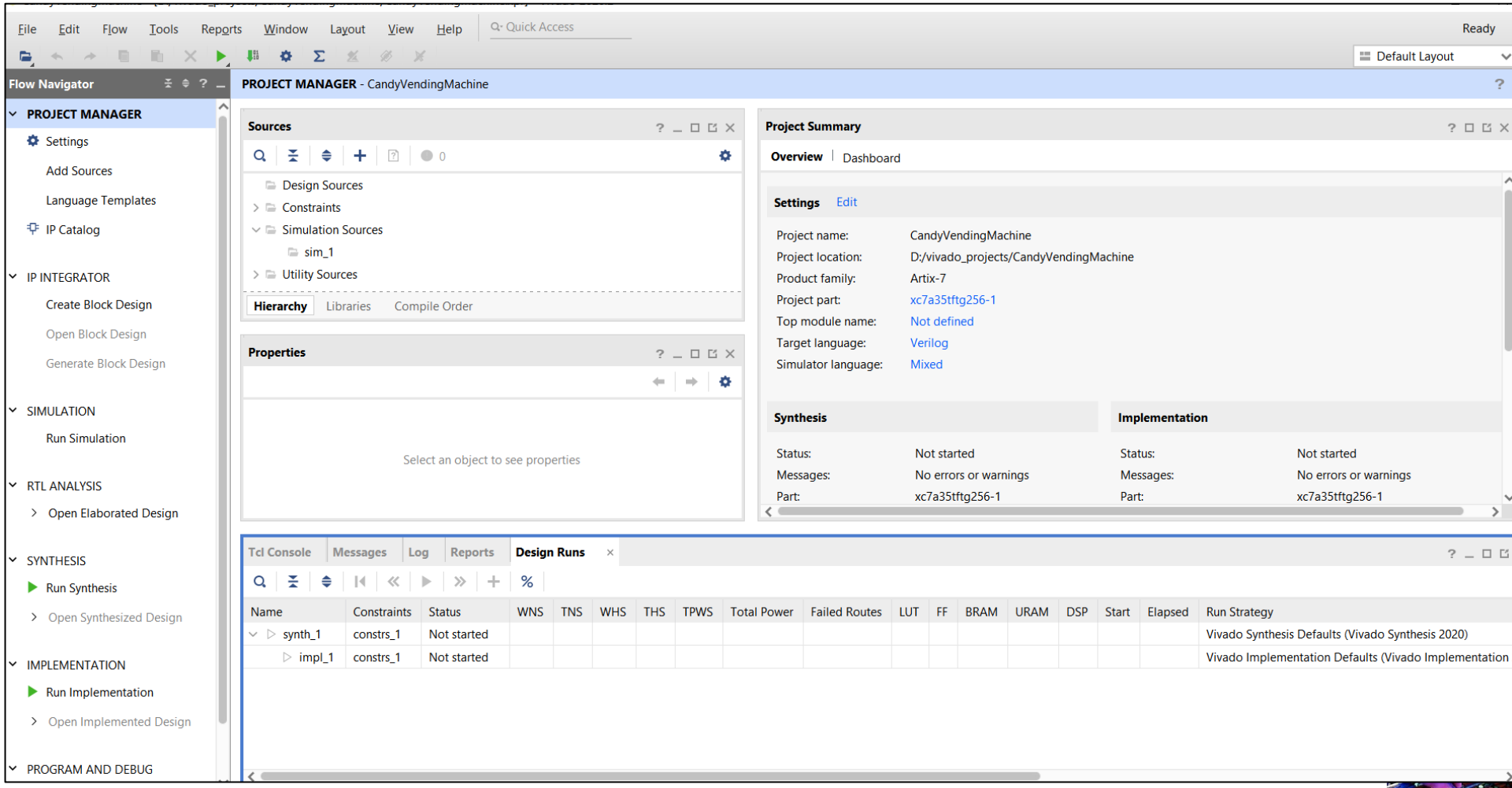
DESIGN EXAMPLE

- Vivado project
  - Creating an new project



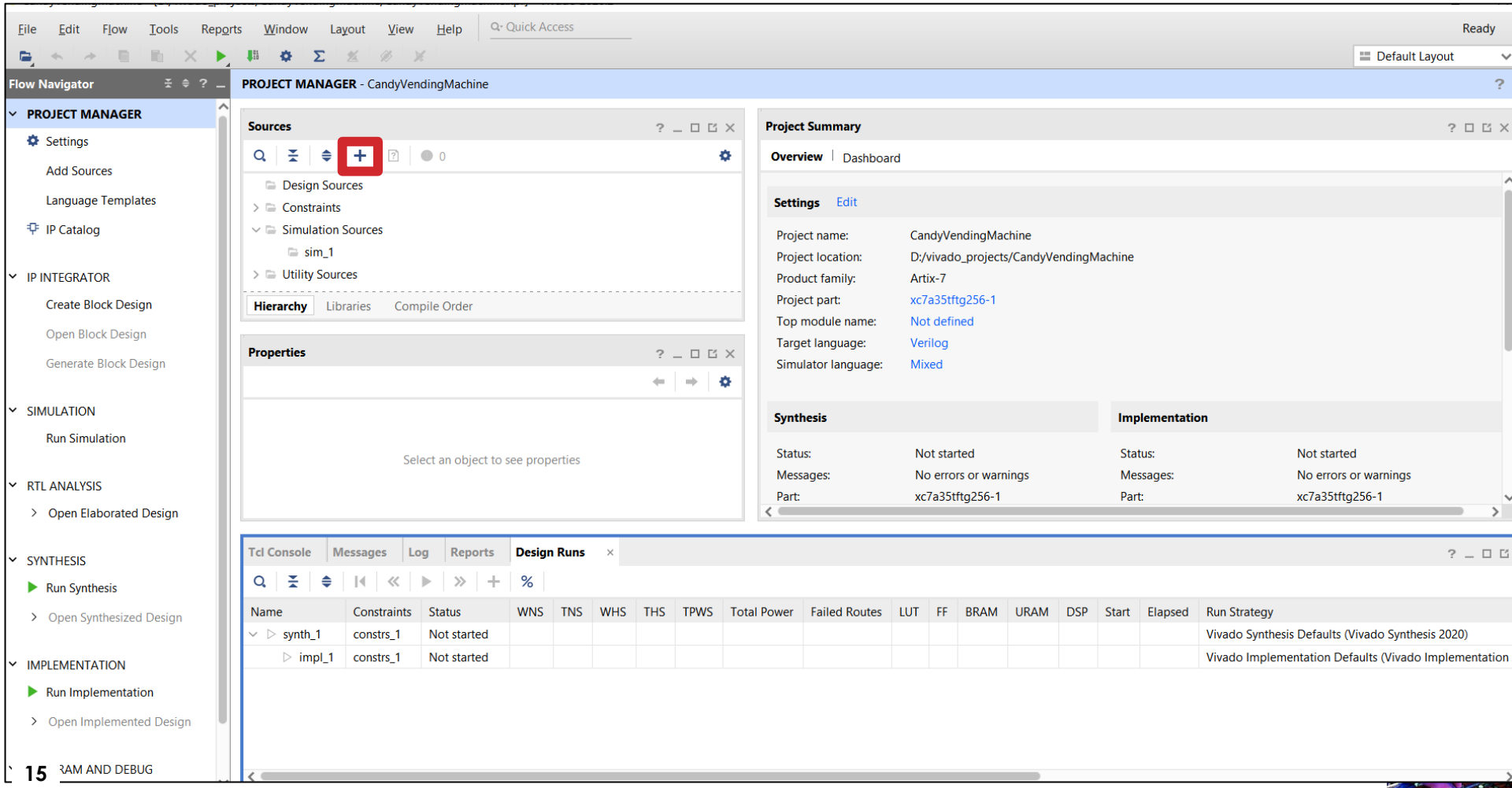
DESIGN EXAMPLE

- Vivado project



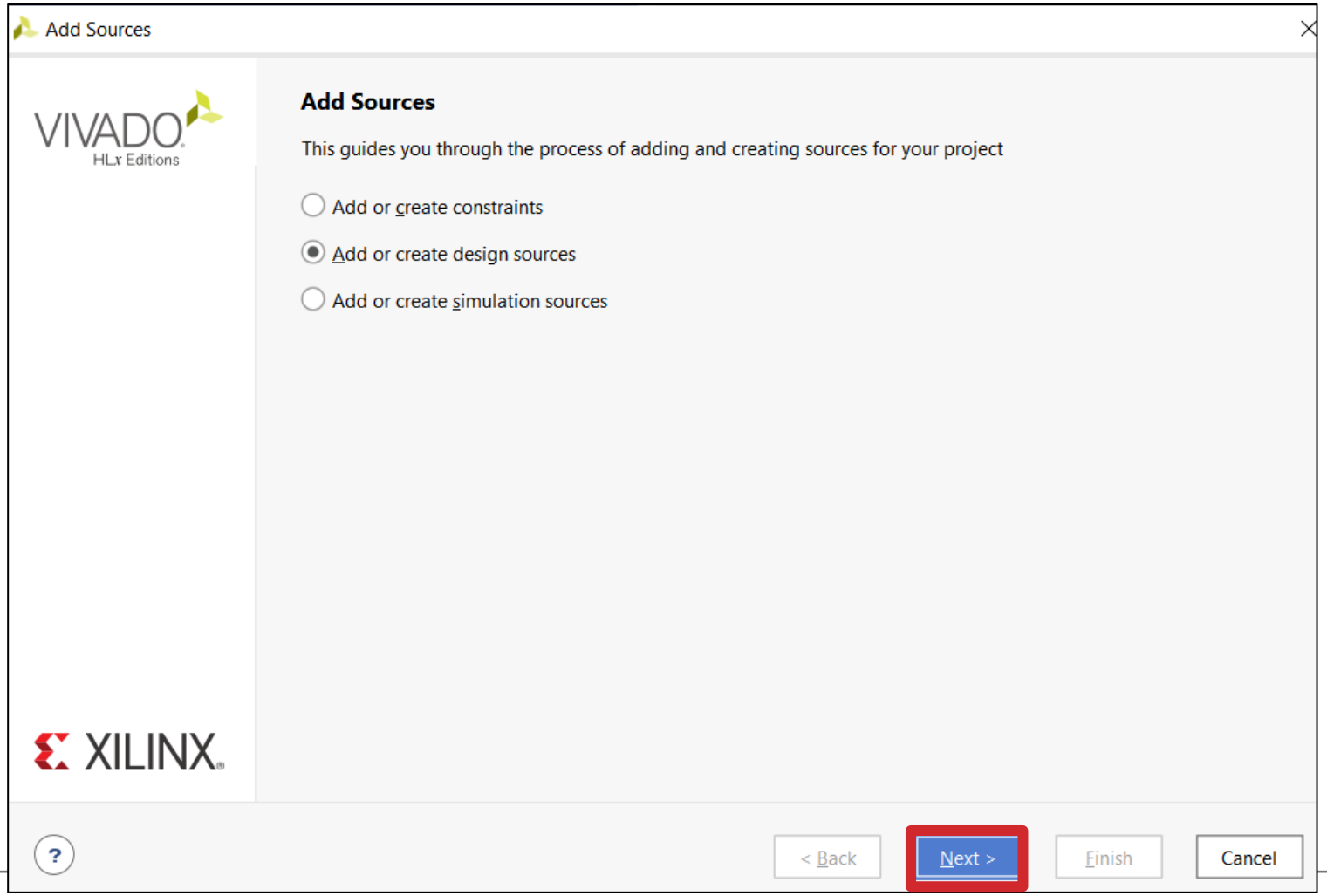
DESIGN EXAMPLE

- Vivado project



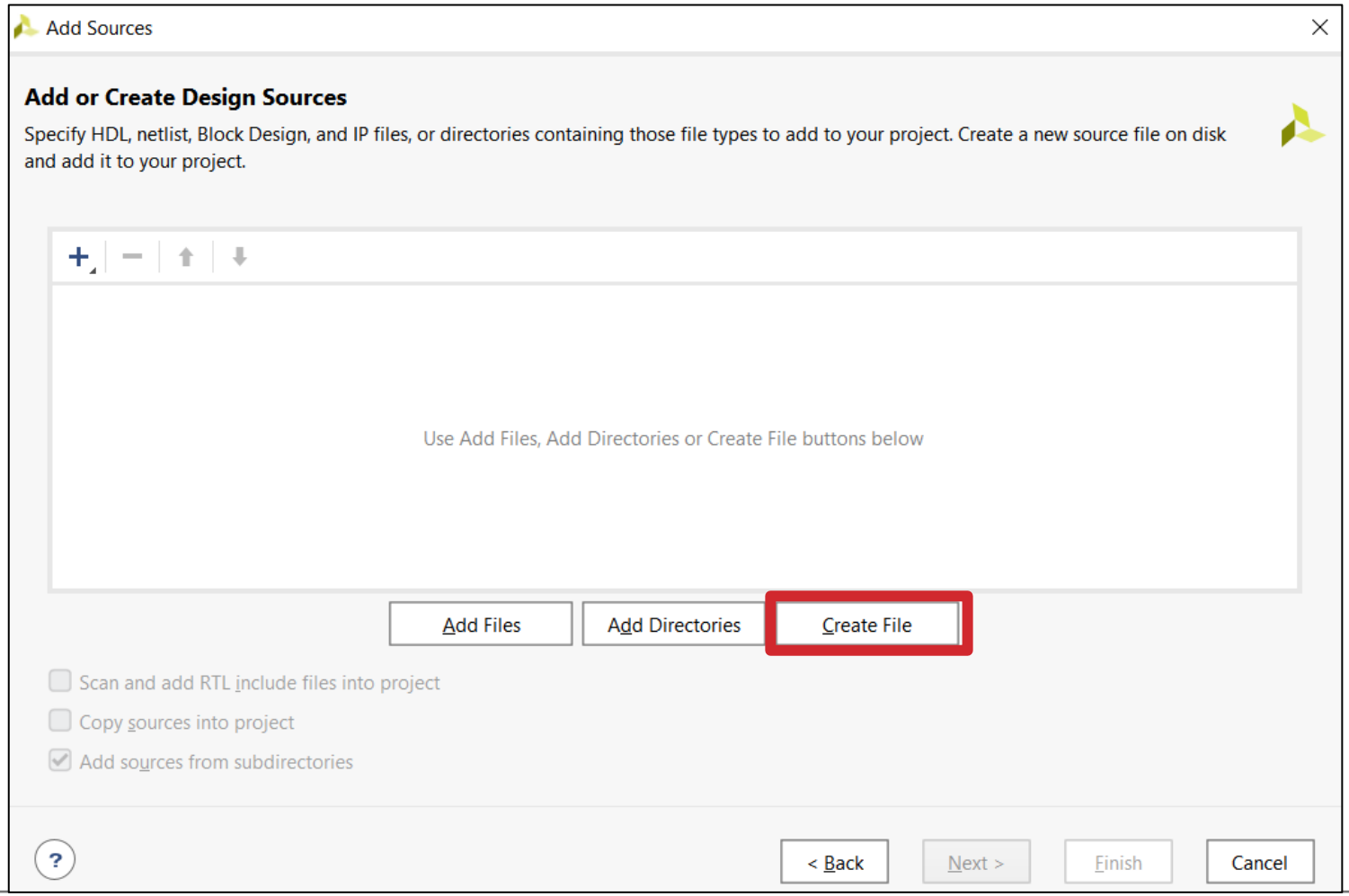
DESIGN EXAMPLE

- Vivado project



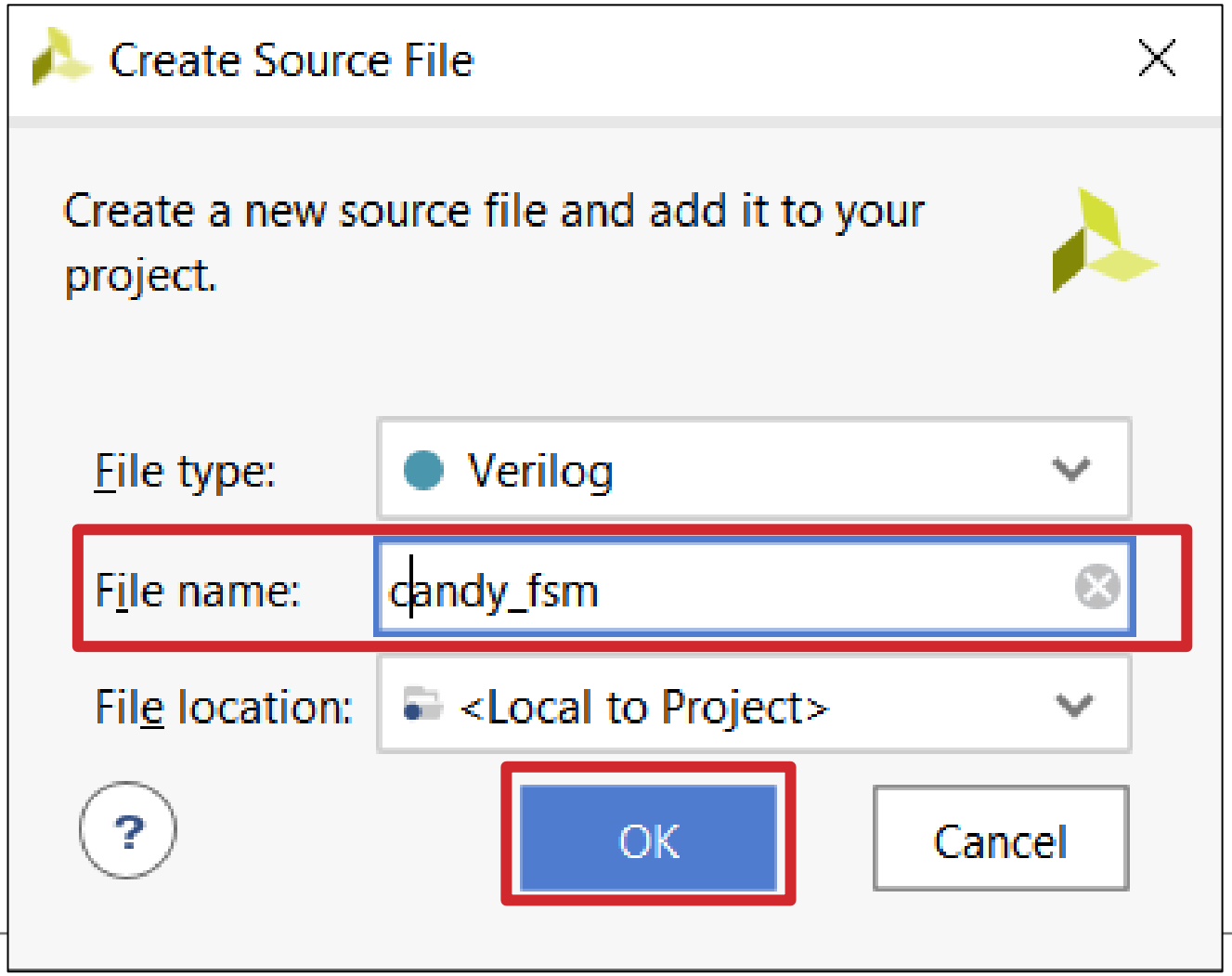
DESIGN EXAMPLE

- Vivado project



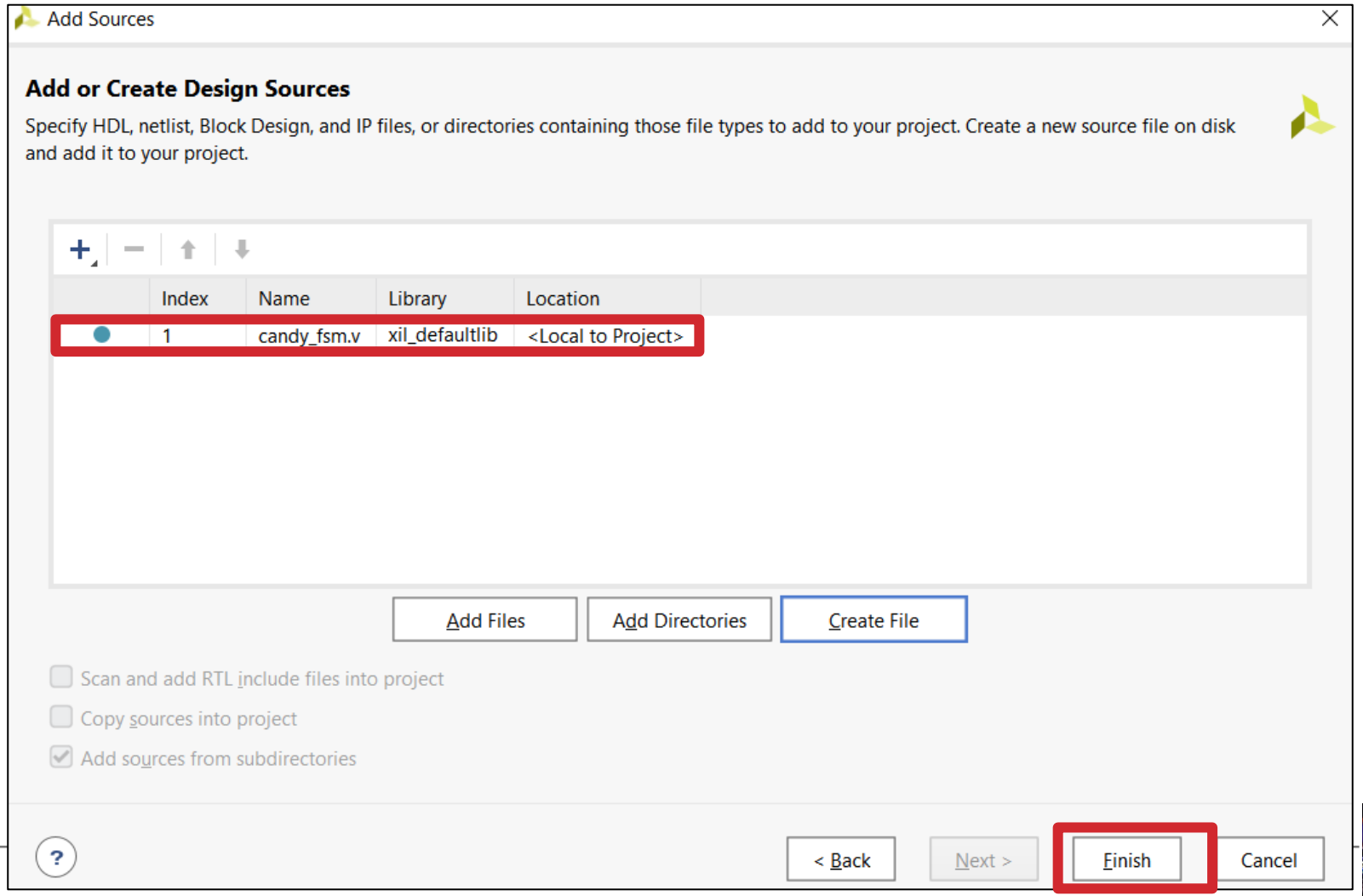
DESIGN EXAMPLE

- Vivado project



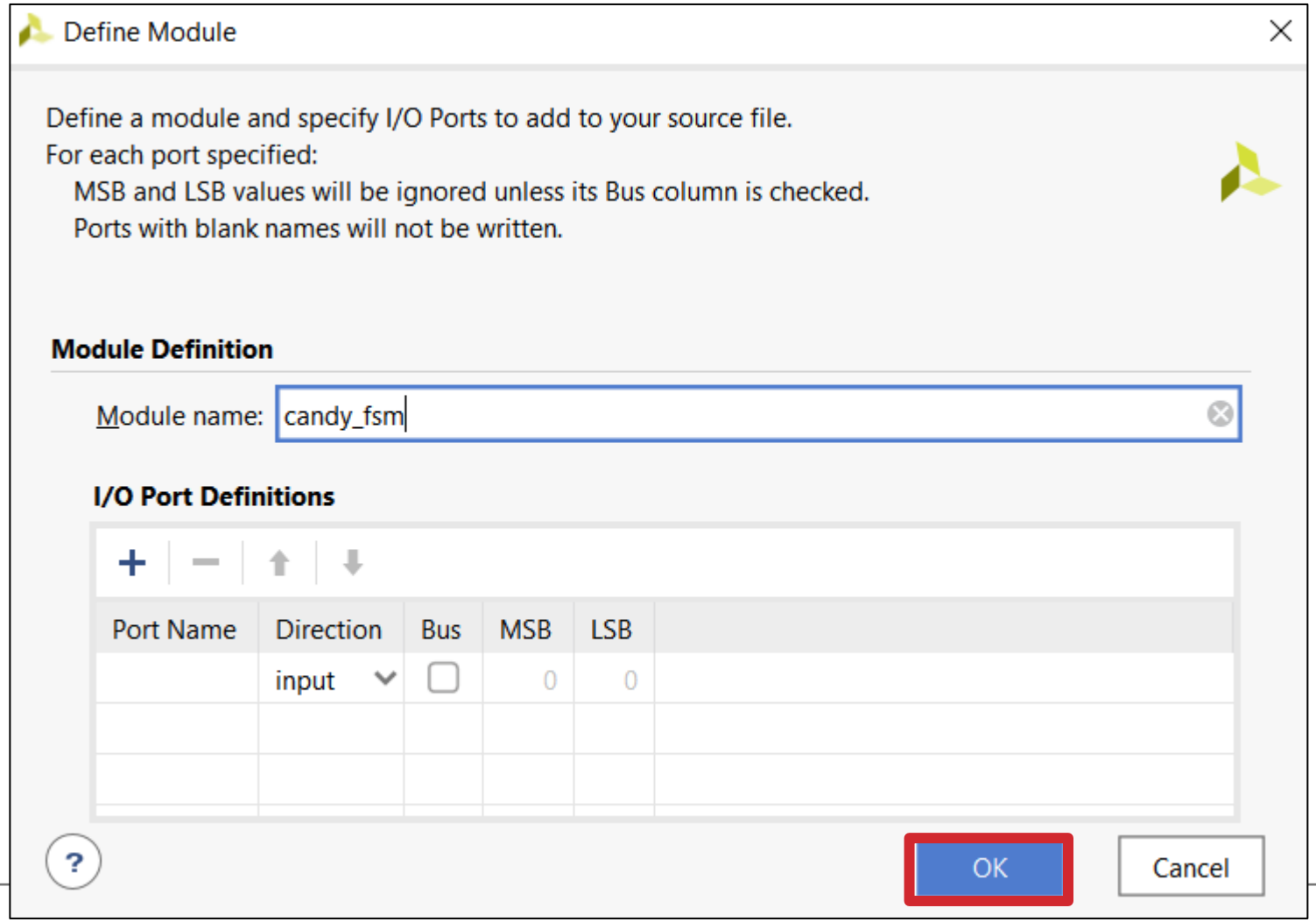
DESIGN EXAMPLE

- Vivado project



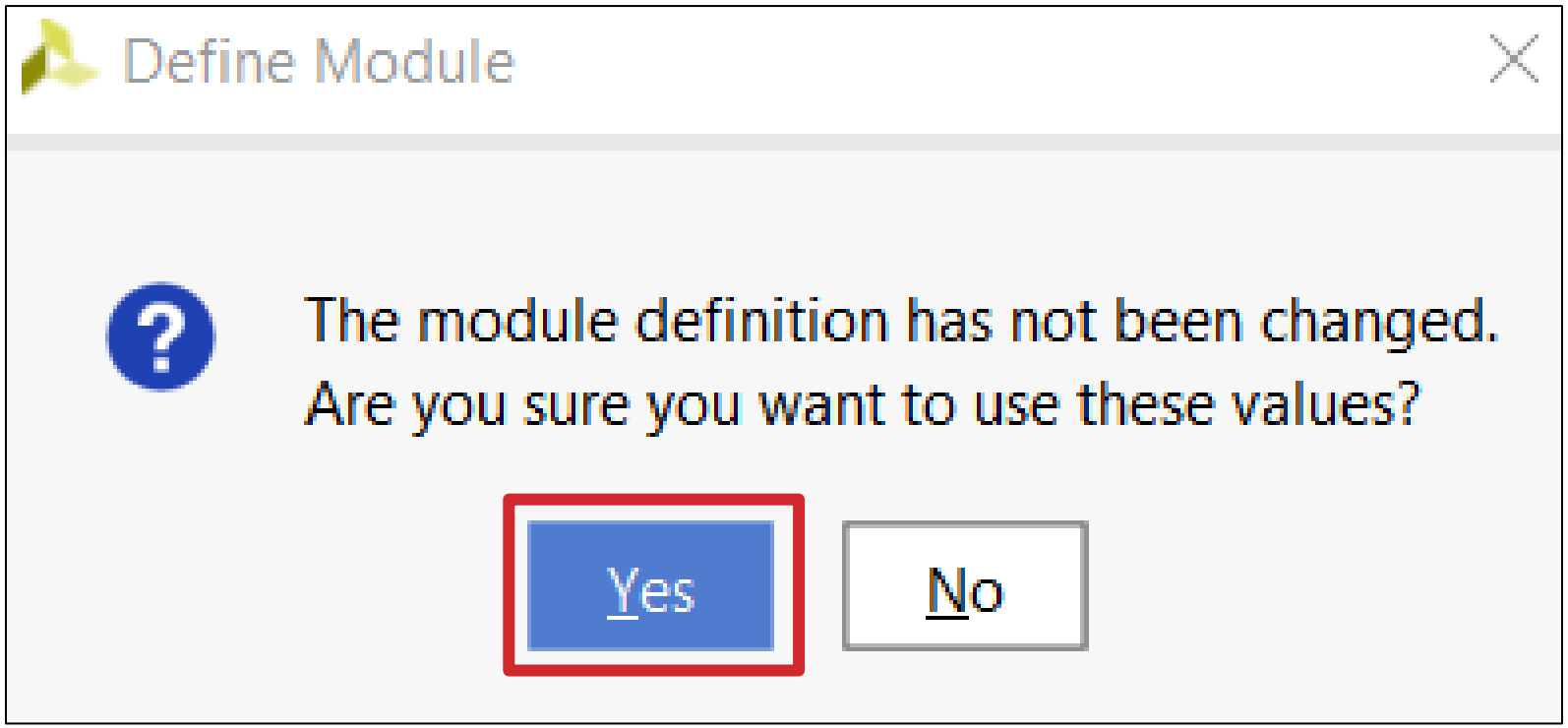
DESIGN EXAMPLE

- Vivado project



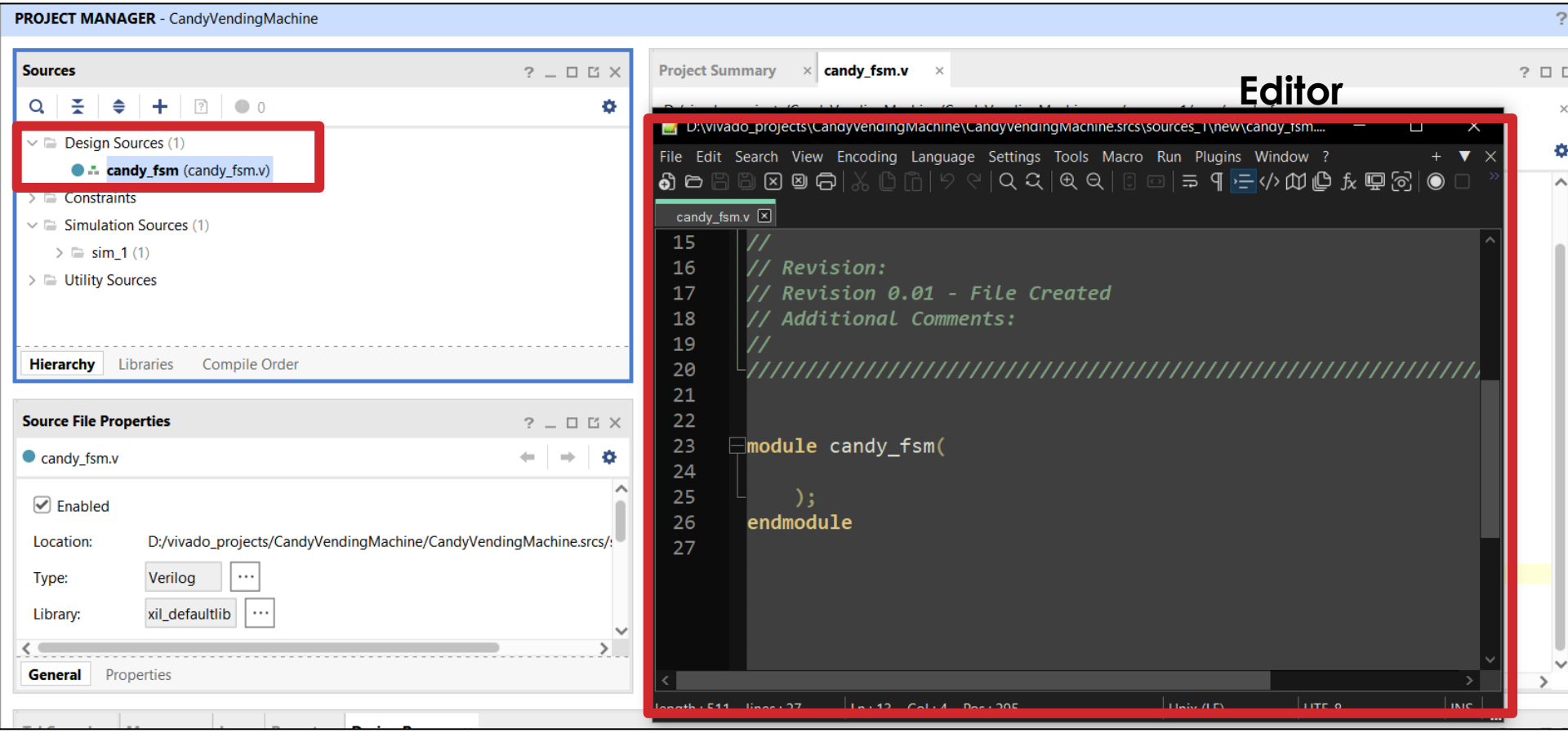
DESIGN EXAMPLE

- Vivado project



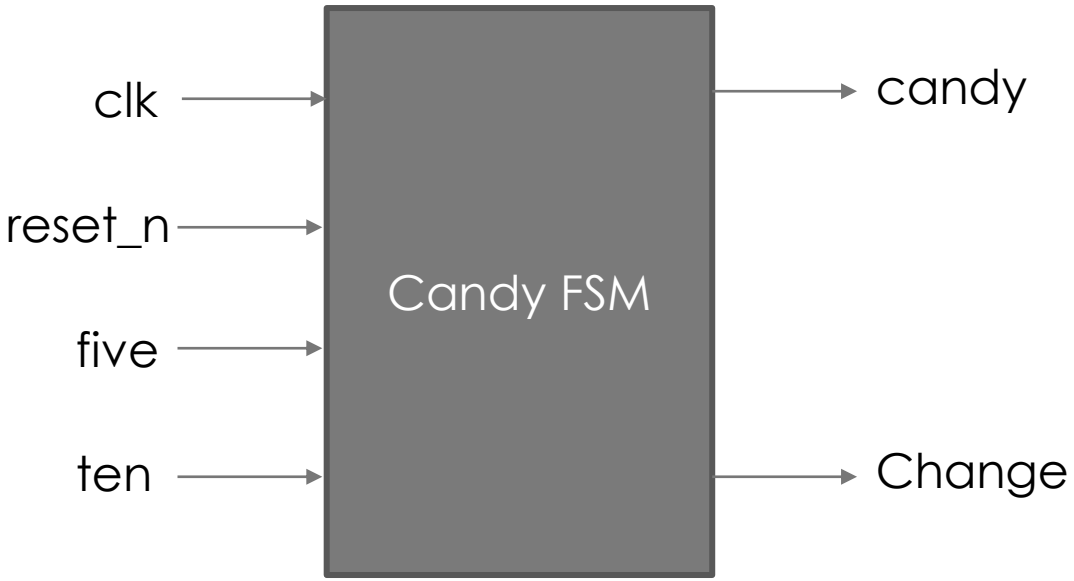
DESIGN EXAMPLE

- Vivado project



DESIGN EXAMPLE

- Candy Vending Machine FSM Block Diagram
  - Block Diagram and Signal Description



Signal	Description
clk	Input Clock
reset_n	Input negedge edge reset
five	Input Five coin
ten	Input Ten coin
candy	Output candy
change	Output change

DESIGN EXAMPLE

- Candy Vending Machine FSM
  - Mealy Coding Style
    - Using 1 sequential process and 2 combinatorial processes

```
module mudule_name (//input/outputs);
    //input/output declaration
    input ...;
    output ...;

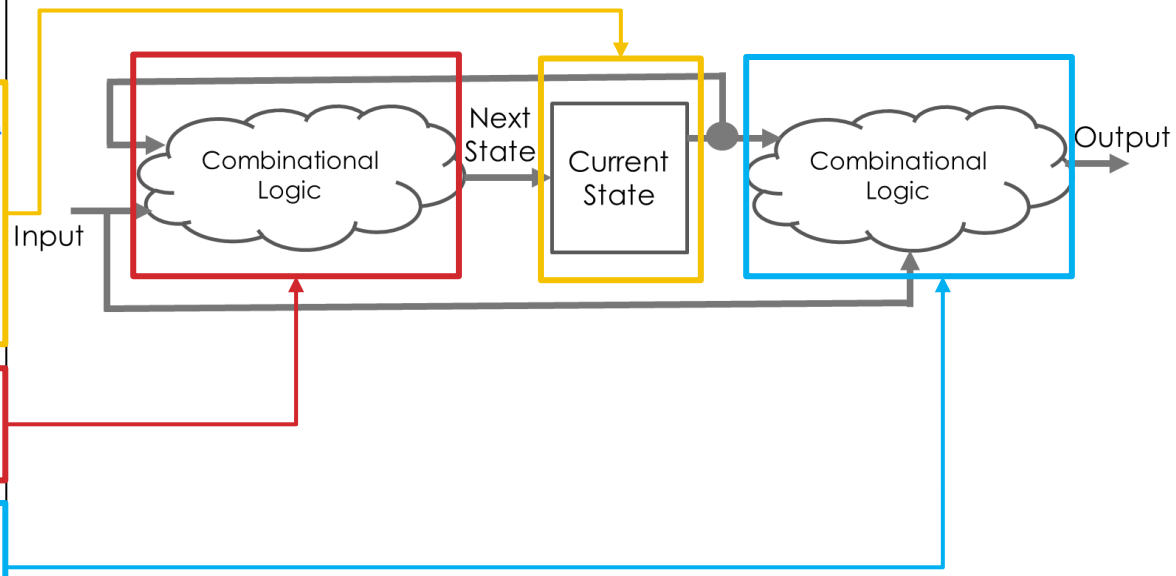
    //state register declaration
    reg [...] current_state, next_state;

    //Part 1: Sequential Process
    always@(posedge clk or negedge reset_n) begin
        if(!reset_n) begin
            //<Initialize current state>
        end
        else begin
            //<Update current state>
        end
    end

    //Part 2: Combinational Logic
    always@(current_state or inputs) begin
        //<compute next state function>
    end

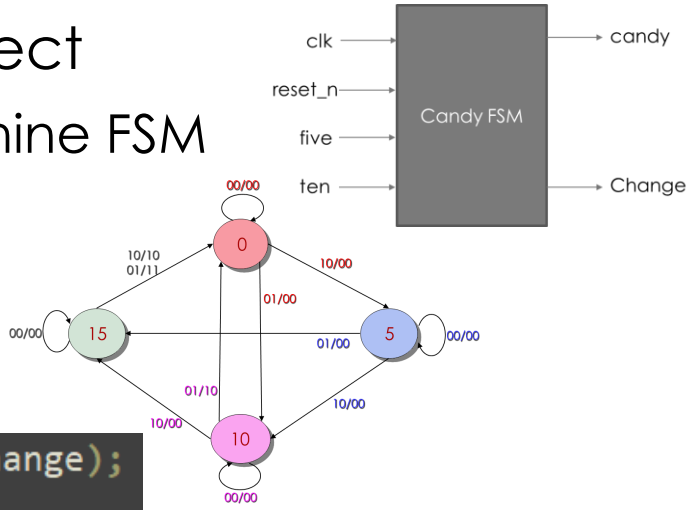
    //Part 3: Combinational Logic
    always@(current_state or inputs) begin
        //<compute output function>
    end

endmodule
```



DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Verilog Code for Candy Vending Machine FSM
    - I/O and state registers declaration



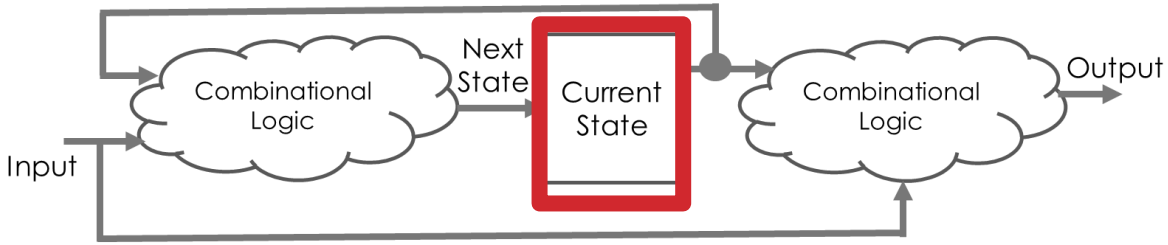

```
23  module candy_fsm(clk, reset_n, five, ten, candy, change);
24      //input/output declaration
25      input clk, reset_n; //global clock and reset inputs
26      input five, ten; // 5 and 10 coin input
27      output reg candy, change; //release candy and change
28
29      //State register declaration
30      reg [1:0] current_state, next_state;
31
32      //Declare states in FSM
33      parameter [1:0] zero_state    = 2'b00;
34      parameter [1:0] five_state    = 2'b01;
35      parameter [1:0] ten_state     = 2'b10;
36      parameter [1:0] fifteen_state = 2'b11;
```

Global signals and Input/output declaration

State registers and state declaration

DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Verilog Code for Candy Vending Machine FSM
    - Current state register initialization and updating

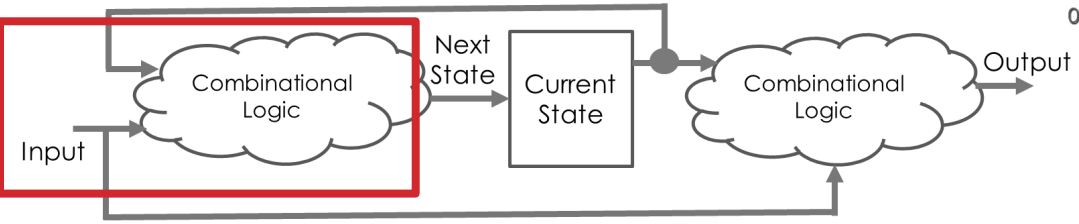


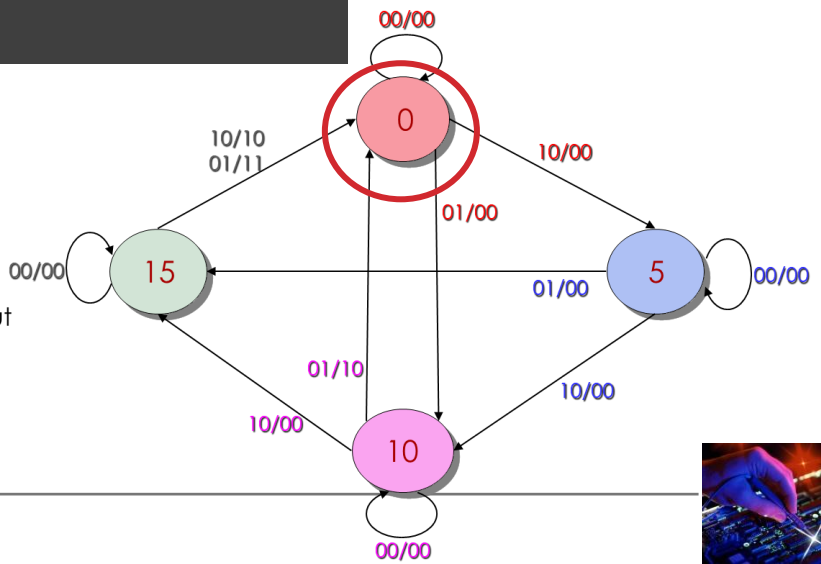

```
38      //current state initialization and update
39      always@(posedge clk or negedge reset_n) begin
40          if(!reset_n) begin
41              current_state <= zero_state; //initialize current_state
42          end
43          else begin
44              current_state <= next_state; //update current_state
45          end
46      end
```

DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Verilog Code for Candy Vending Machine FSM
    - Next state function computation

```
48    //Compute next state function (next_state)
49    always@(current_state or five or ten) begin
50        case (current_state)
51            //zero state
52            zero_state: begin
53                case({five,ten})
54                    2'b10: next_state = five_state;     //input 5 coin
55                    2'b01: next_state = ten_state;      //input 10 coin
56                    default: next_state = zero_state;
57                endcase
58            end
```

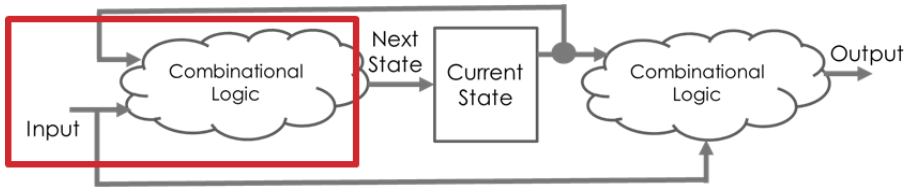


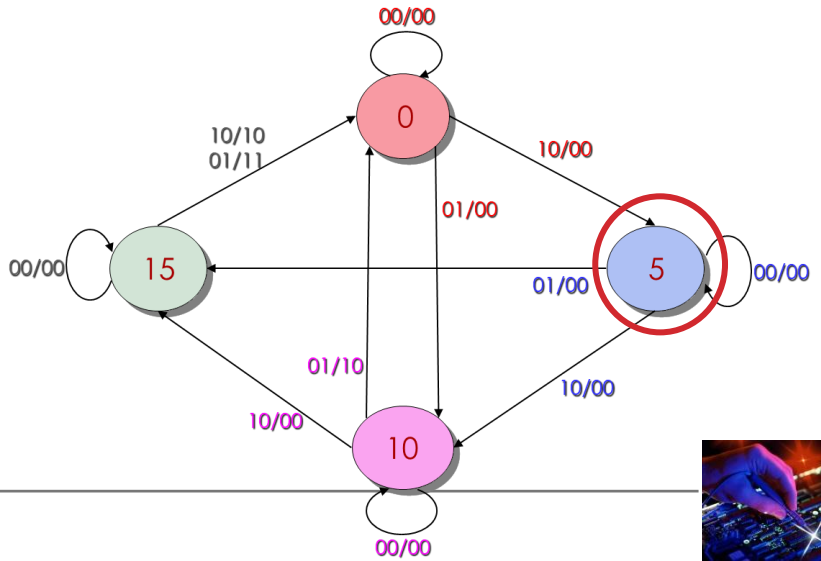


DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Verilog Code for Candy Vending Machine FSM
    - Next state function computation

```
59          //five state
60          five_state: begin
61              case({five,ten})
62                  2'b10: next_state = ten_state;       //input 5 coin
63                  2'b01: next_state = fifteen_state;  //input 10 coin
64                  default: next_state = five_state;
65              endcase
66          end
```

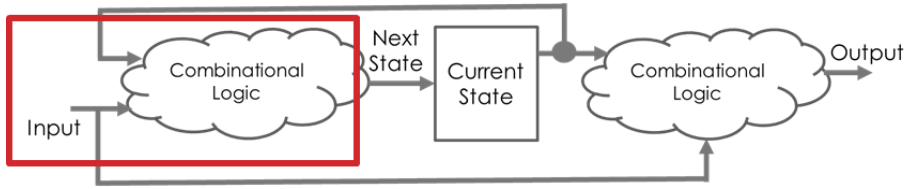


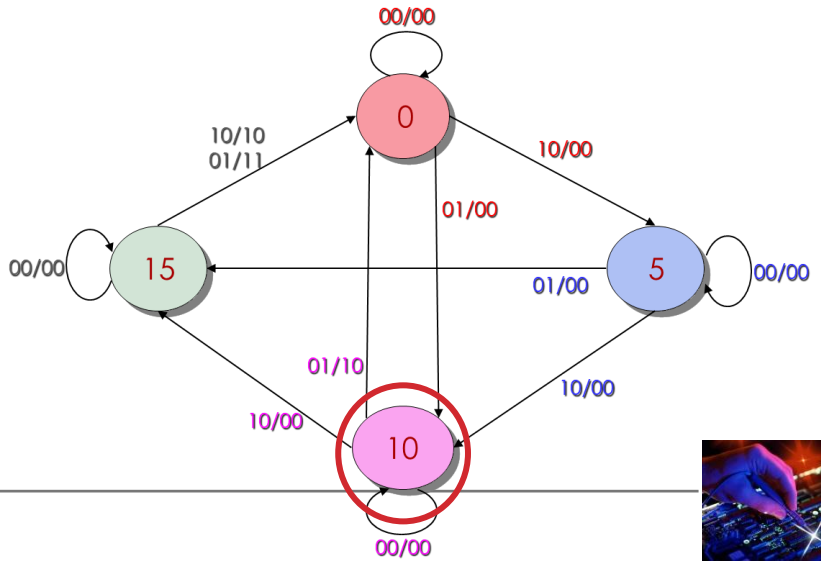


DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Verilog Code for Candy Vending Machine FSM
    - Next state function computation

```
67              //ten state
68              ten_state: begin
69                  case({five,ten})
70                      2'b10: next_state = fifteen_state;  //input 5 coin
71                      2'b01: next_state = zero_state;     //input 10 coin
72                      default: next_state = ten_state;
73                  endcase
74              end
```

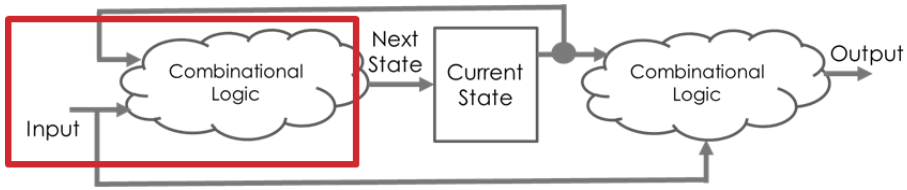


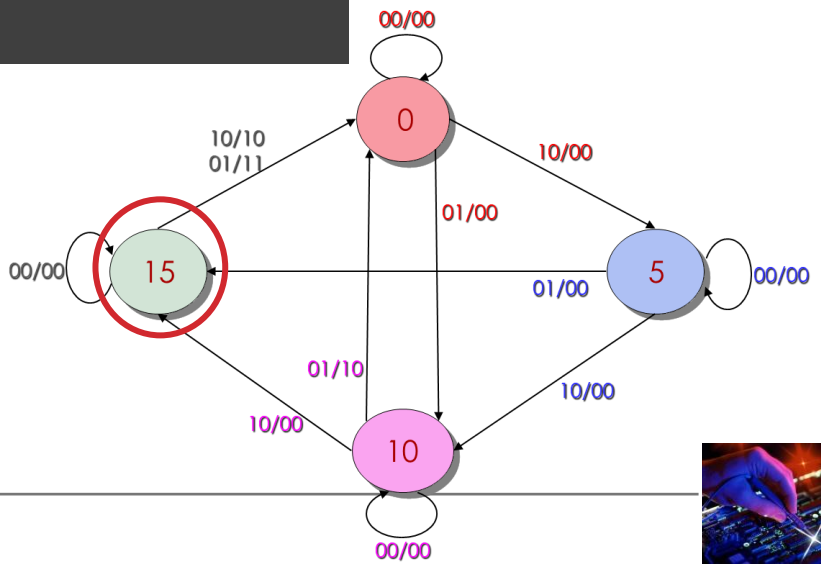


DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Verilog Code for Candy Vending Machine FSM
    - Next state function computation

```
75              //fifteen state
76              fifteen_state: begin
77                  case({five,ten})
78                      2'b10: next_state = zero_state;     //input 5 coin
79                      2'b01: next_state = zero_state;     //input 10 coin
80                      default: next_state = fifteen_state;
81                  endcase
82              end
83              default: next_state = zero_state;
84          endcase
85      end
```

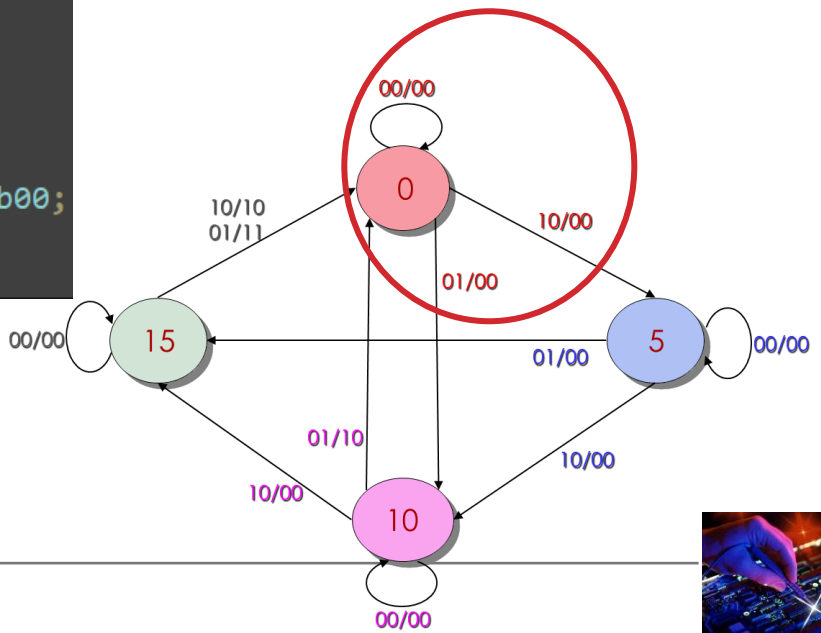




DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Verilog Code for Candy Vending Machine FSM
    - Output function computation

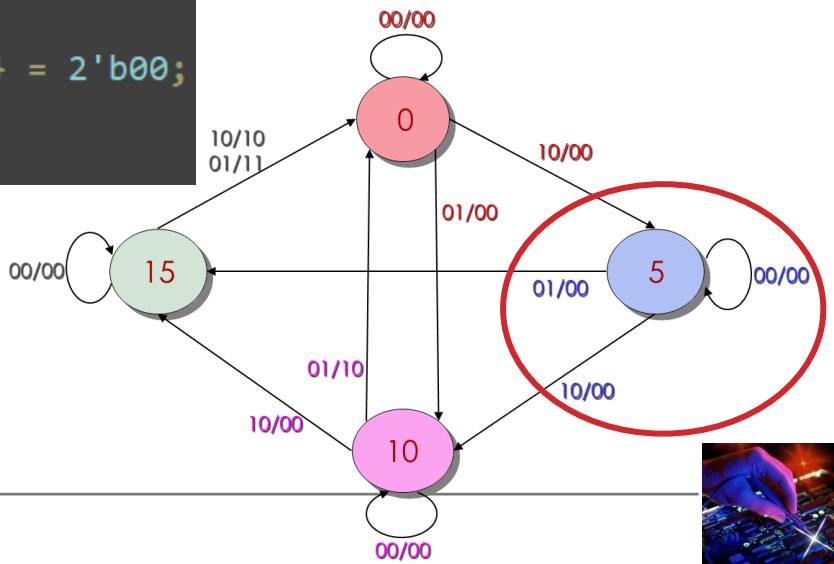
```
87    //Compute output functions (candy and change)
88    always@(current_state or five or ten) begin
89        case (current_state)
90            //zero state
91            zero_state: begin
92                case({five,ten})
93                    2'b10: begin //input 5 coin
94                        {candy, change} = 2'b00;
95                    end
96                    2'b01: begin //input 10 coin
97                        {candy, change} = 2'b00;
98                    end
99                    default: {candy, change} = 2'b00;
100               endcase
101           end
```



DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Verilog Code for Candy Vending Machine FSM
    - Output function computation

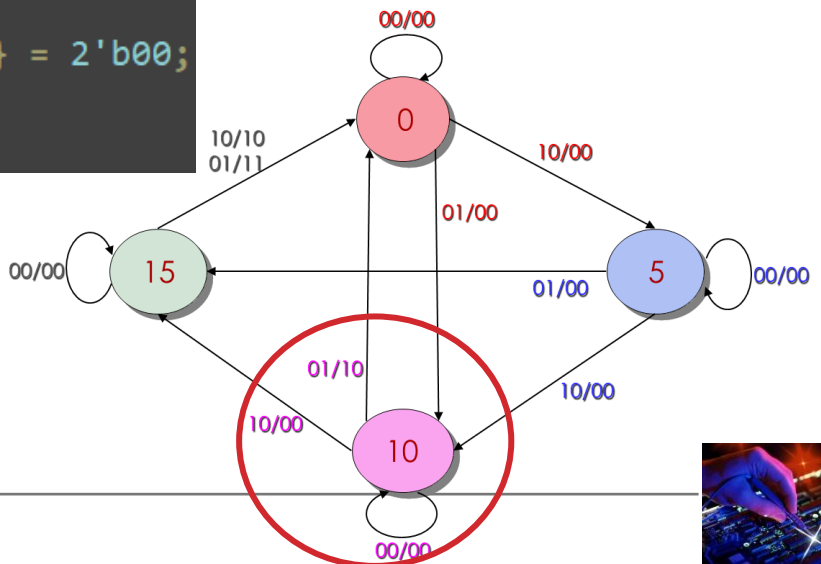
```
102             //five state
103             five_state: begin
104                 case({five,ten})
105                     2'b10: begin //input 5 coin
106                         {candy, change} = 2'b00;
107                     end
108                     2'b01: begin //input 10 coin
109                         {candy, change} = 2'b00;
110                     end
111                     default: {candy, change} = 2'b00;
112                 endcase
113             end
```



DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Verilog Code for Candy Vending Machine FSM
    - Output function computation

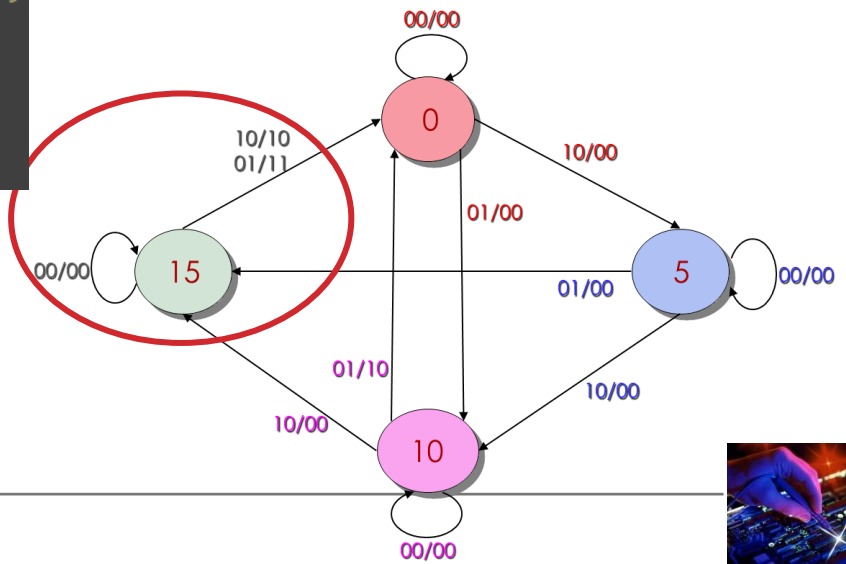
```
114             //ten state
115             ten_state: begin
116                 case({five,ten})
117                     2'b10: begin //input 5 coin
118                         {candy, change} = 2'b00;
119                     end
120                     2'b01: begin //input 10 coin
121                         {candy, change} = 2'b10;
122                     end
123                     default: {candy, change} = 2'b00;
124                 endcase
125             end
```



DESIGN EXAMPLE

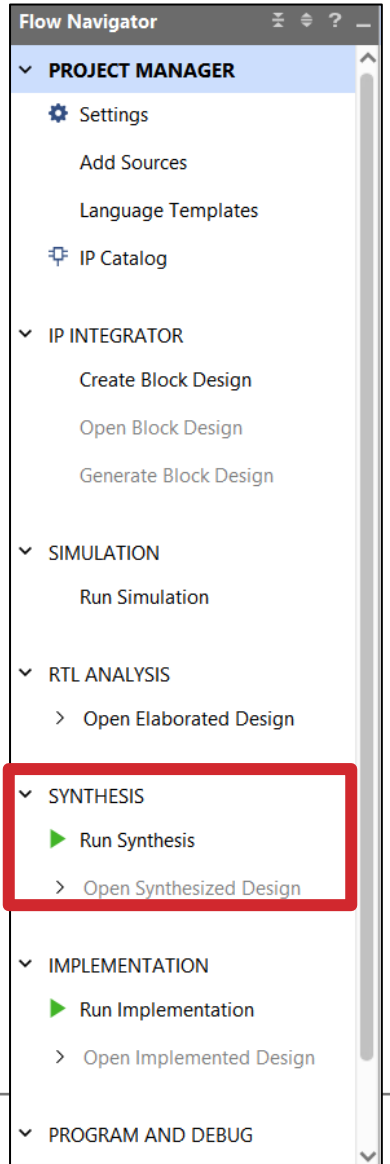
- Candy Vending Machine Project Project
  - Verilog Code for Candy Vending Machine FSM
    - Output function computation

```
126             //fifteen state
127             fifteen_state: begin
128                 case({five,ten})
129                     2'b10: begin //input 5 coin
130                         {candy, change} = 2'b10;
131                     end
132                     2'b01: begin //input 10 coin
133                         {candy, change} = 2'b11;
134                     end
135                     default: {candy, change} = 2'b00;
136                 endcase
137             end
138             default: {candy, change} = 2'b00;
139         endcase
140     end
```



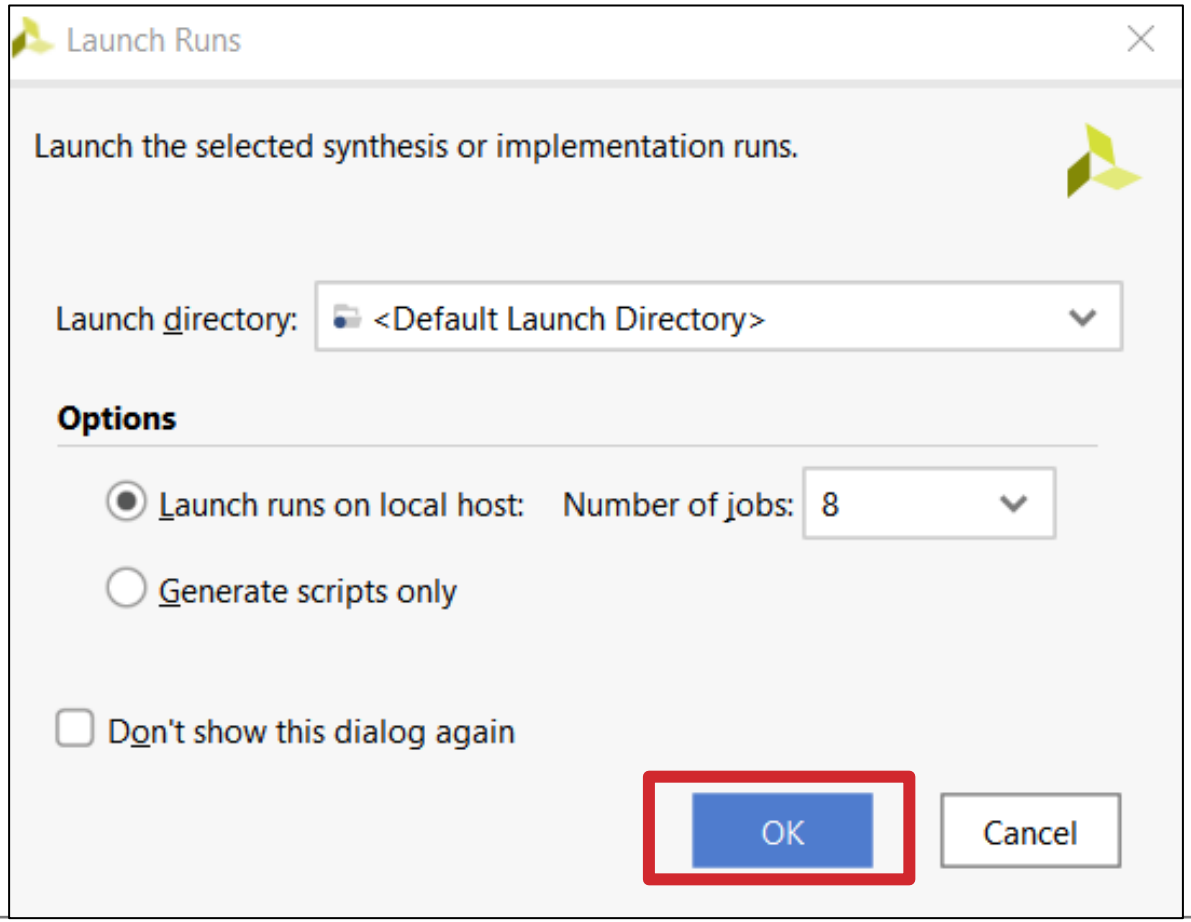
DESIGN EXAMPLE

- Vivado project
  - Logic synthesis



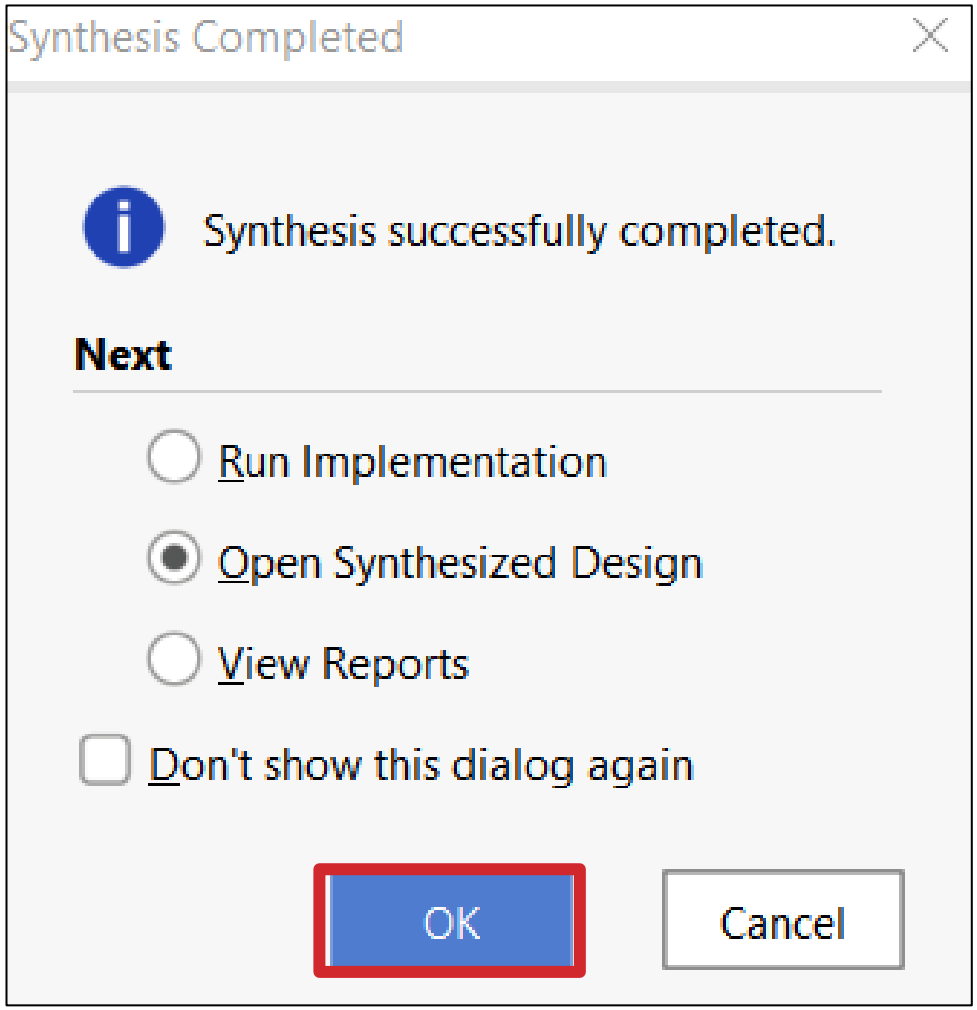
DESIGN EXAMPLE

- Vivado project
  - Logic synthesis



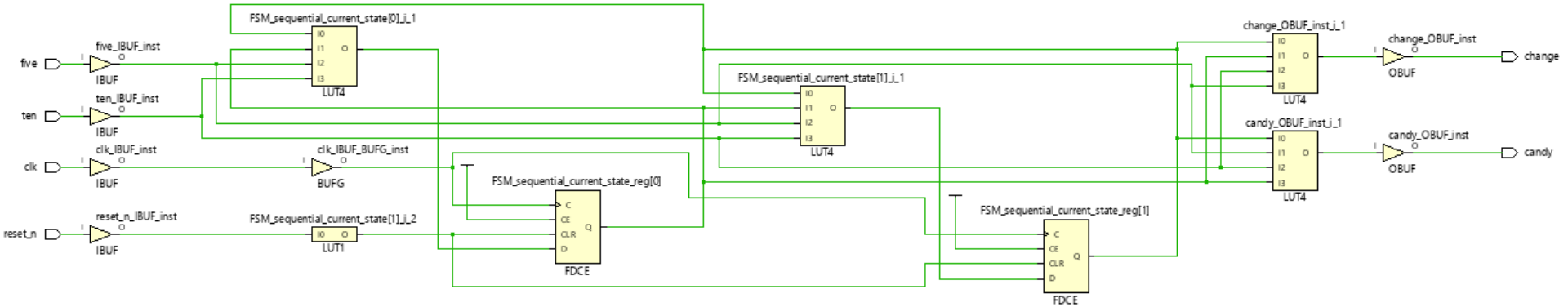
DESIGN EXAMPLE

- Vivado project
  - Logic synthesis



DESIGN EXAMPLE

- Vivado project
  - Logic synthesis (Schematic)



DESIGN EXAMPLE

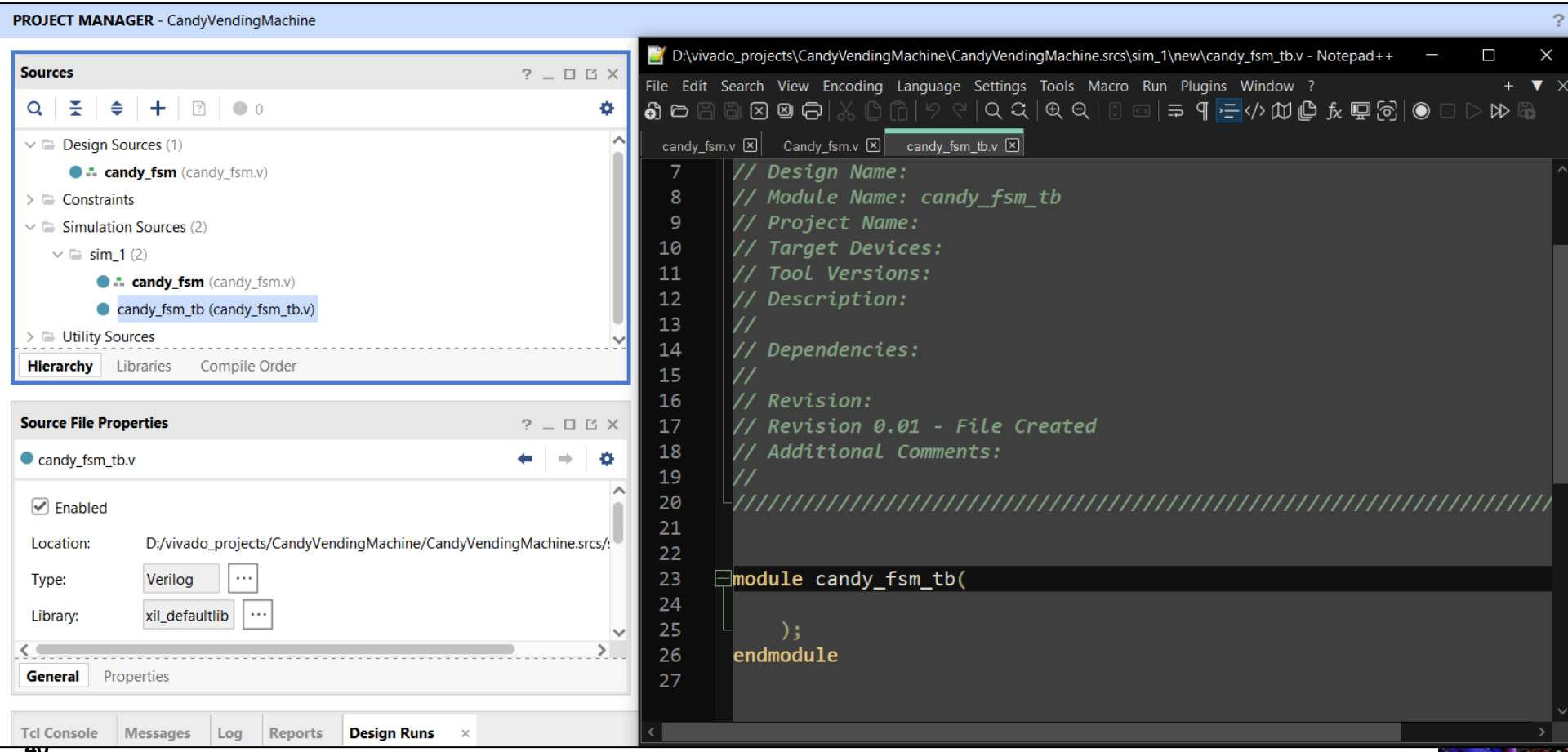
- Candy Vending Machine Project
  - Candy FSM Testbench (Test Vectors)
    - Coin input sequence: five(5) -> ten(10) -> ten(10)

current_state	five	ten	next_state	candy	change
zero	0	0	zero	0	0
zero	1	0	five	0	0
five	0	1	fifteen	0	0
fifteen	0	1	zero	1	1



DESIGN EXAMPLE

- Vivado project
  - Creating a Testbench



DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy FSM Testbench

```
23  module candy_fsm_tb();
24      //inputs
25      reg clk, reset_n, five, ten;
26      //outputs
27      wire candy, change;
28
29      //instantiate the Unit Under Test (UUT)
30      candy_fsm uut(
31          .clk(clk),
32          .reset_n(reset_n),
33          .five(five),
34          .ten(ten),
35          .candy(candy),
36          .change(change)
37      );
38
39      //generate clock pulse
40      parameter clk_period = 20;
41
42      initial begin
43          clk = 1'b0;
44          forever #(clk_period/2) clk = ~clk;
45      end
```

Generate clock pulse using 50Mhz frequency



DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy FSM Testbench

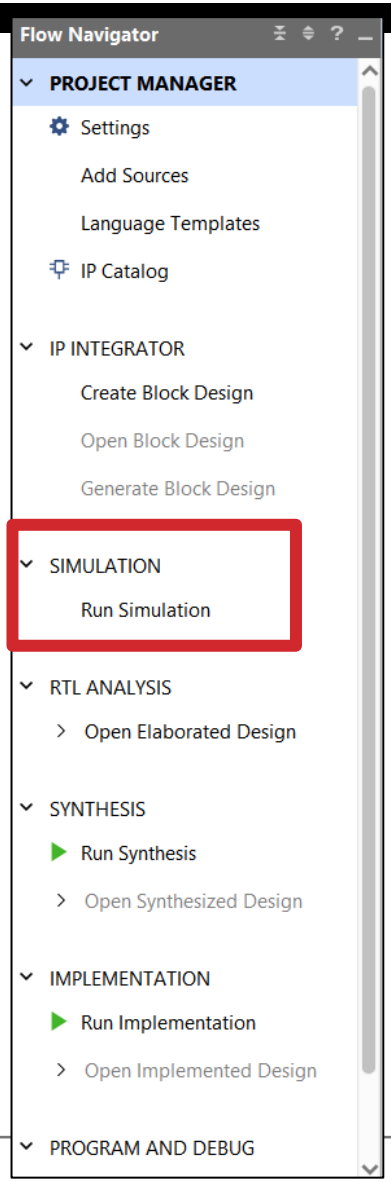
current_state	five	ten	next_state	candy	change
zero	0	0	zero	0	0
zero	1	0	five	0	0
five	0	1	fifteen	0	0
fifteen	0	1	zero	1	1

```
47      initial begin
48          // Initialize Inputs
49          reset_n = 0;
50          five = 0; ten = 0; #100;
51          reset_n = 1;
52          five = 1; ten = 0; #15;
53          five = 0; ten = 0; #5;
54          five = 0; ten = 0; #15;
55          five = 0; ten = 0; #5;
56          five = 0; ten = 1; #15;
57          five = 0; ten = 0; #5;
58      end
59  endmodule
```



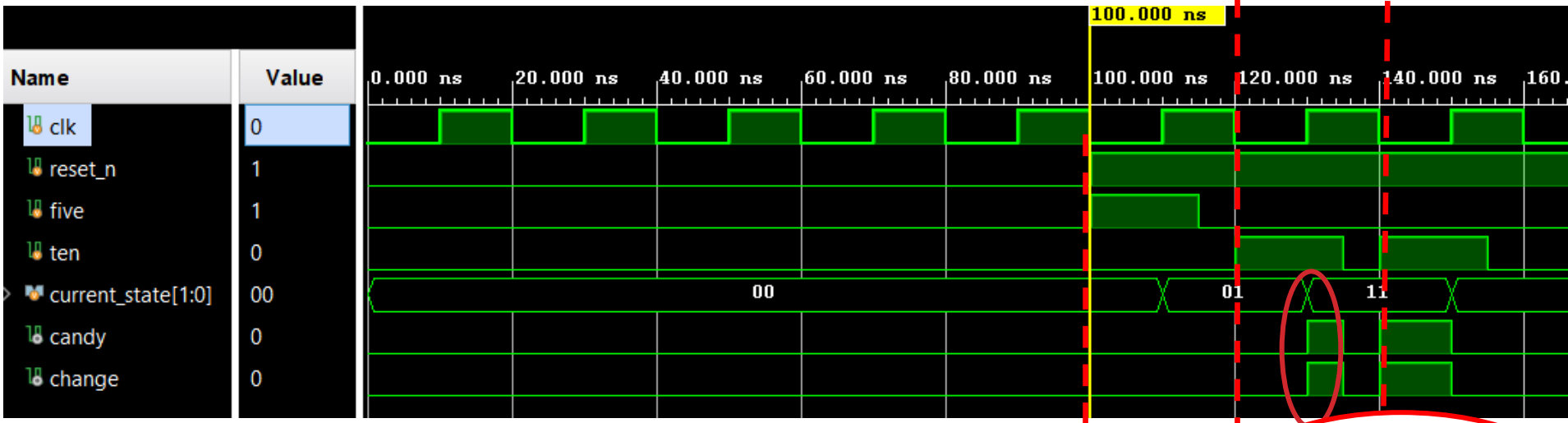
DESIGN EXAMPLE

- Vivado project
  - Running Simulation



DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy FSM Simulation Waveform (With glitches)




```
126             //fifteen state
127             fifteen_state: begin
128                 case({five,ten})
129                     2'b10: begin //input 5 coin
130                         {candy, change} = 2'b10;
131                     end
132                     2'b01: begin //input 10 coin
133                         {candy, change} = 2'b11;
134                     end
135                     default: {candy, change} = 2'b00;
136                 endcase
137             end
138             default: {candy, change} = 2'b00;
139         endcase
140     end
```

DESIGN EXAMPLE

- Candy Vending Machine Project
  - Solving Mealy Glitch
    - One solution is by using registered outputs as shown in the code

```
23  module candy_fsm(clk, reset_n, five, ten, candy_out, change_out);
24      //input/output declaration
25      input clk, reset_n; //global clock and reset inputs
26      input five, ten; // 5 and 10 coin input
27      output reg candy_out, change_out; //release candy and change
28
29      reg candy, change; //unregistered candy and change
```

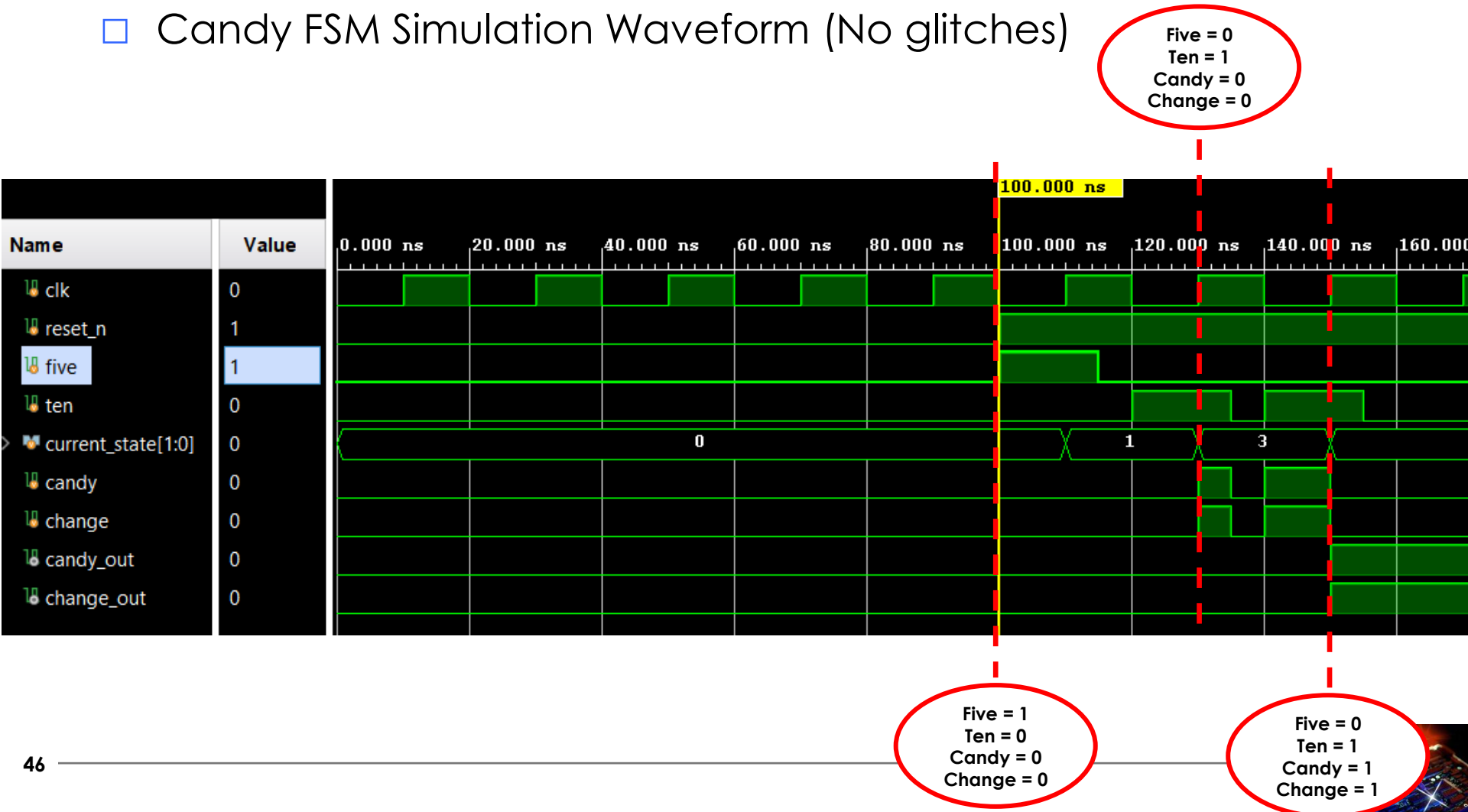
```
144     //registered outputs
145     always@(posedge clk or negedge reset_n) begin
146         if(!reset_n) begin
147             {candy_out, change_out} <= 2'b00;
148         end
149         else if (current_state != 2'b00) begin
150             {candy_out, change_out} <= {candy, change};
151         end
152         else begin
153             {candy_out, change_out} <= {candy_out, change_out};
154         end
155     end
```

The outputs (candy and change) are registered at the rising edge of the clock to prevent glitches



DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy FSM Simulation Waveform (No glitches)



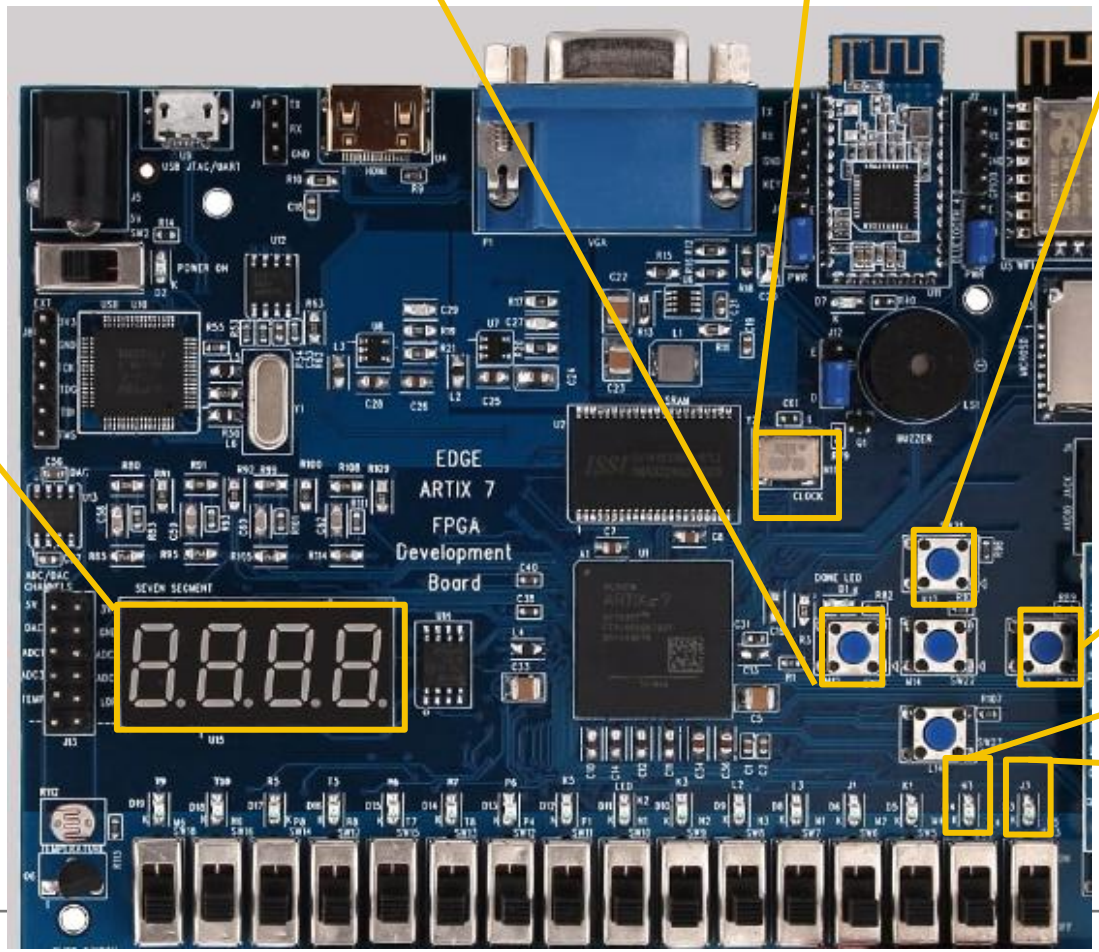
DESIGN EXAMPLE

- Candy Vending Machine Project: Exercise
  - Design and Simulate Testbenches for the input sequences below:
    - Coin input sequence: five(5) -> five(5) -> five(5) -> five(5)
    - Coin input sequence: five(5) -> ten(10) -> five(5)
    - Coin input sequence: ten(10) -> five(5) -> five(5)
    - Coin input sequence: ten(10) -> ten(10)
    - Coin input sequence: ten(10) -> five(5) -> ten(10)



DESIGN EXAMPLE

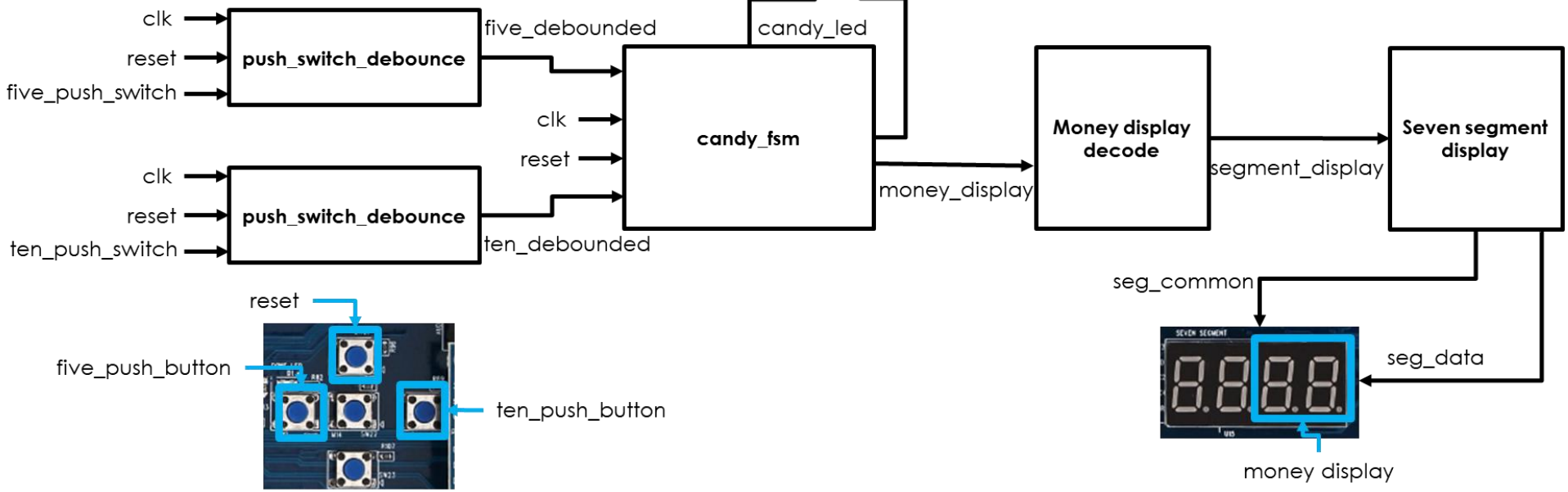
- Programming FPGA
  - Board Demo



DESIGN EXAMPLE

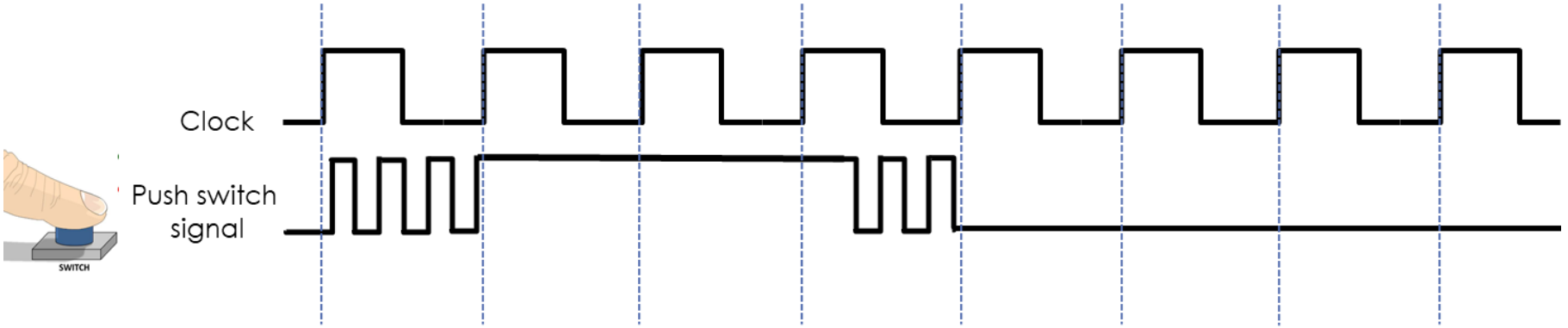
- Candy Vending Machine Project
  - Modules of Candy Vending Machine





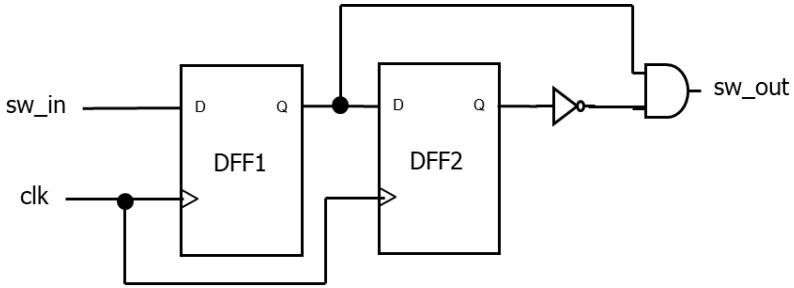
DESIGN EXAMPLE

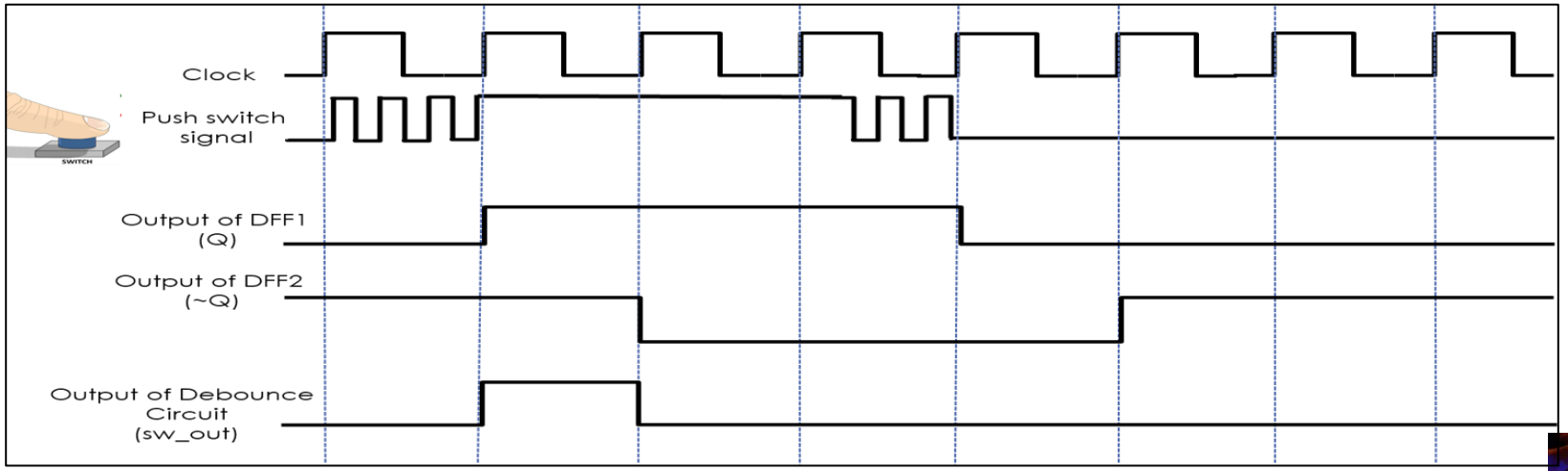
- Candy Vending Machine Project
  - push_switch_debounce Module
    - As shown in the figure below, when a button on FPGA is pressed and released, there are many unexpected up-and-down bounces in the push button signal



DESIGN EXAMPLE

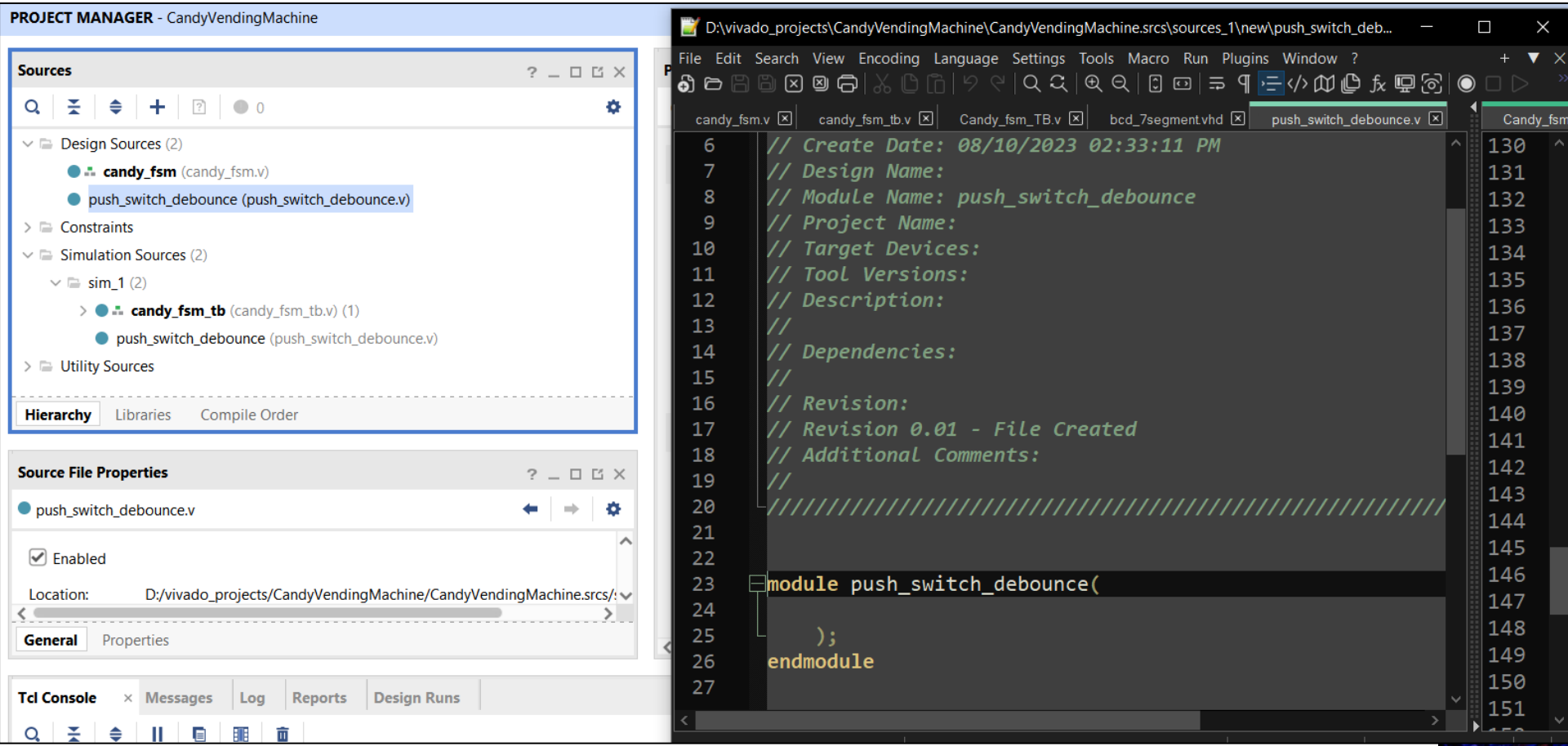
- Candy Vending Machine Project
  - push_switch_debounce Module
    - The debouncing circuit (using two D flip-flops) only generates a single pulse with a period of the slow clock without bouncing as we expected





DESIGN EXAMPLE

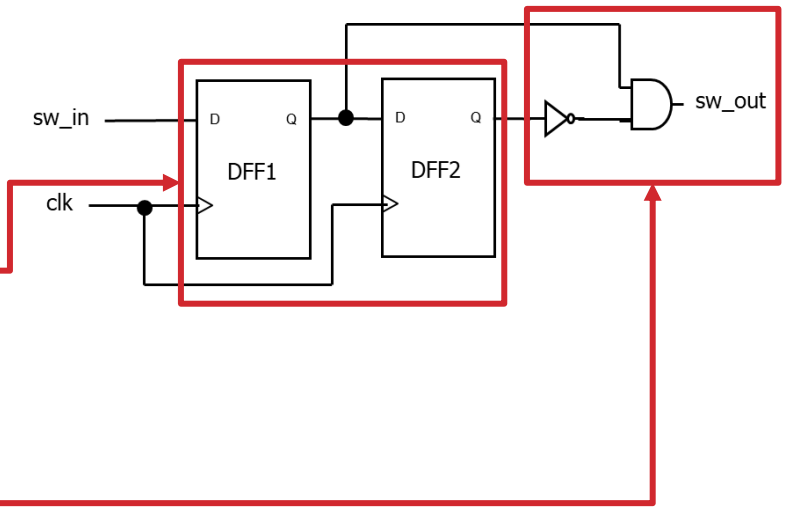
- Candy Vending Machine Project
  - push_switch_debounce Verilog code
    - Module name: push_switch_debounce.v



DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Verilog Code for Push Switch Debounce circuit

```
23  module push_switch_debounce(clk, rst, sw_in, sw_out);
24      input clk, rst, sw_in;
25      output sw_out;
26
27      reg [1:0] sample_edge;
28
29      always@(posedge clk or posedge rst) begin
30          if(rst) begin
31              sample_edge <= 2'b00;
32          end
33          else begin
34              sample_edge[1] <= sample_edge[0];
35              sample_edge[0] <= sw_in;
36          end
37      end
38
39      assign sw_out = sample_edge[0] && ~sample_edge[1];
40
41  endmodule
```



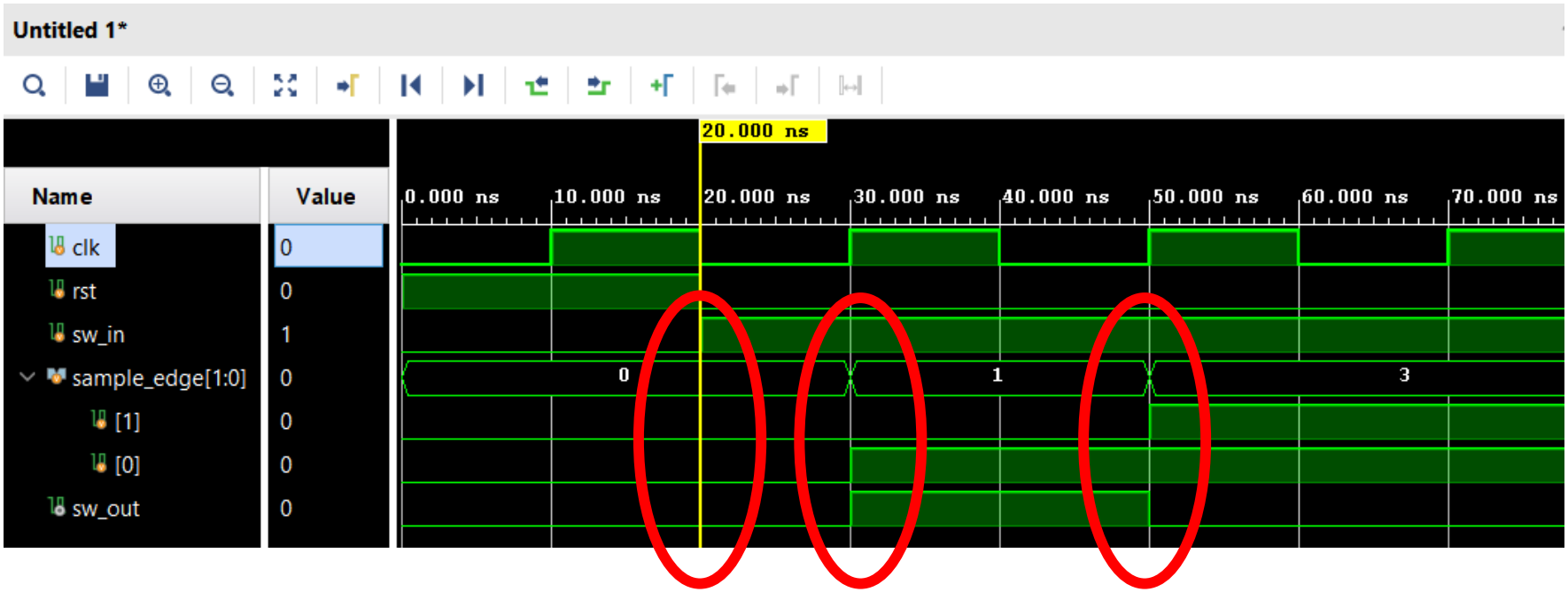
DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Push Switch Debounce circuit Verilog Testbench Code

```
23  module push_switch_debounce_tb();
24      //inputs
25      reg clk, rst, sw_in;
26      //outputs
27      wire sw_out;
28
29      //instantiate the Unit Under Test (UUT)
30      push_switch_debounce uut (
31          .clk(clk), .rst(rst), .sw_in(sw_in), .sw_out(sw_out)
32      );
33
34      //generate clock pulse
35      parameter clk_period = 20;
36      initial begin
37          clk = 1'b0;
38          forever #(clk_period/2) clk = ~clk;
39      end
40
41      initial begin
42          //initialize inputs
43          rst = 1; sw_in = 0; #20;
44          rst = 0; sw_in = 1;
45      end
46  endmodule
```

DESIGN EXAMPLE

- Candy Vending Machine Project
  - Push Switch Debounce circuit Verilog Simulation Waveform



DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Candy FSM Verilog Code (Modified)

1

```
23  module candy_fsm(clk, reset, five, ten, candy_out, change_out, money_display);
24      //input/output declaration
25      input clk, reset; //global clock and reset inputs
26      input five, ten; // 5 and 10 coin input
27      output reg candy_out, change_out; //release candy and change
28
29      output [1:0] money_display; //Encoded money to display on 7-segment
30
31      reg candy, change; //unregistered candy and change
32
33      //State register declaration
34      reg [1:0] current_state, next_state;
35
36      //Declare states in FSM
37      parameter [1:0] zero_state    = 2'b00;
38      parameter [1:0] five_state    = 2'b01;
39      parameter [1:0] ten_state     = 2'b10;
40      parameter [1:0] fifteen_state = 2'b11;
41
42      //current state initialization and update
43      always@(posedge clk or posedge reset) begin
44          if(reset) begin
45              current_state <= zero_state; //initialize current_state
46          end
47          else begin
48              current_state <= next_state; //update current_state
49          end
50      end
```

DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Candy FSM Verilog Code (Modified)

2

```
52      //Compute next state function (next_state)
53      always@(current_state or five or ten) begin
54          case (current_state)
55              //zero state
56              zero_state: begin
57                  case({five,ten})
58                      2'b10: next_state = five_state;     //input 5 coin
59                      2'b01: next_state = ten_state;      //input 10 coin
60                      default: next_state = zero_state;
61                  endcase
62              end
63              //five state
64              five_state: begin
65                  case({five,ten})
66                      2'b10: next_state = ten_state;      //input 5 coin
67                      2'b01: next_state = fifteen_state;  //input 10 coin
68                      default: next_state = five_state;
69                  endcase
70              end
71              //ten state
72              ten_state: begin
73                  case({five,ten})
74                      2'b10: next_state = fifteen_state;  //input 5 coin
75                      2'b01: next_state = zero_state;     //input 10 coin
76                      default: next_state = ten_state;
77                  endcase
78              end
79              //fifteen state
80              fifteen_state: begin
81                  case({five,ten})
82                      2'b10: next_state = zero_state;     //input 5 coin
83                      2'b01: next_state = zero_state;     //input 10 coin
84                      default: next_state = fifteen_state;
85                  endcase
86              end
87              default: next_state = zero_state;
88          endcase
89      end
```

DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Candy FSM Verilog Code (Modified)

```
90
91      //Compute output functions (candy and change)
92      always@(current_state or five or ten) begin
93          case (current_state)
94              //zero state
95              zero_state: begin
96                  case({five,ten})
97                      2'b10: begin //input 5 coin
98                          {candy, change} = 2'b00;
99                      end
100                     2'b01: begin //input 10 coin
101                         {candy, change} = 2'b00;
102                     end
103                     default: {candy, change} = 2'b00;
104                 endcase
105             end
106             //five state
107             five_state: begin
108                 case({five,ten})
109                     2'b10: begin //input 5 coin
110                         {candy, change} = 2'b00;
111                     end
112                     2'b01: begin //input 10 coin
113                         {candy, change} = 2'b00;
114                     end
115                     default: {candy, change} = 2'b00;
116                 endcase
117             end
```

3

```
118             //ten state
119             ten_state: begin
120                 case({five,ten})
121                     2'b10: begin //input 5 coin
122                         {candy, change} = 2'b00;
123                     end
124                     2'b01: begin //input 10 coin
125                         {candy, change} = 2'b10;
126                     end
127                     default: {candy, change} = 2'b00;
128                 endcase
129             end
130             //fifteen state
131             fifteen_state: begin
132                 case({five,ten})
133                     2'b10: begin //input 5 coin
134                         {candy, change} = 2'b10;
135                     end
136                     2'b01: begin //input 10 coin
137                         {candy, change} = 2'b11;
138                     end
139                     default: {candy, change} = 2'b00;
140                 endcase
141             end
142             default: {candy, change} = 2'b00;
143         endcase
144     end
```

4

DESIGN EXAMPLE

- Candy Vending Machine Project Project
  - Candy FSM Verilog Code (Modified)

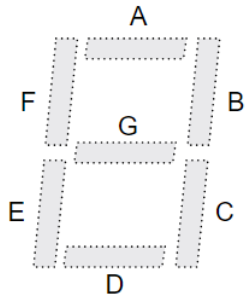
```
146     //registered outputs
147     always@(posedge clk or posedge reset) begin
148         if(reset) begin
149             {candy_out, change_out} <= 2'b00;
150         end
151         else if (current_state != 2'b00) begin
152             {candy_out, change_out} <= {candy, change};
153         end
154         else begin
155             {candy_out, change_out} <= {candy_out, change_out};
156         end
157     end
158
159     //money display
160     assign money_display = current_state;
161 endmodule
```

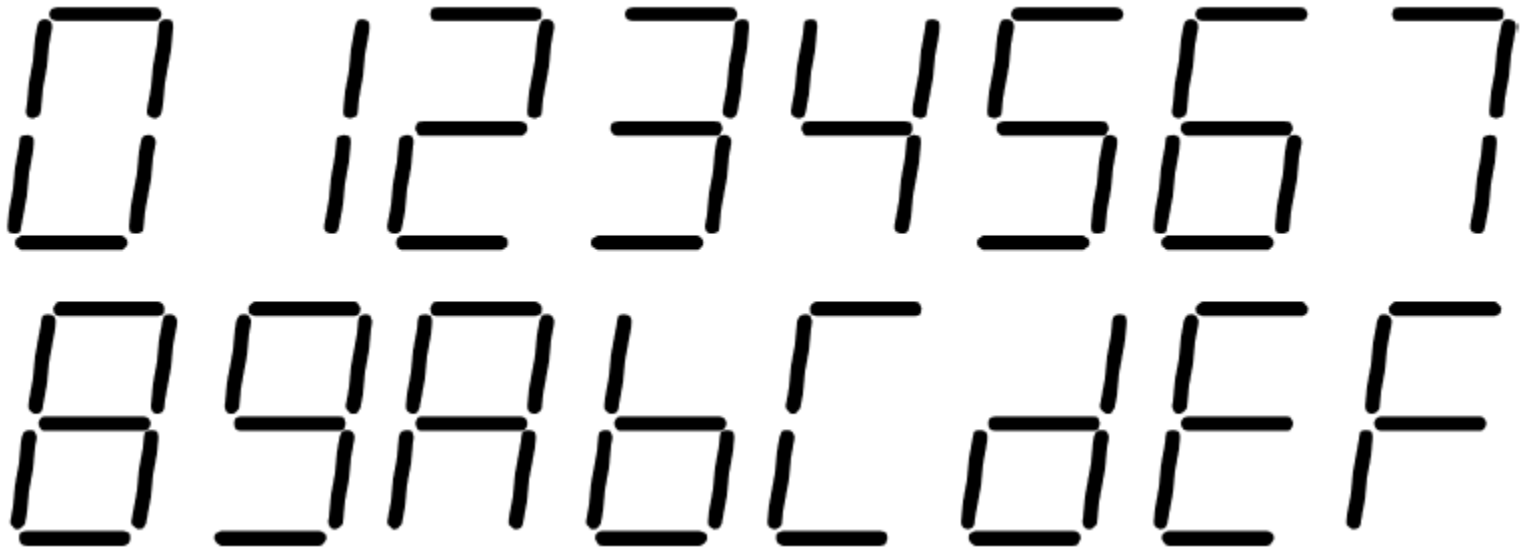
5



DESIGN EXAMPLE

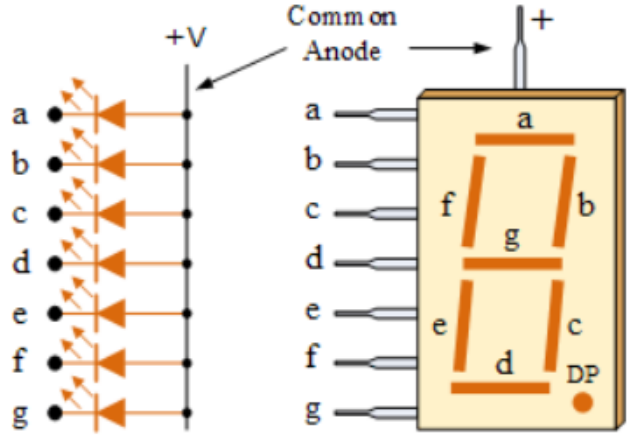
- Candy Vending Machine Project
  - 7-Segment Display Module
    - The following numbers can be displayed by setting the segments a, b, c, d, e, f, g, appropriately

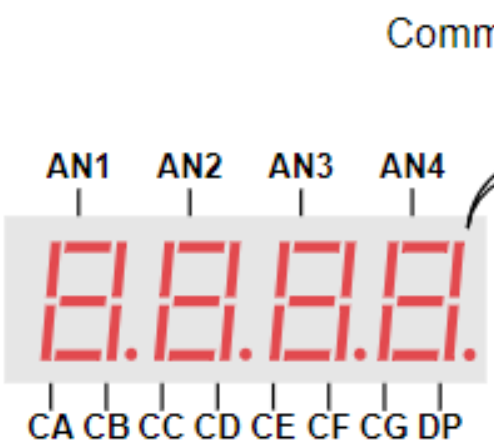




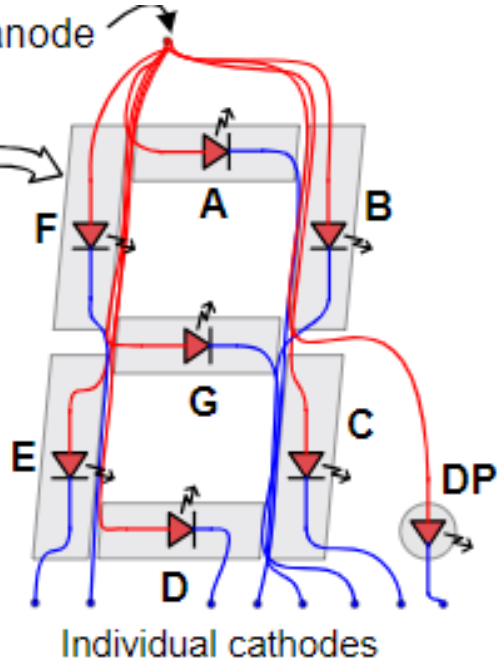
DESIGN EXAMPLE

- Candy Vending Machine Project
  - 7-Segment Display Module
    - The board uses a common anode display, which means that all of the anode connections for a given digit are tied together into a common circuit node
    - To illuminate a given segment in a given digit, a '1' must be applied to the digit's anode, and '0' must be applied to the segment's cathodes





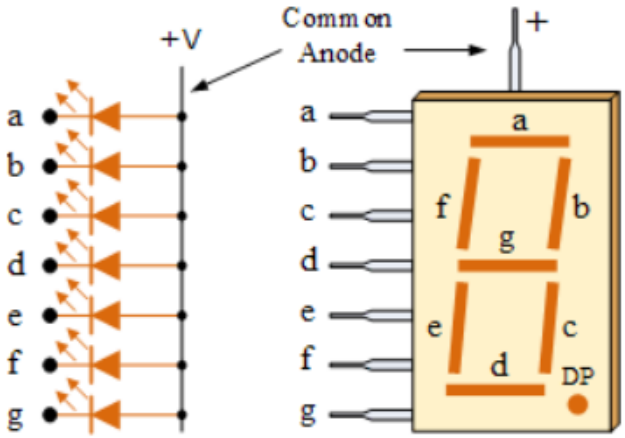
Four-digit Seven Segment Display



Individual cathodes

DESIGN EXAMPLE

- Candy Vending Machine Project
  - 7-Segment Display Module



	dp	*g*	*f*	*e*	*d*	*c*	*b*	*a*	binary	hexadecimal
0	1	1	0	0	0	0	0	0	0*b*11000000	0*xc*0
1	1	1	1	1	1	0	0	1	0*b*11111001	0*xf*9
2	1	0	1	0	0	1	0	0	0*b*10100100	0*xa*4
3	1	0	1	1	0	0	0	0	0*b*10110000	0*xb*0
4	1	0	0	1	1	0	0	1	0*b*10011001	0*x*99
5	1	0	0	1	0	0	1	0	0*b*10010010	0*x*92
6	1	0	0	0	0	0	1	0	0*b*10000010	0*x*82
7	1	1	1	1	1	0	0	0	0*b*11111000	0*xf*8
8	1	0	0	0	0	0	0	0	0*b*10000000	0*x*80
9	1	0	0	1	0	0	0	0	0*b*10010000	0*x*90
A	1	0	0	0	1	0	0	0	0*b*10001000	0*x*88
b	1	0	0	0	0	0	1	1	0*b*10000011	0*x*83
C	1	1	0	0	0	1	1	0	0*b*11000110	0*xc*6
d	1	0	1	0	0	0	0	1	0*b*10100001	0*xa*1
E	1	0	0	0	0	1	1	0	0*b*10000110	0*x*86
F	1	0	0	0	1	1	1	0	0*b*10001110	0*x*8*e*
OFF	1	1	1	1	1	1	1	1	0*b*11111111	0*xf f*

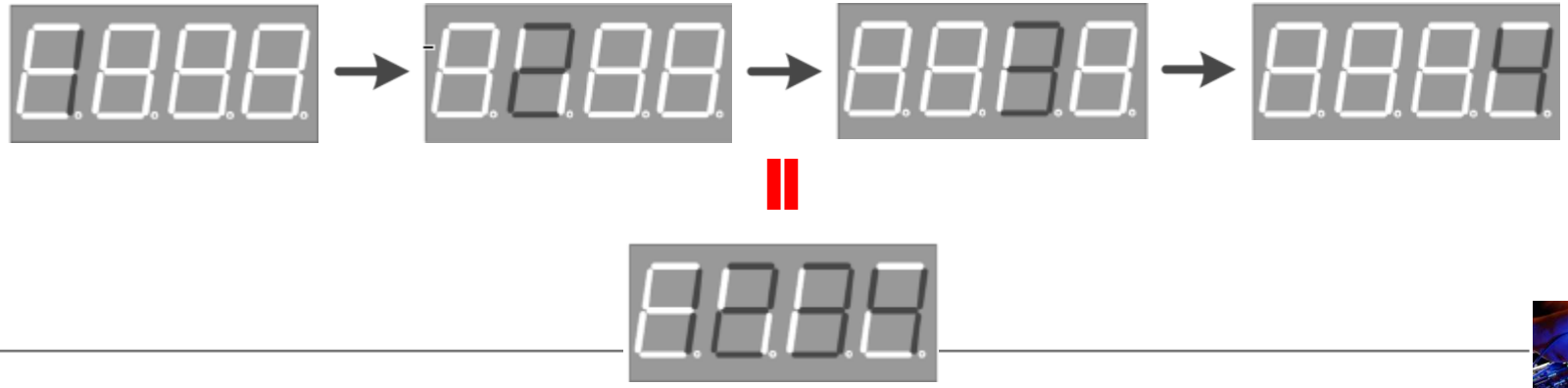
DESIGN EXAMPLE

- Candy Vending Machine Project
  - 7-Segment Display Module
    - Controlling 4 common terminals is termed scanning method
      - Reduces number of utilized I/O of a device
      - Common data line for all the 4 segment
      - Select lines for each 7-Segment in the array
    - If the control inputs a value of '1' to the common terminal of the 7-segment A number or letter is displayed on the selected Segment



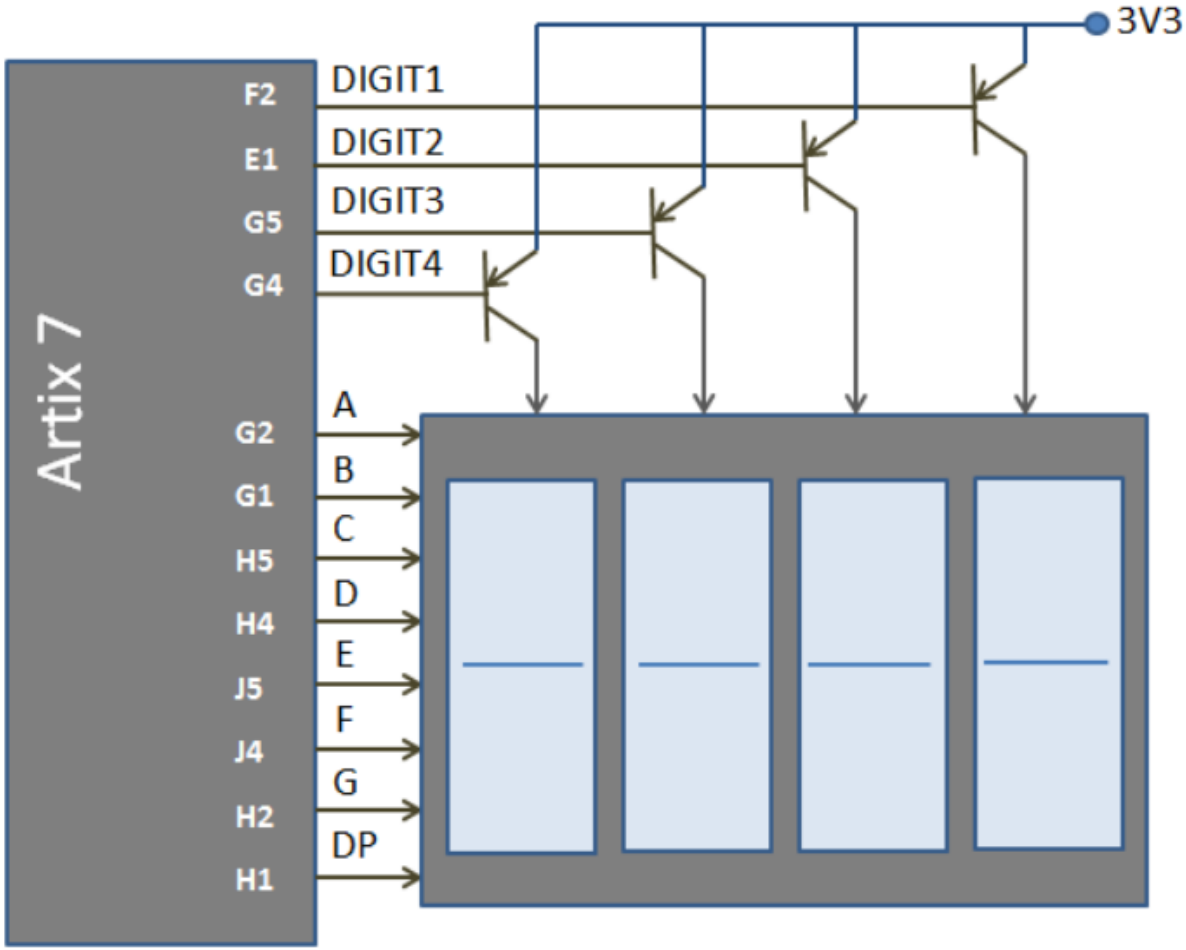
DESIGN EXAMPLE

- Candy Vending Machine Project
  - 7-Segment Display Module
    - To display the number "1234" on four 7-Segments, pass value '1' to the first common terminal and '0' to the remainder of the common terminal to select the first
      - Data to be displayed (1) is passed to seg_data
    - The second segment is selected with a '1' that common terminal and '0'to the rest
      - The data to be displayed (2) is passed to seg_data.
      - This is repeated for the 3rd and 4th digits



DESIGN EXAMPLE

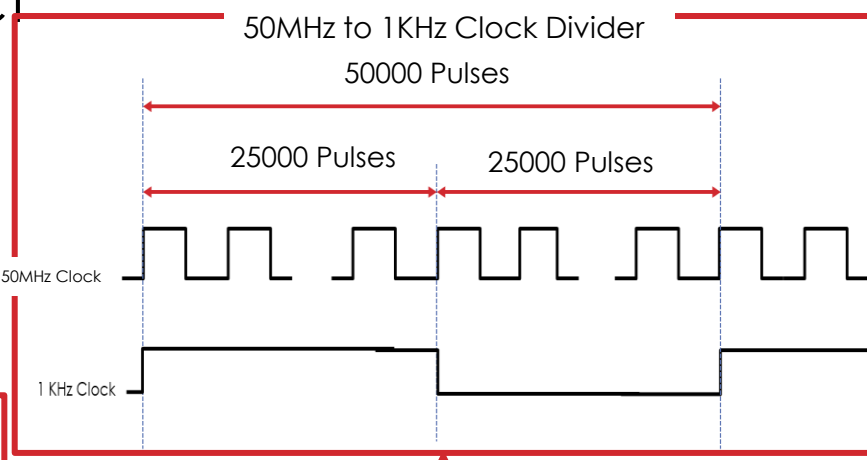
- Candy Vending Machine Project
  - 7-Segment Display Module (Schematic)



DESIGN EXAMPLE

- Candy Vending Machine Project
  - 7-Segment Display Verilog Code

```
21  module seven_segment_display(clk, rst, seg_num_in,
22                               seg_data, seg_common);
23
24  input clk, rst;
25  input [15:0] seg_num_in;
26  output reg [7:0] seg_data;
27  output reg [3:0] seg_common;
28
29  reg clk_1khz; //1 Khz clk for segment common
30  reg [1:0] cnt; //counter for switching
31  reg [15:0] clk50Mhz_period_cnt;
32  reg [3:0] seg_num; //
33  parameter clk_1khz_half = 25000;
34
35  //50 Mhz to 1Khz clk divider
36  always@(posedge clk or posedge rst) begin
37      if(rst) begin
38          clk50Mhz_period_cnt <= 0;
39          clk_1khz <= 0;
40      end
41      else if (clk50Mhz_period_cnt == (clk_1khz_half - 1)) begin
42          clk50Mhz_period_cnt <= 0;
43          clk_1khz <= ~clk_1khz;
44      end
45      else begin
46          clk50Mhz_period_cnt <= clk50Mhz_period_cnt + 1;
47          clk_1khz <= clk_1khz;
48      end
49  end
```



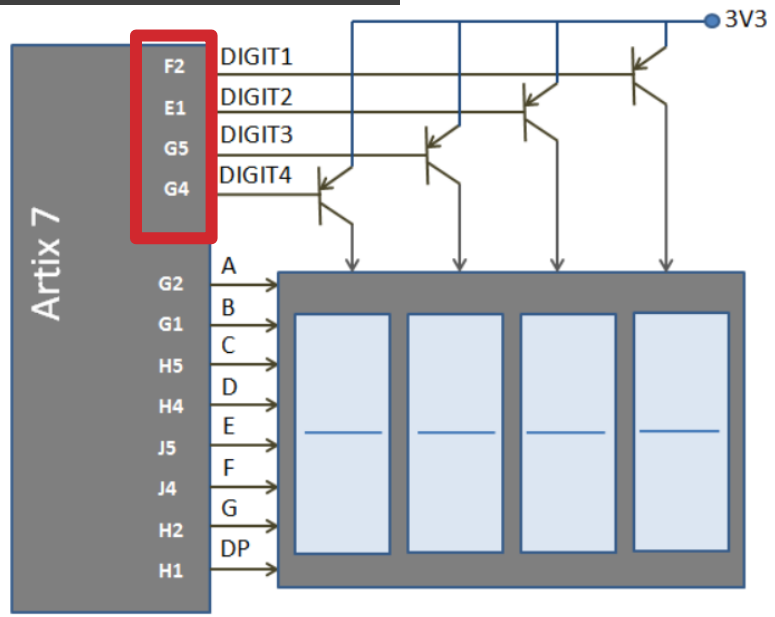


DESIGN EXAMPLE

- Candy Vending Machine Project
  - 7-Segment Display Verilog Code

```
51  //cnt calculation (Counts from 0 to 3)
52  always@(posedge clk_1khz or posedge rst) begin
53      if(rst)
54          cnt <= 0;
55      else if (cnt >= 3)
56          cnt <= 0;
57      else
58          cnt <= cnt + 1;
59  end
```

Count from 0 to 3

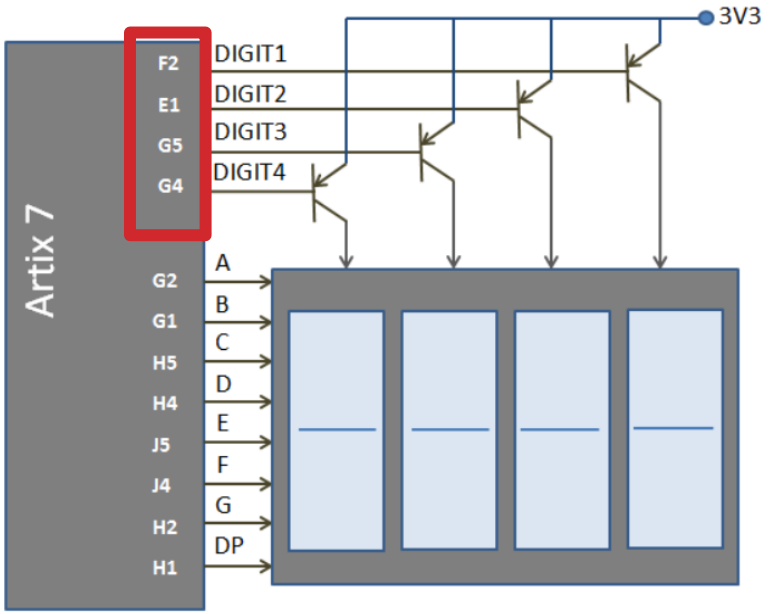


DESIGN EXAMPLE

- Candy Vending Machine Project
  - 7-Segment Display Verilog Code

```
60  //7 segment output
61  always@(posedge clk_1khz or posedge rst) begin
62      if(rst) begin
63          seg_common <= 4'b1000;
64          seg_num <= 4'd5;
65      end
66      else begin
67          case(cnt)
68              2'd0: begin
69                  seg_common <= 4'b1000;
70                  seg_num <= seg_num_in[15:12];
71              end
72              2'd1: begin
73                  seg_common <= 4'b0100;
74                  seg_num <= seg_num_in[11:8];
75              end
76              2'd2: begin
77                  seg_common <= 4'b0010;
78                  seg_num <= seg_num_in[7:4];
79              end
80              2'd3: begin
81                  seg_common <=  4'b0001;
82                  seg_num <= seg_num_in[3:0];
83              end
84              default: begin
85                  seg_common <= 4'b1111;
86                  seg_num  <= 4'd0;
87              end
88          endcase
89      end
90  end
```

Count from 0 to 3

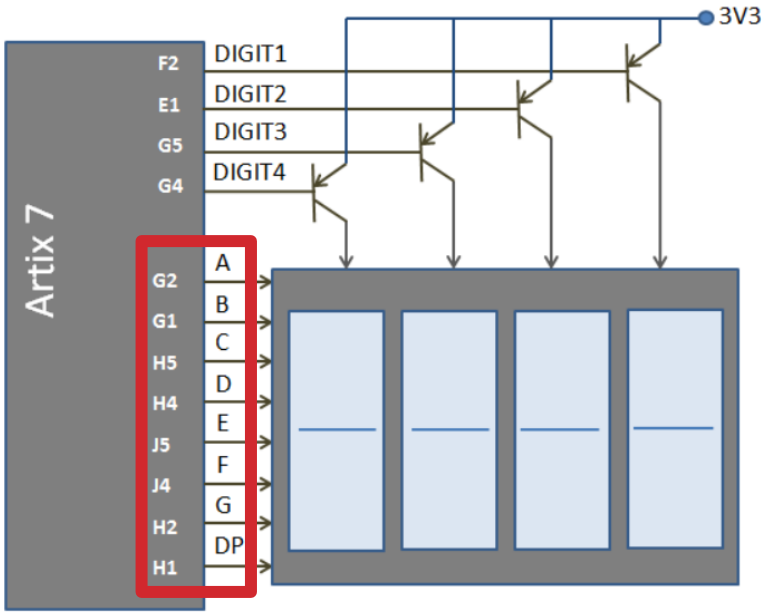


DESIGN EXAMPLE

- Candy Vending Machine Project
  - 7-Segment Display Verilog Code

```
92    //decoding seg_num to seg_data
93    always@(*) begin
94        case(seg_num)
95            4'd0:  seg_data = 8'b00000011;
96            4'd1:  seg_data = 8'b10011111;
97            4'd2:  seg_data = 8'b00100101;
98            4'd3:  seg_data = 8'b00001101;
99            4'd4:  seg_data = 8'b10011001;
100           4'd5:  seg_data = 8'b01001001;
101           4'd6:  seg_data = 8'b01000001;
102           4'd7:  seg_data = 8'b00011111;
103           4'd8:  seg_data = 8'b00000001;
104           4'd9:  seg_data = 8'b00001001;
105           /*4'd10: seg_data = 8'b11101110;
106           4'd11: seg_data = 8'b00111110;
107           4'd12: seg_data = 8'b10011100;
108           4'd13: seg_data = 8'b01111010;
109           4'd14: seg_data = 8'b11011110;
110           4'd15: seg_data = 8'b10001110;  */
111       endcase
112   end
113
114   endmodule
```

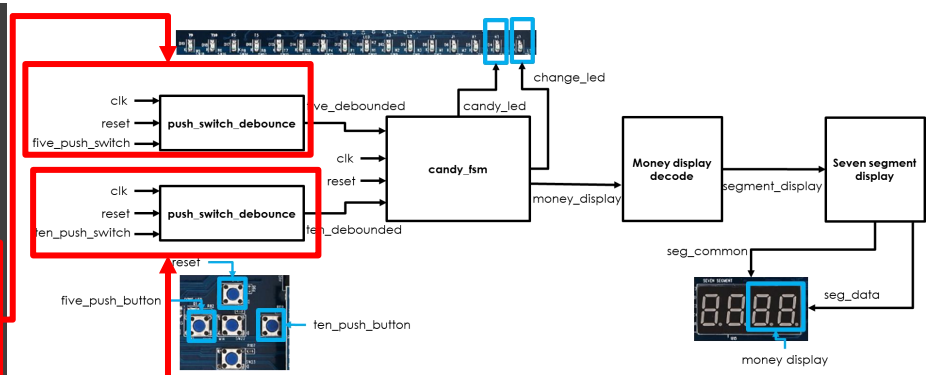
	dp	g	f	e	d	c	b	a	binary	hexadecimal
0	1	1	0	0	0	0	0	0	0b11000000	0xc0
1	1	1	1	1	1	0	0	1	0b11111001	0xf9
2	1	0	1	0	0	1	0	0	0b10100100	0xa4
3	1	0	1	1	0	0	0	0	0b10110000	0xb0
4	1	0	0	1	1	0	0	1	0b10011001	0x99
5	1	0	0	1	0	0	1	0	0b10010010	0x92
6	1	0	0	0	0	0	1	0	0b10000010	0x82
7	1	1	1	1	1	0	0	0	0b11111000	0xf8
8	1	0	0	0	0	0	0	0	0b10000000	0x80
9	1	0	0	1	0	0	0	0	0b10010000	0x90
A	1	0	0	0	1	0	0	0	0b10001000	0x88
b	1	0	0	0	0	0	1	1	0b10000011	0x83
C	1	1	0	0	0	1	1	0	0b11000110	0xc6
d	1	0	1	0	0	0	0	1	0b10100001	0xa1
E	1	0	0	0	0	1	1	0	0b10000110	0x86
F	1	0	0	0	1	1	1	0	0b10001110	0x8e
OFF	1	1	1	1	1	1	1	1	0b11111111	0xff



DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy Vending Machine Top Module Verilog Code

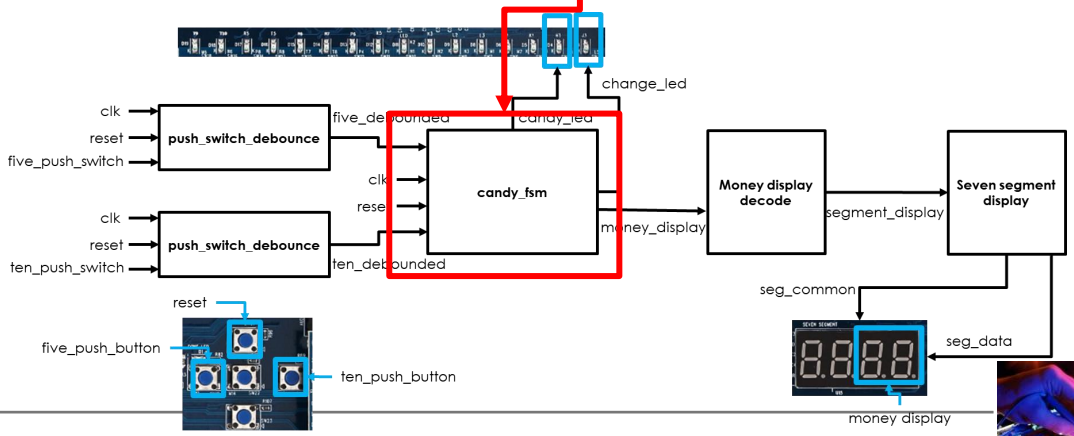
```
21  module candy_vending_machine_TOP(clk, reset, five_push_switch,
22                                   ten_push_switch, candy_led, change_led,
23                                   seg_data, seg_common);
24  input clk, reset, five_push_switch, ten_push_switch;
25  output candy_led, change_led;
26  output [7:0] seg_data;
27  output [3:0] seg_common;
28  //debounce push switches
29  wire five_debounced, ten_debounced;
30  //five
31  push_switch_debounce push_switch_debounce_five (.clk(clk),
32                                                  .rst(reset),
33                                                  .sw_in(five_push_switch),
34                                                  .sw_out(five_debounced)
35                                                  );
36  //ten
37  push_switch_debounce push_switch_debounce_ten (.clk(clk),
38                                                 .rst(reset),
39                                                 .sw_in(ten_push_switch),
40                                                 .sw_out(ten_debounced)
41                                                 );
```



DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy Vending Machine Top Module Verilog Code

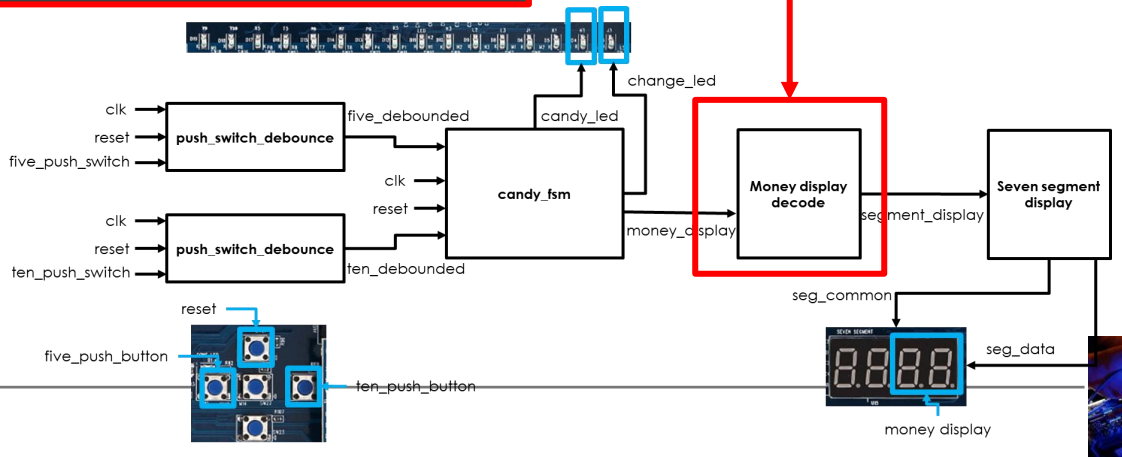
```
42  //candy vending machine fsm
43  wire [1:0] money_display;
44  candy_fsm candy_controller (.clk(clk),
45                              .reset(reset),
46                              .five(five_debounced),
47                              .ten(ten_debounced),
48                              .candy_out(candy_led),
49                              .change_out(change_led),
50                              .money_display(money_display)
51                              );
```



DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy Vending Machine Top Module Verilog Code

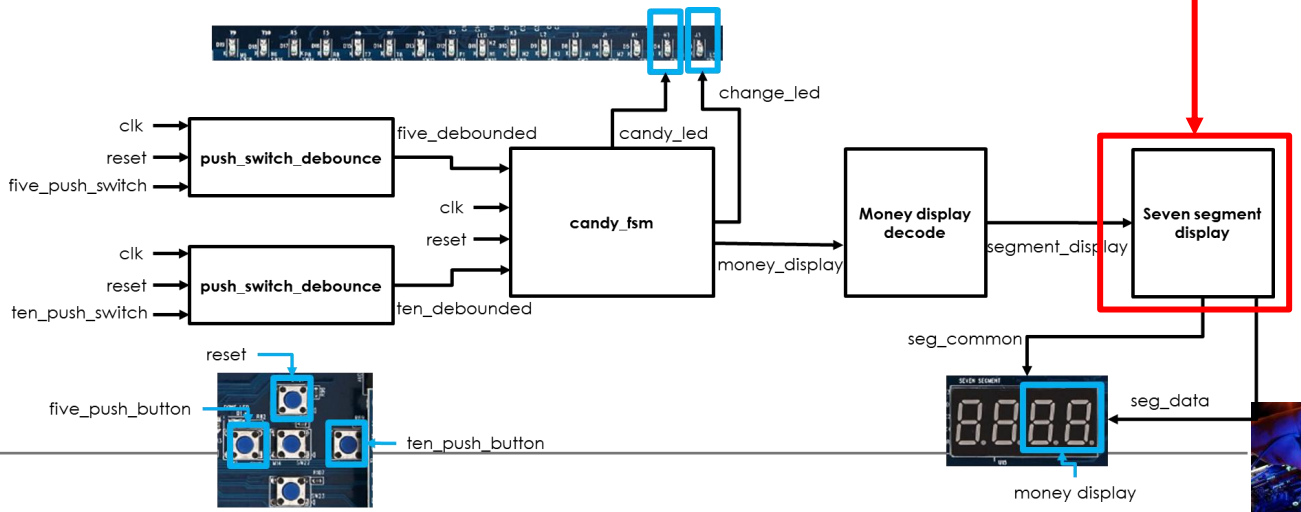
```
52  //money display decode
53  reg [15:0] segment_display;
54  always@(*) begin
55      case (money_display)
56          2'b00: segment_display = 16'h0000;
57          2'b01: segment_display = 16'h0005;
58          2'b10: segment_display = 16'h0010;
59          2'b11: segment_display = 16'h0015;
60      endcase
61  end
```



DESIGN EXAMPLE

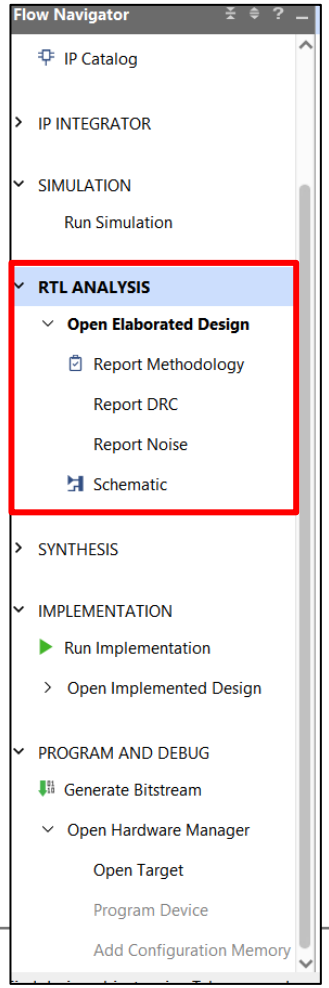
- Candy Vending Machine Project
  - Candy Vending Machine Top Module Verilog Code

```
62    //Seven Segment Display Controller
63    seven_segment_display display1 (.clk(clk),
64                                    .rst(reset),
65                                    .seg_num_in(segment_display),
66                                    .seg_data(seg_data),
67                                    .seg_common(seg_common)
68                                    );
69    endmodule
```



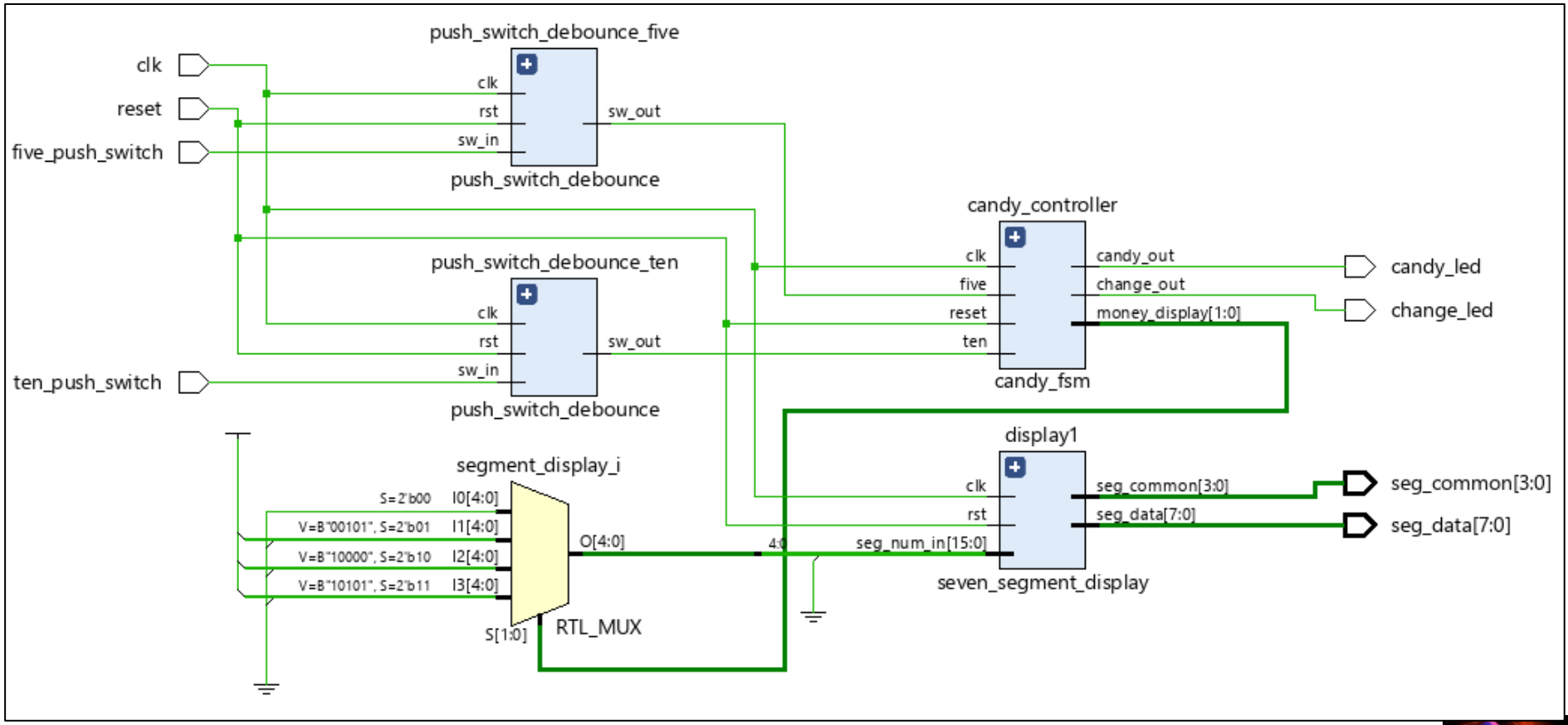
DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy Vending Machine Top Module Elaborated Schematic



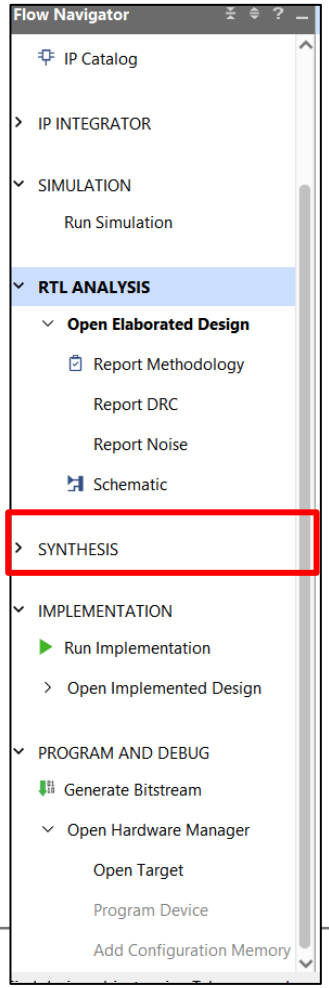
DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy Vending Machine Top Module Elaborated Schematic



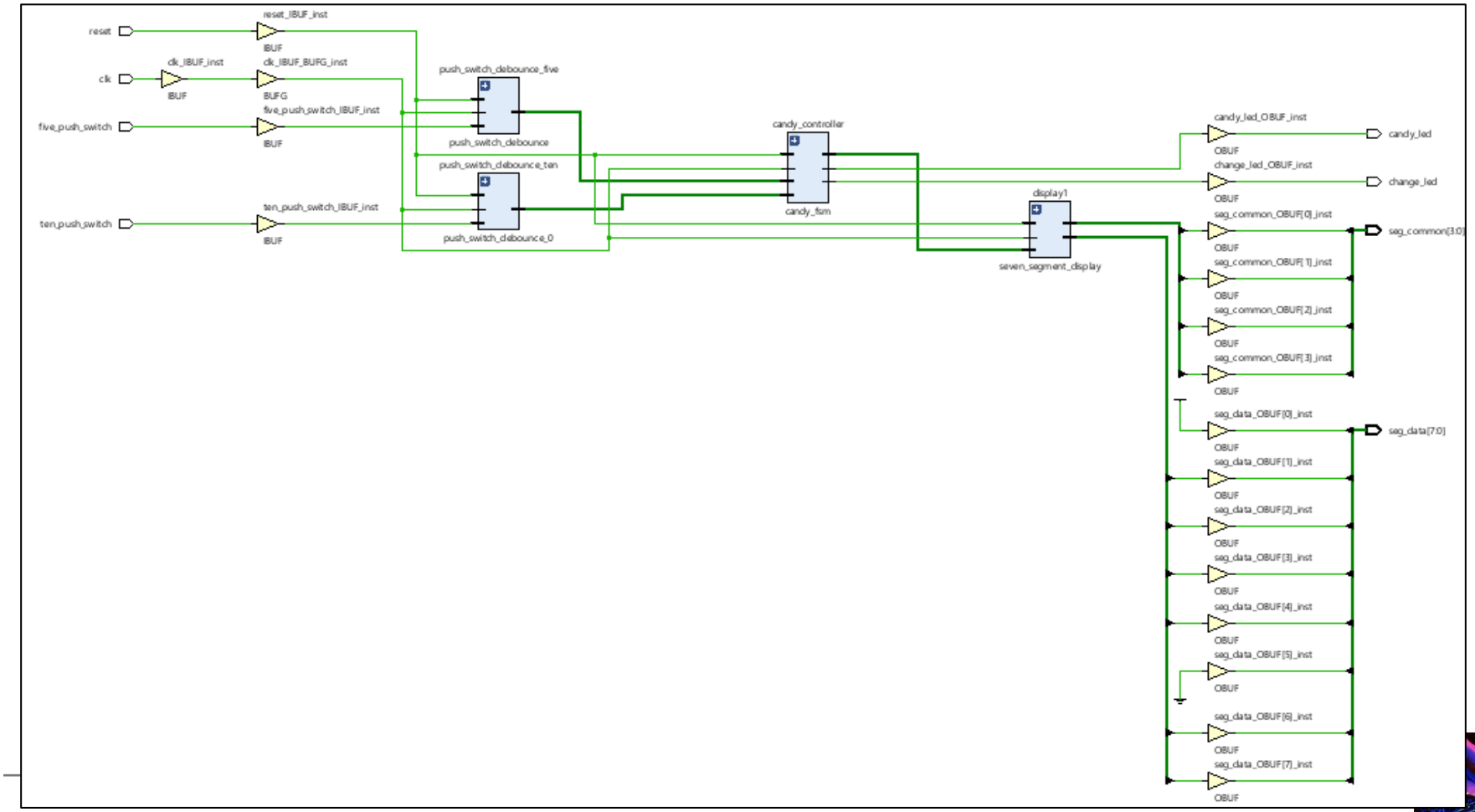
DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy Vending Machine Top Module Synthesis Schematic



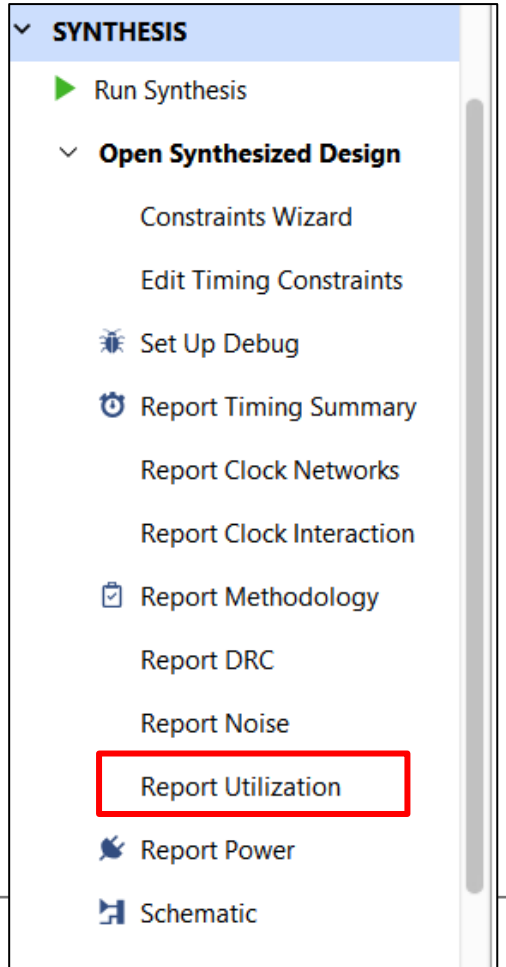
DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy Vending Machine Top Module Synthesis Schematic



DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy Vending Machine Top Module Synthesis Results



DESIGN EXAMPLE

- Candy Vending Machine Project
  - Candy Vending Machine Top Module Synthesis Results

Reports | Design Runs | **Utilization** ×

Hierarchy

Name	Slice LUTs (20800)	Slice Registers (41600)	Bonded IOB (170)	BUFGCTRL (32)
N candy_vending_machine_TOP	33	33	18	1
candy_controller (candy_fsm)	5	4	0	0
display1 (seven_segment_display)	28	25	0	0
push_switch_debounce_five (push_switch_debounce)	0	2	0	0
push_switch_debounce_ten (push_switch_debounce_0)	0	2	0	0

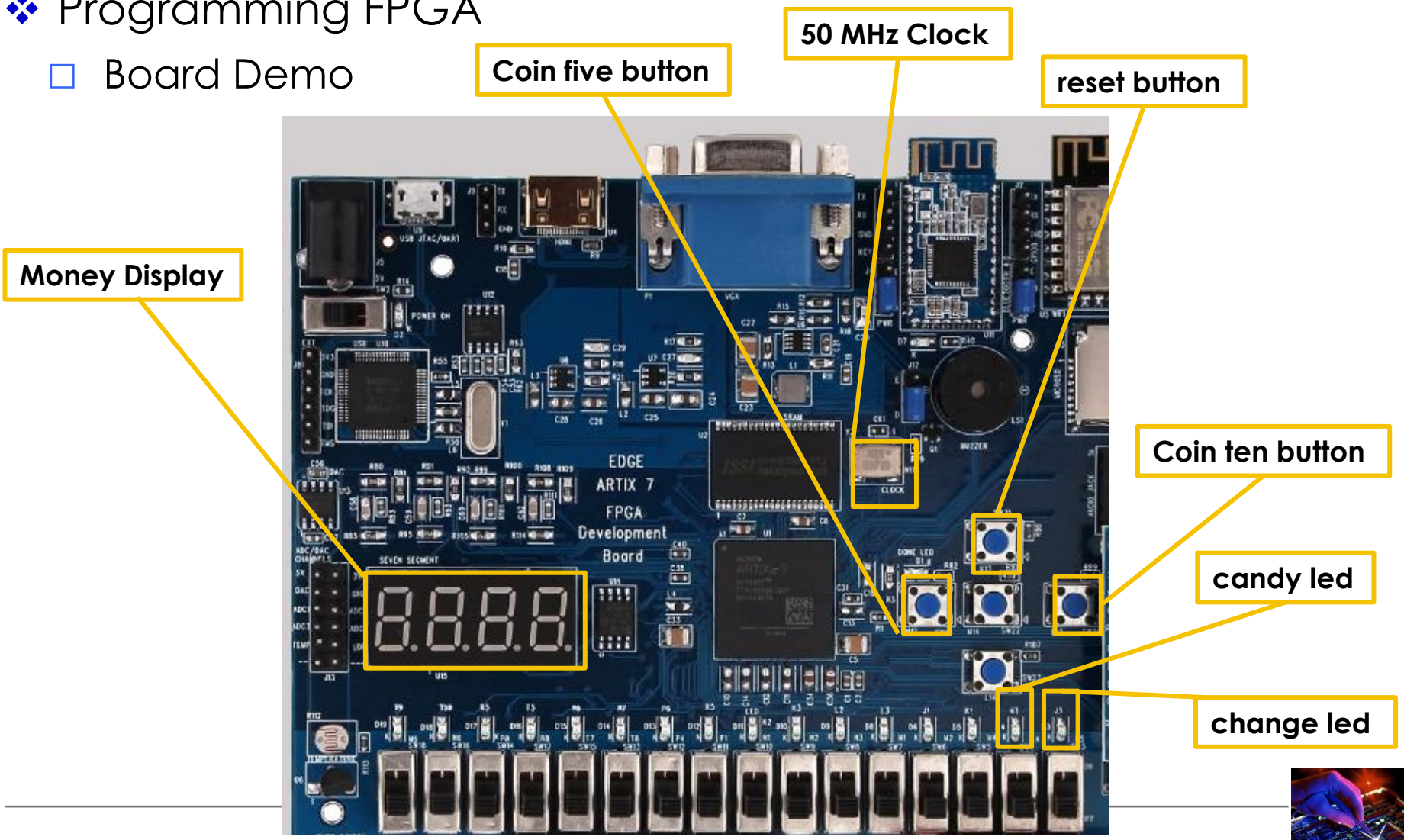
Reports | Design Runs | **Utilization** ×

Hierarchy

Name	Slice LUTs (20800)	Slice Registers (41600)	Bonded IOB (170)	BUFGCTRL (32)
N candy_vending_machine_TOP	0.16%	0.08%	10.59%	3.13%
candy_controller (candy_fsm)	0.02%	<0.01%	0.00%	0.00%
display1 (seven_segment_display)	0.13%	0.06%	0.00%	0.00%
push_switch_debounce_five (push_switch_debounce)	0.00%	<0.01%	0.00%	0.00%
push_switch_debounce_ten (push_switch_debounce_0)	0.00%	<0.01%	0.00%	0.00%

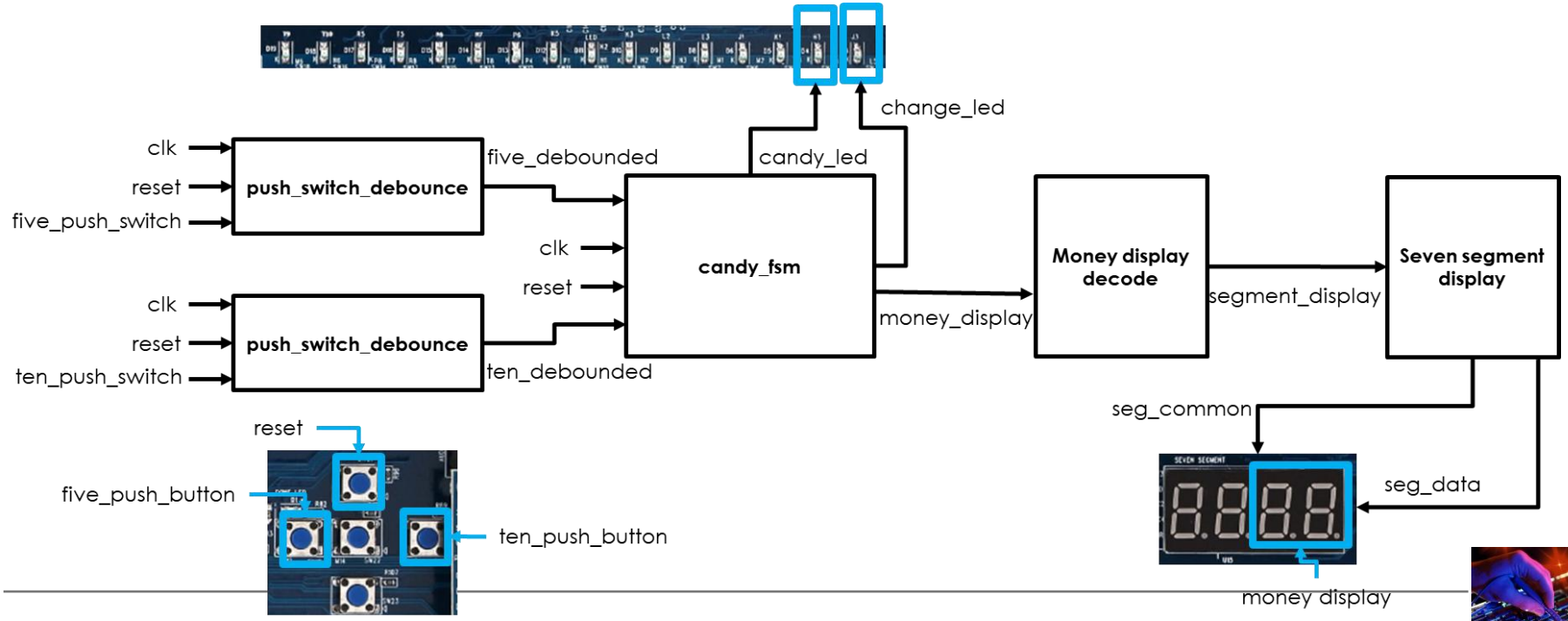
DESIGN EXAMPLE

- Programming FPGA
  - Board Demo



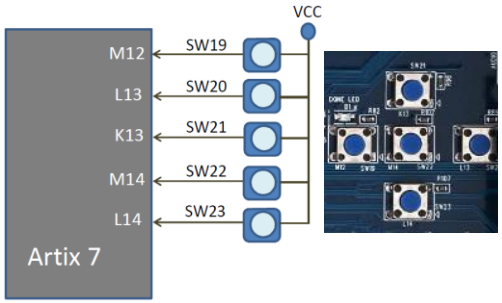
DESIGN EXAMPLE

- Implement Candy Vending Machine on Edge Artix 7 FPGA Board
  - Input: clk, reset, five_push_switch, ten_push_switch
  - Output: candy_led, change_led
  - Output: [7:0] seg_data
  - Output: [7:0] seg_common

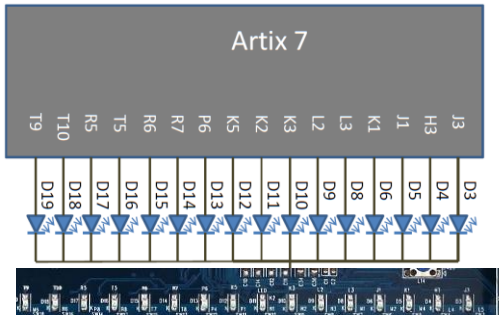


DESIGN EXAMPLE

- Vivado project
  - FPGA programming (pin mapping)



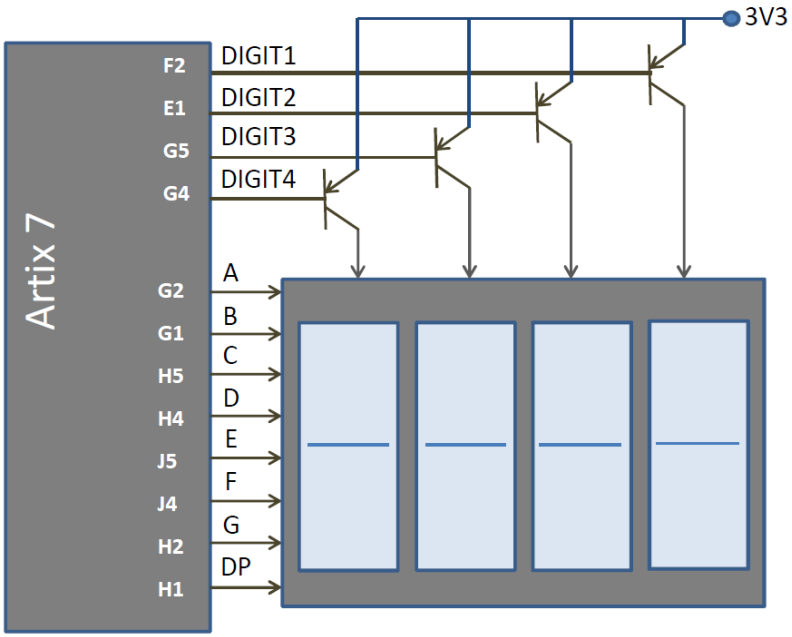

```
# Push Button
set_property -dict {PACKAGE_PIN K13 IOSTANDARD LVCMOS33 PULLDOWN true} [get_ports {pb[0]}]; #Button-top
set_property -dict {PACKAGE_PIN L14 IOSTANDARD LVCMOS33 PULLDOWN true} [get_ports {pb[1]}]; #Button-bottom
set_property -dict {PACKAGE_PIN M12 IOSTANDARD LVCMOS33 PULLDOWN true} [get_ports {pb[2]}]; #Button-left
set_property -dict {PACKAGE_PIN L13 IOSTANDARD LVCMOS33 PULLDOWN true} [get_ports {pb[3]}]; #Button-right
set_property -dict {PACKAGE_PIN M14 IOSTANDARD LVCMOS33 PULLDOWN true} [get_ports {pb[4]}]; #Button-center
```




```
# LEDs
set_property -dict { PACKAGE_PIN J3    IOSTANDARD LVCMOS33 } [get_ports { led[0] }];#LSB
set_property -dict { PACKAGE_PIN H3    IOSTANDARD LVCMOS33 } [get_ports { led[1] }];
set_property -dict { PACKAGE_PIN J1    IOSTANDARD LVCMOS33 } [get_ports { led[2] }];
set_property -dict { PACKAGE_PIN K1    IOSTANDARD LVCMOS33 } [get_ports { led[3] }];
set_property -dict { PACKAGE_PIN L3    IOSTANDARD LVCMOS33 } [get_ports { led[4] }];
set_property -dict { PACKAGE_PIN L2    IOSTANDARD LVCMOS33 } [get_ports { led[5] }];
set_property -dict { PACKAGE_PIN K3    IOSTANDARD LVCMOS33 } [get_ports { led[6] }];
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { led[7] }];
set_property -dict { PACKAGE_PIN K5    IOSTANDARD LVCMOS33 } [get_ports { led[8] }];
set_property -dict { PACKAGE_PIN P6    IOSTANDARD LVCMOS33 } [get_ports { led[9] }];
set_property -dict { PACKAGE_PIN R7    IOSTANDARD LVCMOS33 } [get_ports { led[10] }];
set_property -dict { PACKAGE_PIN R6    IOSTANDARD LVCMOS33 } [get_ports { led[11] }];
set_property -dict { PACKAGE_PIN T5    IOSTANDARD LVCMOS33 } [get_ports { led[12] }];
set_property -dict { PACKAGE_PIN R5    IOSTANDARD LVCMOS33 } [get_ports { led[13] }];
set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { led[14] }];
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { led[15] }];#MSB
```

DESIGN EXAMPLE

- Vivado project
  - FPGA programming (pin mapping)



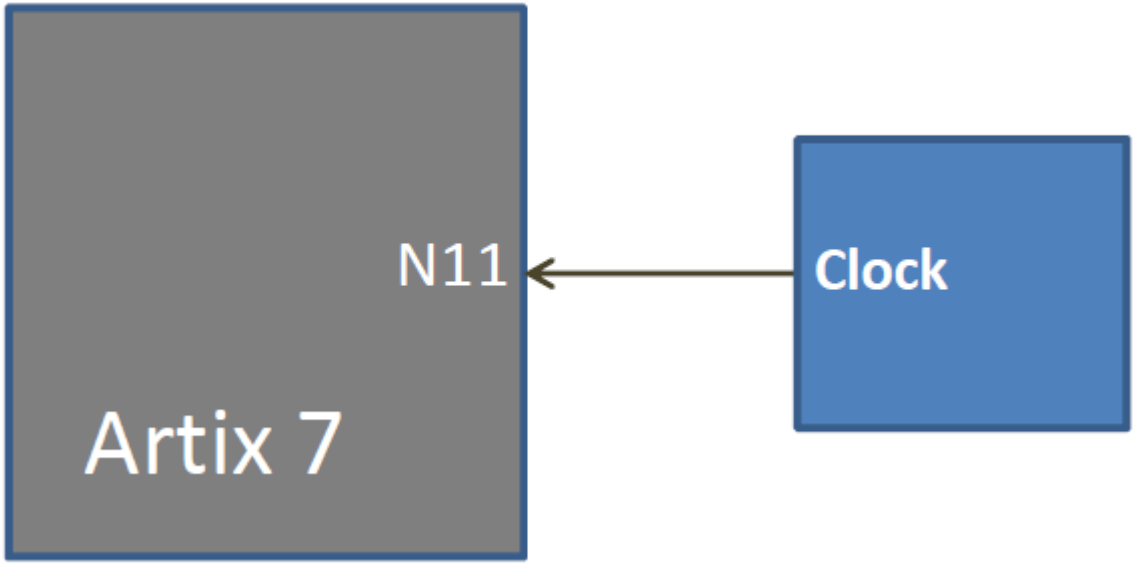

```
#7 segment display
set_property -dict { PACKAGE_PIN F2    IOSTANDARD LVCMOS33 } [get_ports {digit[0]}]; #LSB
set_property -dict { PACKAGE_PIN E1    IOSTANDARD LVCMOS33 } [get_ports {digit[1]}];
set_property -dict { PACKAGE_PIN G5    IOSTANDARD LVCMOS33 } [get_ports {digit[2]}];
set_property -dict { PACKAGE_PIN G4    IOSTANDARD LVCMOS33 } [get_ports {digit[3]}]; #MSB

set_property -dict { PACKAGE_PIN G2    IOSTANDARD LVCMOS33 } [get_ports {Seven_Seg[0]}];#A
set_property -dict { PACKAGE_PIN G1    IOSTANDARD LVCMOS33 } [get_ports {Seven_Seg[1]}];#B
set_property -dict { PACKAGE_PIN H5    IOSTANDARD LVCMOS33 } [get_ports {Seven_Seg[2]}];#C
set_property -dict { PACKAGE_PIN H4    IOSTANDARD LVCMOS33 } [get_ports {Seven_Seg[3]}];#D
set_property -dict { PACKAGE_PIN J5    IOSTANDARD LVCMOS33 } [get_ports {Seven_Seg[4]}];#E
set_property -dict { PACKAGE_PIN J4    IOSTANDARD LVCMOS33 } [get_ports {Seven_Seg[5]}];#F
set_property -dict { PACKAGE_PIN H2    IOSTANDARD LVCMOS33 } [get_ports {Seven_Seg[6]}];#G
set_property -dict { PACKAGE_PIN H1    IOSTANDARD LVCMOS33 } [get_ports {Seven_Seg[7]}];#DP
```



DESIGN EXAMPLE

- Vivado project
  - FPGA programming (pin mapping)






```
# Clock signal
set_property -dict { PACKAGE_PIN N11    IOSTANDARD LVCMOS33 } [get_ports { clk }];
```

DESIGN EXAMPLE

- Vivado project
  - FPGA programming (mapping) – Create a constraint file
    - candy_vending_machine_pin_mapping.xdc

candy_vending_machine_pin_mapping.xdc

```
#Clock signal
set_property -dict { PACKAGE_PIN N11    IOSTANDARD LVCMOS33 } [get_ports { clk }];

# Push Button
set_property -dict {PACKAGE_PIN K13 IOSTANDARD LVCMOS33 PULLDOWN true} [get_ports {reset}]; #Button-top
set_property -dict {PACKAGE_PIN M12 IOSTANDARD LVCMOS33 PULLDOWN true} [get_ports {five_push_switch}]; #Button-left
set_property -dict {PACKAGE_PIN L13 IOSTANDARD LVCMOS33 PULLDOWN true} [get_ports {ten_push_switch}]; #Button-right

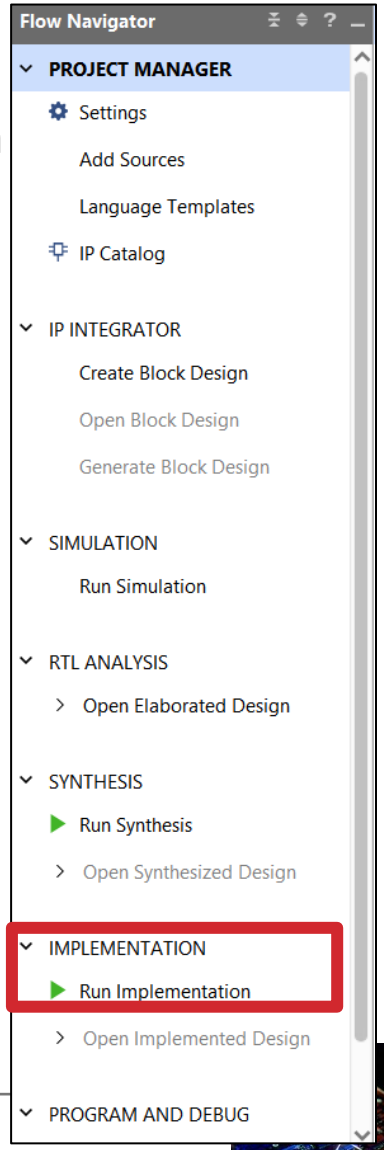
# LEDs
set_property -dict { PACKAGE_PIN J3    IOSTANDARD LVCMOS33 } [get_ports { change_led }];#LSB
set_property -dict { PACKAGE_PIN H3    IOSTANDARD LVCMOS33 } [get_ports { candy_led }];


#7 segment display
set_property -dict { PACKAGE_PIN F2    IOSTANDARD LVCMOS33 } [get_ports {seg_common[0]}]; #LSB
set_property -dict { PACKAGE_PIN E1    IOSTANDARD LVCMOS33 } [get_ports {seg_common[1]}];
set_property -dict { PACKAGE_PIN G5    IOSTANDARD LVCMOS33 } [get_ports {seg_common[2]}];
set_property -dict { PACKAGE_PIN G4    IOSTANDARD LVCMOS33 } [get_ports {seg_common[3]}]; #MSB

set_property -dict { PACKAGE_PIN G2    IOSTANDARD LVCMOS33 } [get_ports {seg_data[7]}];#A
set_property -dict { PACKAGE_PIN G1    IOSTANDARD LVCMOS33 } [get_ports {seg_data[6]}];#B
set_property -dict { PACKAGE_PIN H5    IOSTANDARD LVCMOS33 } [get_ports {seg_data[5]}];#C
set_property -dict { PACKAGE_PIN H4    IOSTANDARD LVCMOS33 } [get_ports {seg_data[4]}];#D
set_property -dict { PACKAGE_PIN J5    IOSTANDARD LVCMOS33 } [get_ports {seg_data[3]}];#E
set_property -dict { PACKAGE_PIN J4    IOSTANDARD LVCMOS33 } [get_ports {seg_data[2]}];#F
set_property -dict { PACKAGE_PIN H2    IOSTANDARD LVCMOS33 } [get_ports {seg_data[1]}];#G
set_property -dict { PACKAGE_PIN H1    IOSTANDARD LVCMOS33 } [get_ports {seg_data[0]}];#DP
```

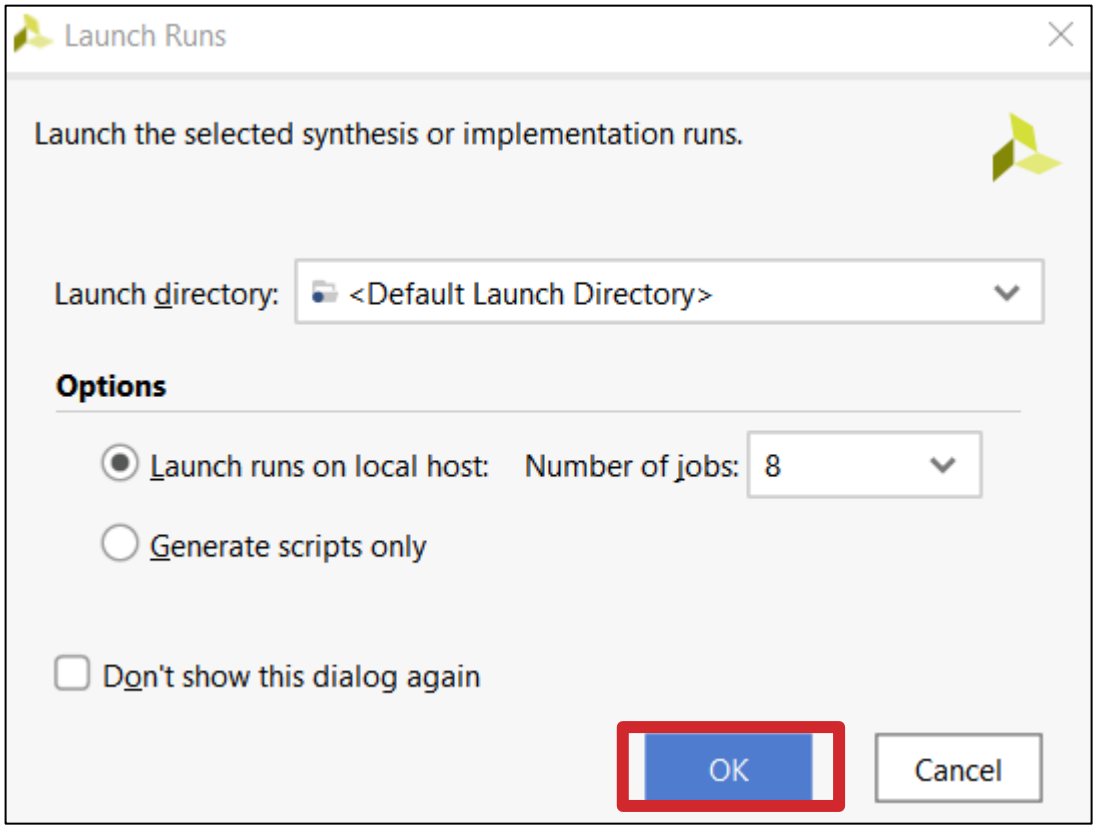
DESIGN EXAMPLE

- Vivado project
  - FPGA programming (mapping) – Implement Design



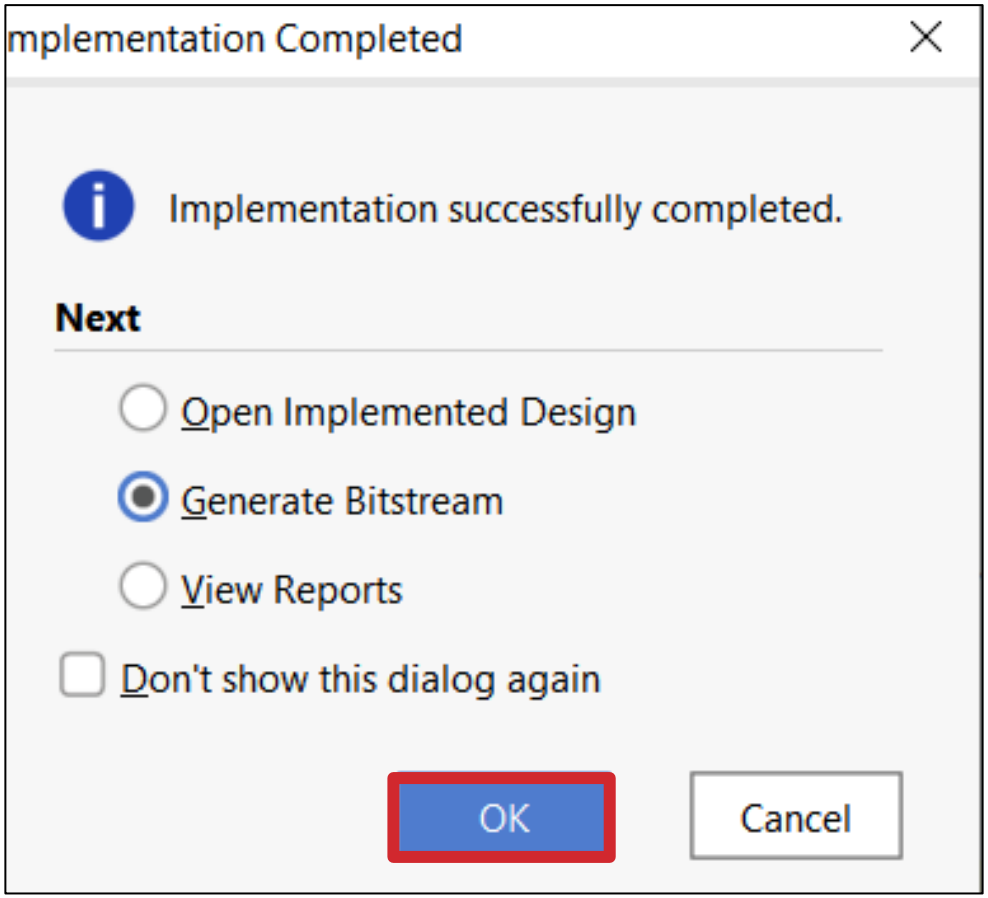
DESIGN EXAMPLE

- Vivado project
  - FPGA programming (mapping) – Implement Design



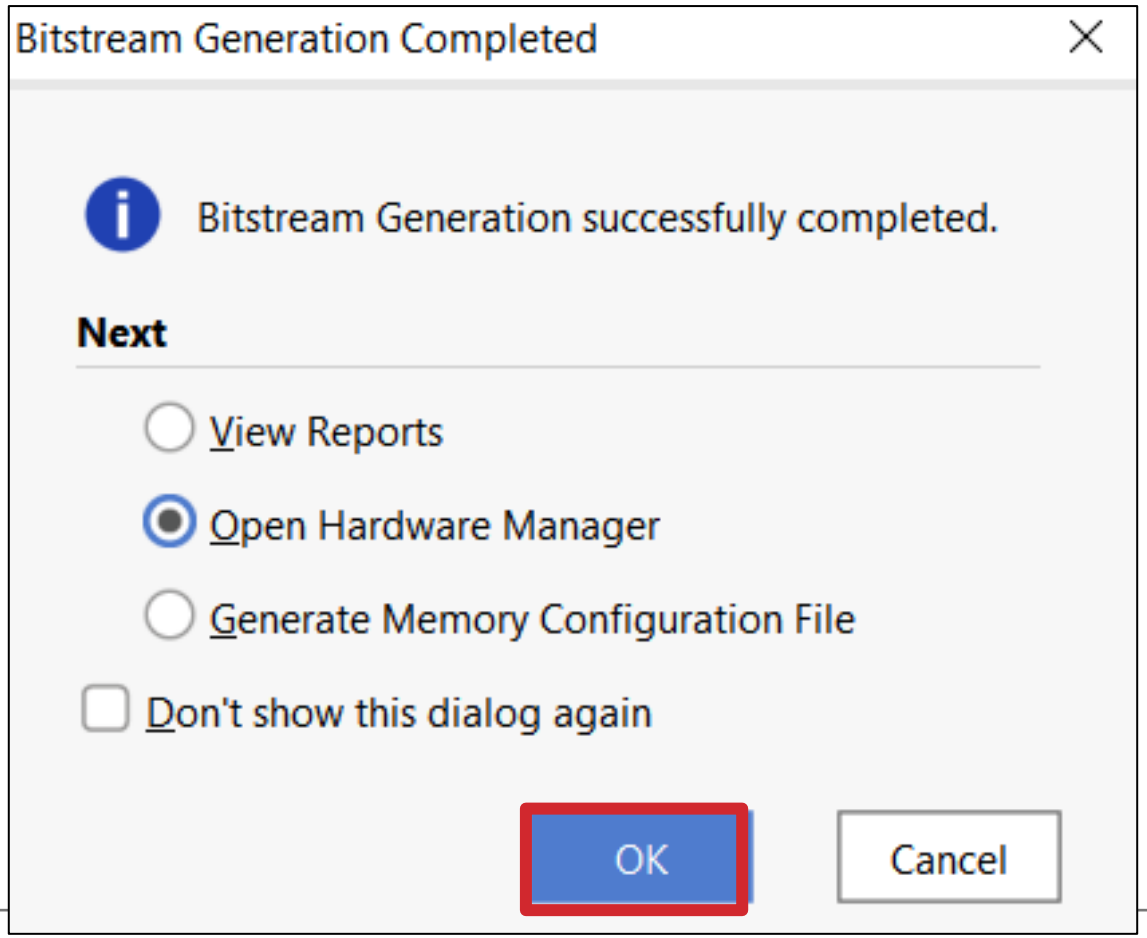
DESIGN EXAMPLE

- Vivado project
  - FPGA programming (mapping) – Generate Programming File



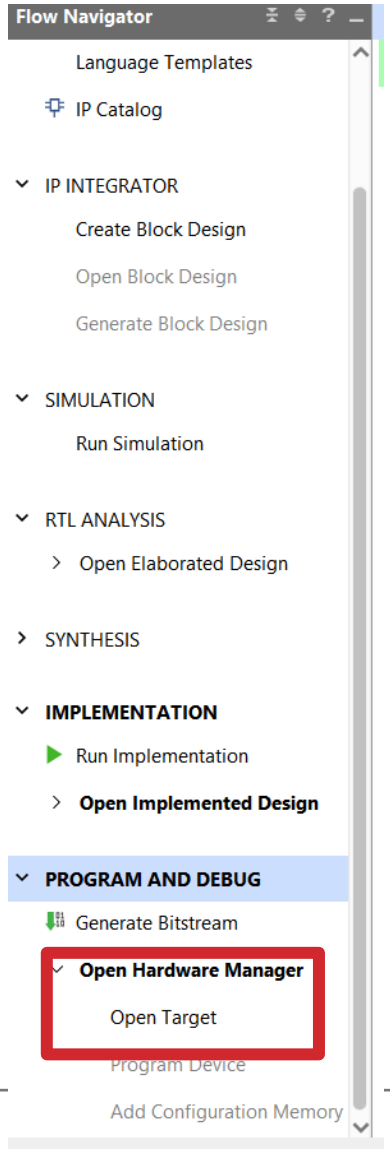
DESIGN EXAMPLE

- Vivado project
  - FPGA programming (mapping) – Generate Programming File



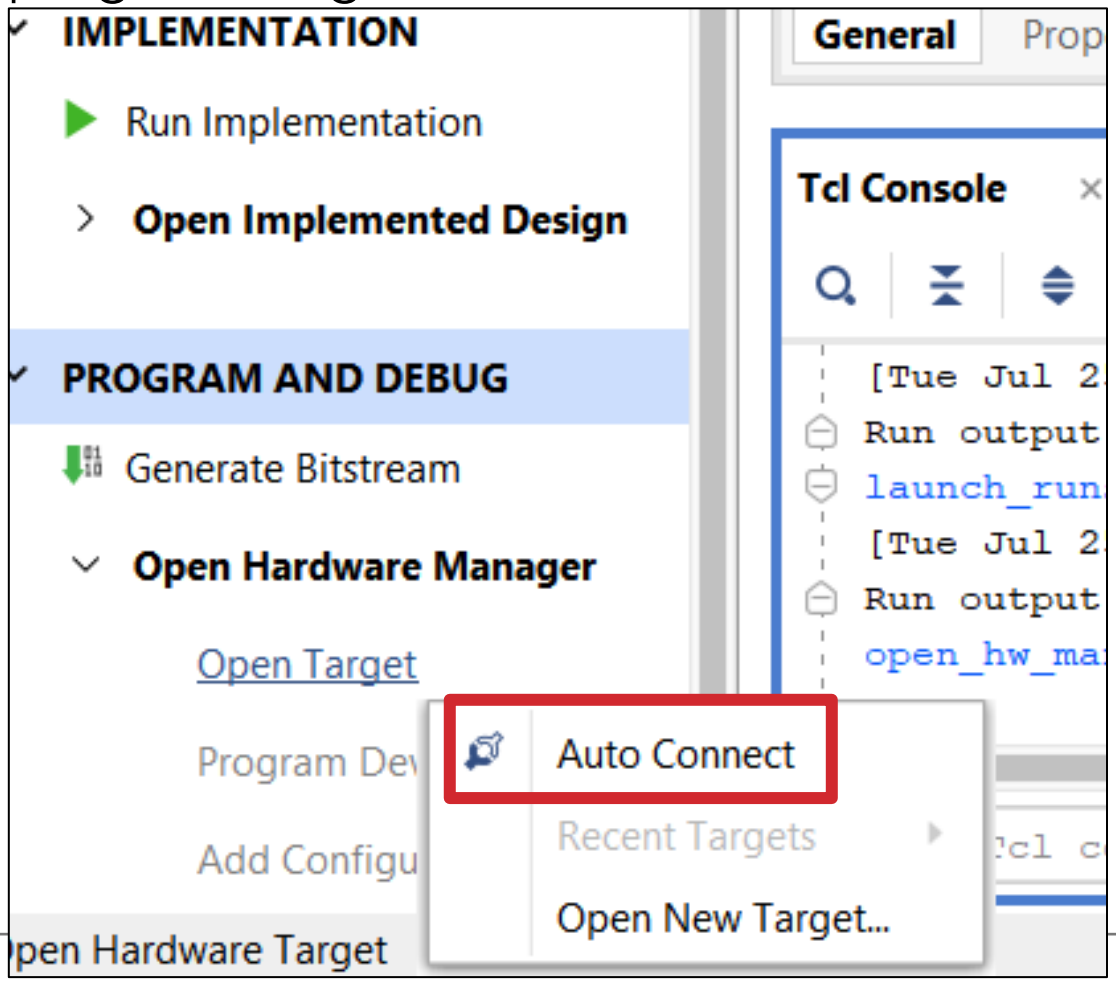
DESIGN EXAMPLE

- Vivado project
  - FPGA programming
    - Connect the FPGA board to the computer and click on Open Target



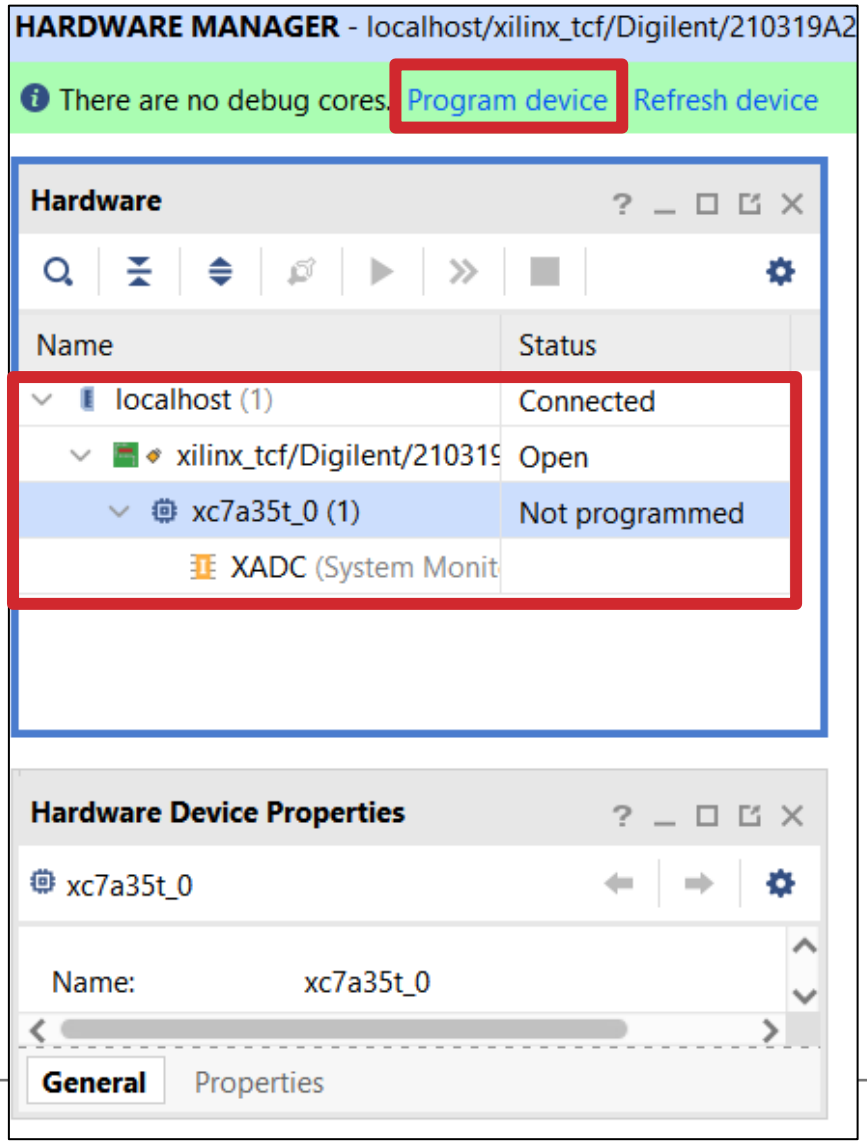
DESIGN EXAMPLE

- Vivado project
  - FPGA programming



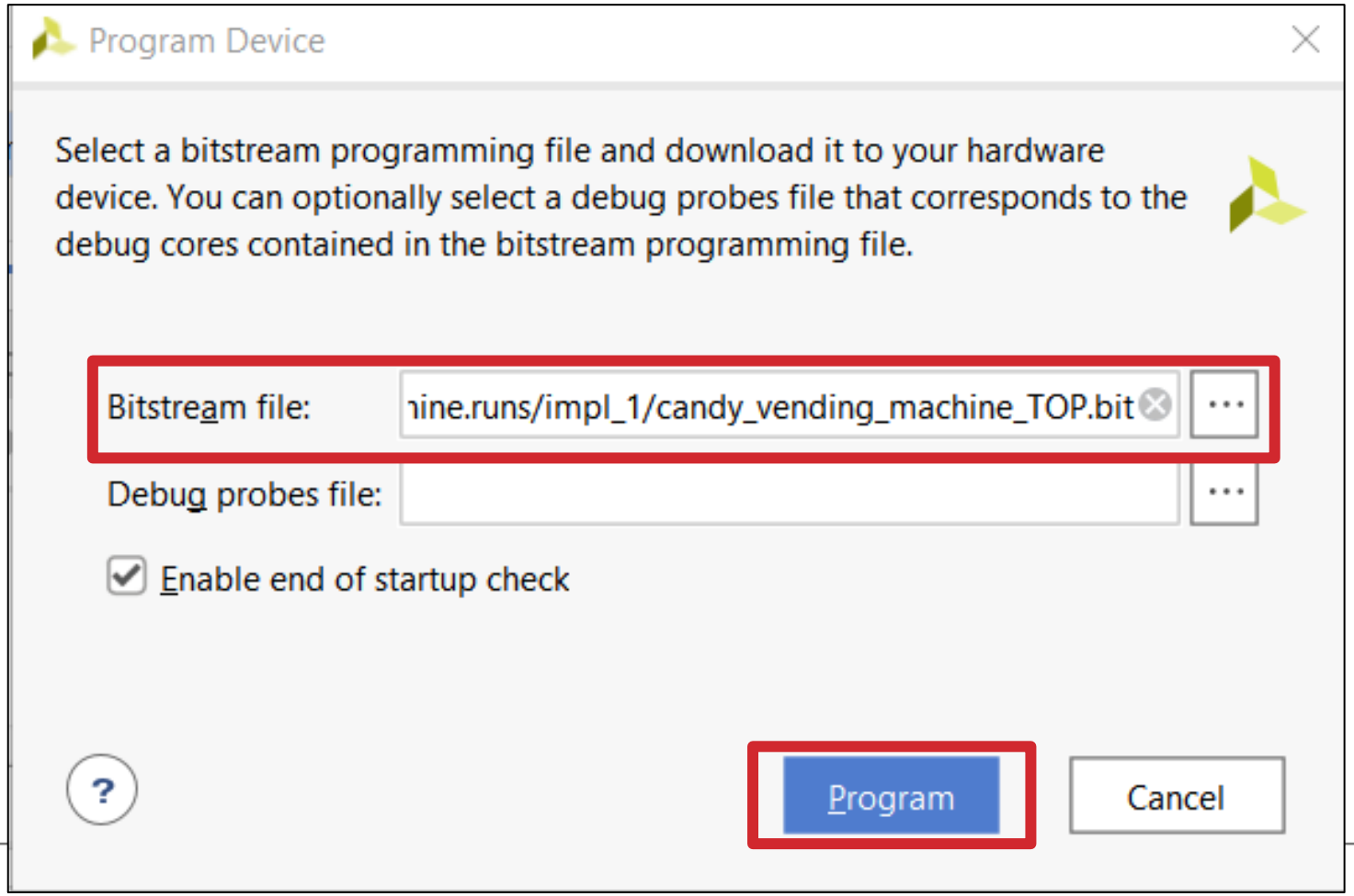
DESIGN EXAMPLE

- Vivado project
  - FPGA programming



DESIGN EXAMPLE

- Vivado project
  - FPGA programming



DESIGN EXAMPLE

- Programming FPGA
  - Board Demo

